
Firmware Design

A Ground-Up Textbook for the Embedded Engineer

From the GCC toolchain and linker sections,
through STM32 peripherals, FreeRTOS and HAL/LL,
to power MOSFETs, full-bridges and serial buses.

A custom course companion built from the
Firmware Design syllabus

Written for a third-year Computer Engineering student

May 2026

Typeset in L^AT_EX with TikZ, circuitikz and pgfplots.

Preface

This book exists because the firmware-design syllabus is a deceptively short list of bullet points that hides an enormous amount of *connected* knowledge. Each line — “review of executable code generation”, “meaning of `const` and `volatile`”, “full-bridges and the shoot-through risk” — is the tip of an iceberg. The goal here is to melt the iceberg: to take every topic and explain not just *what* it is, but *why* it exists, *how* it works at the level of registers, bytes and transistors, and *when* it bites you in practice.

Who this is for. You. A third-year Computer Engineering student who can already write C, knows what a pointer is, has seen a transistor and an op-amp at least once, and has survived a digital-logic course. You do *not* need prior embedded experience. Where a topic leans on electronics, the relevant ideas are re-derived from Ohm’s law and Kirchhoff’s laws rather than assumed.

How this book is organised. The material is grouped into seven parts that follow the natural arc of an embedded project:

Part I The landscape — how embedded differs from the PC world, and the interrupt mechanism that underpins everything.

Part II
From source code to a binary that boots — the GCC model, cross-compilation, ELF sections, variable allocation, and the on-chip debug unit.

Part III
Disciplined C — dynamic memory, the constructs we ban, DMA-safe buffers, `const/volatile`, and file hygiene.

Part IV
Peripherals — the memory-mapped register model, the STM32CubeIDE workflow, HAL versus LL, and timers.

Part V
Concurrency and numerics — FreeRTOS, and fixed- versus floating-point arithmetic.

Part VI
Sensors and serial buses — the NTC thermistor, the MEMS accelerometer, and the I²C/SPI protocols.

Part VII
Power electronics for firmware engineers — load switches, high-side drivers, power dissipation, and full-bridges.

How to read it. Read Part I and Part II first; they build the mental model everything else relies on. After that the parts are fairly independent. Throughout you will find three kinds of coloured callout boxes:

Key Idea

The single sentence you should remember if you forget everything else on the page.

Watch Out

A mistake that is easy to make and painful to debug. Read these twice.

Note

A useful aside, historical context, or a pointer to a deeper rabbit hole.

Worked examples are numbered per chapter, and most chapters end with exercises (solutions immediately follow each one, so cover them if you want to try first).

A note on hardware. Concrete examples target the **STM32** family (ARM Cortex-M) because that is what the course uses, but the principles are vendor-neutral. Where a number is specific to a particular chip it is flagged as such.

Now: power on, attach the debugger, and let's go.

Contents

Preface	ii
Notation and Conventions	x
I The Embedded Landscape	1
1 Classic Computers versus Embedded Systems	2
1.1 Two design philosophies	2
1.2 Hardware differences	2
1.2.1 Processing core	2
1.2.2 Memory hierarchy	3
1.2.3 Memory protection and address spaces	3
1.2.4 Peripherals are first-class citizens	4
1.2.5 Power, cost, and physical envelope	4
1.3 Software differences	4
1.3.1 There may be no operating system	4
1.3.2 The code is cross-compiled	4
1.3.3 No dynamic loading, fixed memory layout	4
1.3.4 Real-time constraints are part of correctness	5
1.3.5 Reliability and longevity	5
1.4 The spectrum of devices	5
1.5 Why every later chapter inherits from this one	6
2 Microprocessor Interrupts	7
2.1 The problem: how do you wait for the world?	7
2.2 The mechanism, step by step	7
2.3 The vector table	8
2.4 Latency, tail-chaining, and why it matters	9
2.5 Priorities, preemption, and nesting	9
2.6 The hard part: reentrancy and shared data	9
2.7 Rules for writing good ISRs	10

II	From Source Code to a Booting Binary	12
3	How an Executable is Built — the GCC Model	13
3.1	The four stages	13
3.1.1	Stage 1 — Preprocessing (<code>cpp</code>)	13
3.1.2	Stage 2 — Compilation (<code>cc1</code>)	14
3.1.3	Stage 3 — Assembly (<code>as</code>)	14
3.1.4	Stage 4 — Linking (<code>ld</code>)	14
3.2	Reading the binaries you produced: <code>objdump</code> and friends	14
4	Cross-Compilation for Embedded Targets	17
4.1	Why you cannot just compile on the target	17
4.2	The three machines and the GNU triplet	17
4.3	What changes versus native <code>gcc</code>	18
4.3.1	You must tell it exactly which core	18
4.3.2	There is no hosted environment	18
4.3.3	The output must be turned into a flashable image	19
5	Sections in Object Files and Executables	21
5.1	Why a file needs sections at all	21
5.2	The crucial subtlety: VMA versus LMA	21
5.3	The linker script	22
5.4	What the startup code actually does	23
5.5	Custom sections via attributes	23
6	Where Variables Live: Static versus Local Allocation	25
6.1	Storage duration decides the home	25
6.2	The stack and a function's stack frame	25
6.3	The effect of optimisation: the same code, twice	26
6.3.1	Without optimisation (<code>-O0</code>)	26
6.3.2	With optimisation (<code>-O2</code>)	26
6.4	RISC versus CISC variations	27
6.5	Stack overflow: the no-MMU hazard	27
7	The On-Chip Debug Unit	29
7.1	Why hardware debug exists	29
7.2	The pieces (the CoreSight family)	29
7.3	Hardware versus software breakpoints	30
7.4	Watchpoints: stopping on data, not code	30
7.5	What all this lets you do	30

7.6	Trace and “printf” without a UART	31
III Disciplined C for Embedded Systems		32
8	Dynamic Memory Allocation and Why We Distrust It	33
8.1	What <code>malloc</code> actually does	33
8.2	Where the heap lives, and <code>_sbrk</code>	33
8.3	The four reasons we distrust the heap	34
8.3.1	1. Fragmentation	34
8.3.2	2. Non-deterministic timing	34
8.3.3	3. Run-time failure with nowhere to go	34
8.3.4	4. Reentrancy / thread-safety	35
8.4	What we do instead	35
9	C Constructs to Avoid in Embedded	37
9.1	Always use braces, even for one-line blocks	37
9.2	No unlimited recursion	38
9.3	No <code>malloc/free</code>	38
9.4	Prefer indexes to pointers for array traversal	38
9.5	The other classics worth internalising	39
10	Dedicated Memory for I/O Buffers and DMA	41
10.1	What DMA is, briefly	41
10.2	Trap 1 — the DMA must be able to reach the memory	41
10.3	Trap 2 — cache coherency	41
10.4	Trap 3 — alignment	42
10.5	Putting it together: a dedicated, safe buffer	42
11	The <code>const</code> and <code>volatile</code> Modifiers	44
11.1	<code>const</code> : “I promise not to write through this”	44
11.2	<code>volatile</code> : “this may change behind your back”	44
11.3	The classic <code>volatile</code> bug: the vanishing poll	45
11.4	<code>const volatile</code> : yes, both at once	45
12	The Anatomy of <code>.c</code> and <code>.h</code> Files	47
12.1	The mental model: declaration versus definition	47
12.2	What belongs in a header (<code>.h</code>)	47
12.2.1	Include guards	47
12.3	What belongs in a source file (<code>.c</code>)	48
12.4	Linkage: <code>static</code> and <code>extern</code>	49

12.5 Function prototyping is mandatory	49
IV Peripherals and the STM32 Ecosystem	51
13 Peripherals in the Memory Map	52
13.1 Memory-mapped I/O	52
13.2 Three flavours of register	52
13.3 Getting at individual bits	53
13.4 The typedef-struct overlay	53
14 The STM32CubeIDE Workflow	56
14.1 What is in the box	56
14.2 The configuration-first workflow	56
14.3 The “USER CODE” contract	57
14.4 FreeRTOS integration	57
15 HAL vs. LL, Callbacks, and Weak Symbols	59
15.1 Two libraries, two philosophies	59
15.2 Interrupt handling through HAL and LL	60
15.3 Callbacks: the concept	60
15.4 Weak symbols	60
16 The Timer Peripheral: Prescaler and Period	63
16.1 The counter and its two knobs	63
16.2 The master formula	63
16.3 What timers do for you	64
V Concurrency and Numerics	66
17 FreeRTOS Best Practices: Tasks, Queues, and Events	67
17.1 What the scheduler gives you	67
17.2 Tasks	67
17.3 Talking between tasks: queues first	68
17.4 Mutexes, semaphores, and priority inversion	68
17.5 The ISR boundary: deferred interrupt handling	69
18 Fixed-Point versus Floating-Point Arithmetic	71
18.1 Floating-point: IEEE 754	71
18.1.1 Hardware FPU versus software emulation	71
18.2 Fixed-point: integers with an implied scale	72

18.2.1	Doing arithmetic	72
18.2.2	The trade-offs	72
VI	Sensors and Serial Buses	74
19	Analog Sensor Interfacing: the NTC Thermistor	75
19.1	What an NTC is	75
19.2	Resistance to voltage: the divider	76
19.3	The full firmware chain	76
19.4	Design choices that matter	77
20	Digital-Interface Sensors: the Accelerometer	78
20.1	Why a digital interface is nicer	78
20.2	What a MEMS accelerometer measures, briefly	78
20.3	The register interface	79
20.4	The hard-disk-drive free-fall story	79
21	The I²C and SPI Protocols	81
21.1	SPI — Serial Peripheral Interface	81
21.2	I ² C — Inter-Integrated Circuit	82
21.3	Side by side	82
21.4	In the STM32: the peripheral view	82
VII	Power Electronics for Firmware Engineers	85
22	Power Electronics from Scratch	86
22.1	A transistor is a controlled valve	86
22.2	Two kinds of valve: voltage-operated vs current-operated	87
22.3	What actually turns a MOSFET on	87
22.4	The BJT as a switch: saturation and overdrive	87
22.5	Making a voltage above the rail	88
22.6	Why switching wastes energy	88
22.7	Heat: Ohm's law for temperature	89
23	Load Switches: Low-Side and High-Side	90
23.1	Why a transistor at all	90
23.2	Low-side switch	90
23.3	High-side switch	91
23.4	NPN BJT as a low-side switch, in detail	91

24 Integrated High-Side Drivers	94
24.1 What is inside	94
24.2 The features you get for free	94
24.3 Why turn-on/turn-off speed matters	95
25 Switch Power Dissipation and Base-Resistor Sizing	97
25.1 Where the power goes in a MOSFET	97
25.2 Where the power goes in a BJT	97
25.3 The thermal check	98
25.4 Sizing an NPN base resistor	98
26 Full-Bridges and Shoot-Through	100
26.1 Topology and operation	100
26.2 Shoot-through: the cardinal sin	100
26.3 The fix: dead-time	101
 VIII Laboratories	 103
27 Lab: Digital I/O and Timers (HAL vs. LL)	105
28 Lab: An NTC Thermometer	107
29 Lab: Closed-Loop Heater Control	109
30 Lab: An I²C Memory (Final Project)	110
A Syllabus Coverage Map	111
B Glossary of Acronyms	113

Notation and Conventions

Convention	Meaning
<code>monospace</code>	C identifiers, registers, shell commands, file names.
<code>SANS</code>	A single named bit inside a register (e.g. <code>TXE</code>).
<code>0x1FFF</code>	Hexadecimal literal (C notation).
<code>0b1010</code>	Binary literal.
<code>[a:b]</code>	A contiguous range of bits, MSB first (e.g. <code>CR[3:0]</code>).
<code>3.3 V</code> , <code>72 MHz</code>	Physical quantities, always with units.
<code>MCU</code> / <code>MPU</code>	<i>Microcontroller</i> (CPU + memory + peripherals on one chip) versus <i>microprocessor</i> (CPU that needs external memory).
<code>ISR</code>	Interrupt Service Routine.

Numerical answers are given to engineering precision (3 significant figures) unless more are needed to make a point. “ $\approx$ ” means a deliberate approximation, not sloppiness.

Part I

The Embedded Landscape

Chapter 1

Classic Computers versus Embedded Systems

You are almost certainly reading this on a “classic” data-processing system: a laptop, a desktop, or a phone that is really a small computer. Its job is to run *whatever software you throw at it* — a browser today, a game tomorrow, a compiler the day after. An **embedded system** is the opposite: a computer built into a product to do *one job*, usually invisibly, for years, with no operating system you ever see and no software you ever install. It is the chip in your washing machine, the engine control unit in a car, a drone’s flight controller, a pacemaker, the controller in an SSD, the thermostat on the wall.

Both are “computers”. Both execute machine code fetched from memory. But the constraints they live under are so different that almost every engineering decision flips sign. This chapter builds the mental model that the rest of the book assumes.

Key Idea

A PC is a *general-purpose* machine optimised for average-case throughput and programmer convenience. An embedded system is a *special-purpose* machine optimised for cost, power, determinism, and reliability — usually at the expense of convenience.

1.1 Two design philosophies

The PC world is built on the assumption of *abundance and abstraction*: abundant RAM, abundant storage, a megawatt of cooling somewhere, and a thick stack of abstractions (hardware → firmware → OS → libraries → your app) whose entire purpose is to hide the hardware from you. You allocate memory whenever you like; if you run low, the OS pages to disk. You never think about which physical address a variable lives at.

The embedded world is built on *scarcity and direct control*. A typical microcontroller (MCU) might have 64 kB of SRAM and 256 kB of flash — not gigabytes, kilobytes. There is often no operating system, no virtual memory, no second chance. Your code *is* the lowest layer; when it writes to address 0x40020014 it is toggling a physical pin, immediately. The abstractions are thin and you are expected to know what is underneath them.

1.2 Hardware differences

1.2.1 Processing core

PC CPUs are wide, deeply pipelined, super-scalar, multi-core machines running at several gigahertz with multi-megabyte caches and aggressive branch-prediction and out-of-order execution. All of that complexity exists to maximise *average* instructions-per-second on unpredictable workloads.

MCU cores (e.g. ARM Cortex-M0/M3/M4/M7) are comparatively tiny: a single core, 8 MHz to 600 MHz, a short pipeline, often *no* cache (so RAM access is single-cycle and perfectly predictable),

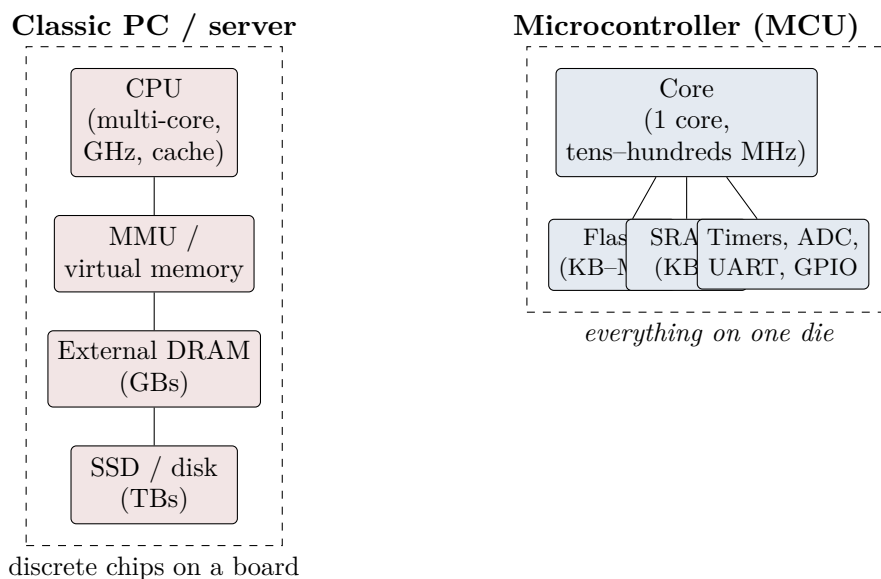


Figure 1.1: The structural contrast. A PC is a set of powerful discrete chips behind a memory-management unit; an MCU integrates a modest core, its memory, and its peripherals onto a single piece of silicon.

and frequently *no out-of-order execution*. Predictability is the feature. An anti-lock braking controller does not want a CPU that is fast *on average*; it wants one whose worst-case timing it can prove.

1.2.2 Memory hierarchy

This is the starkest difference. The PC has a deep hierarchy (registers → L1/L2/L3 cache → DRAM → swap on disk), all of it *volatile* for code/data and backed by a huge non-volatile disk. The MCU typically has just two levels that matter:

- **On-chip flash** (non-volatile): holds the program and constants. Survives power-off. This is where execution happens *from* — the CPU fetches instructions directly out of flash.
- **On-chip SRAM** (volatile): holds variables, the stack, and the heap (if any). Tiny and precious.

There is usually *no DRAM* and *no disk*. There is no swap; if you run out of RAM, you do not page out — you crash. We will spend several chapters (Part III) on the consequences of this.

1.2.3 Memory protection and address spaces

A PC has a Memory-Management Unit (MMU) giving every process its own virtual address space, hardware-enforced isolation, and demand paging. Most MCUs have *no MMU*. The whole chip lives in one flat physical address space that your single program owns entirely. Some Cortex-M parts have an optional Memory-Protection Unit (MPU) — a much simpler device that can mark regions read-only or no-execute, but it does *not* provide virtual addresses. The practical upshot: a wild pointer in firmware can scribble over anything, including the peripheral registers and the stack, with no OS to catch it.

1.2.4 Peripherals are first-class citizens

On a PC, talking to hardware means asking the OS through a driver. On an MCU, peripherals (timers, ADCs, UARTs, SPI/I²C controllers, GPIO) are *on the same die* and appear as **memory-mapped registers** — special addresses that, when read or written, control hardware directly. This memory-mapped model (Chapter 13) is the heart of firmware.

1.2.5 Power, cost, and physical envelope

A server CPU may dissipate 200 W. A coin-cell sensor node must run for years on a battery the size of a button, idling at microamps and waking up only to take a reading. An MCU might cost \$0.30 in volume, and shaving a few cents matters when you ship millions. These pressures explain “why don’t they just add more memory?” — because more memory is more die area is more cost and more leakage current.

Table 1.1: Hardware at a glance (representative, order-of-magnitude figures).

	Classic PC / server	Typical MCU
Cores / clock	4–64 cores, 2 GHz to 5 GHz	1 core, 8 MHz to 600 MHz
Cache	MBs, multi-level	often none
RAM	8 GB to 512 GB DRAM	2 kB to 1024 kB SRAM
Non-volatile store	TBs of SSD/disk	16 kB to 2048 kB flash
Memory management	full MMU, virtual memory	none, or simple MPU
Operating system	always (Windows/Linux/...)	often none, or an RTOS
Peripheral access	via OS drivers	direct, memory-mapped
Power	15 W to 300 W	1 μ W–1 W
Determinism	best-effort	hard real-time achievable

1.3 Software differences

1.3.1 There may be no operating system

A huge amount of firmware runs **bare-metal**: there is no OS, just your code in an infinite loop (the “super-loop”) plus interrupt handlers. When there *is* a kernel it is usually a **Real-Time Operating System (RTOS)** like FreeRTOS (Chapter 17): a few kilobytes of scheduler and synchronisation primitives, with no notion of users, files, processes, or dynamic program loading. You do not “install” a program; the firmware *is* the system.

1.3.2 The code is cross-compiled

You cannot compile on the target — it has no keyboard, screen, disk, or compiler. You build on your PC (the *host*) with a cross-compiler that emits machine code for the *target* architecture (Chapter 4), then flash the resulting binary into the chip’s program memory.

1.3.3 No dynamic loading, fixed memory layout

On a PC the loader maps your executable and shared libraries into memory at run time, relocating as needed. In firmware the memory layout is decided at *link time* by a linker script: “code goes in flash starting at 0x08000000, variables in SRAM starting at 0x20000000” (Chapter 5). There are no shared libraries to load; everything is statically linked into one image.

1.3.4 Real-time constraints are part of correctness

On a PC, slow is annoying. In firmware, late can be *wrong*: if the motor-control loop misses its deadline the motor stalls or the bridge blows up. “Real-time” does not mean “fast”, it means “provably on time, every time”. This is why we avoid constructs whose timing we cannot bound — unbounded recursion, `malloc`, blocking calls in interrupt context (Chapter 9).

1.3.5 Reliability and longevity

Desktop software can crash, show a dialog, and restart. A firmware crash in a fielded product may be unrecoverable and dangerous, and the product may need to run untouched for ten or twenty years. Hence watchdog timers (a hardware timer that resets the chip if your code stops “petting” it), defensive coding, and a strong preference for behaviour you can analyse statically.

Note

The line between “embedded” and “classic” is a spectrum, not a wall. A \$5 Linux-capable application processor (Cortex-A, with MMU and DRAM) running a trimmed Linux sits in the middle: it *is* an embedded product but inherits much of the PC software model. In this course “embedded” means the microcontroller end of the spectrum — no MMU, bare-metal or RTOS, KBs of RAM.

1.4 The spectrum of devices

It helps to name the three points on the spectrum you will meet:

MCU — microcontroller

CPU + flash + SRAM + peripherals on one chip. Runs bare-metal or an RTOS. Example: STM32F4 (Cortex-M4, 168 MHz, 192 kB SRAM). *This course.*

MPU — microprocessor / application processor

A powerful CPU (Cortex-A) that needs *external* DRAM and flash, has an MMU, and runs a full OS like Linux. Example: the chip in a Raspberry Pi.

SoC — system-on-chip

A blurry umbrella term for any chip integrating a CPU with substantial peripherals; both of the above are SoCs.

Worked Example 1.1 Sizing a problem before you start

You are asked to log one temperature reading per second for a week and keep all samples in RAM. Each sample is a `float` (4 bytes). Will it fit in a microcontroller with 64 kB of SRAM?

Samples in a week: $7 \times 24 \times 3600 = 604\,800$. At 4 bytes each that is 2 419 200 bytes ≈ 2.42 MB. The MCU has 64 kB (65 536 bytes) — short by a factor of about 37. On a PC you would not even think about it; here, the answer dictates the architecture. You must either store to external flash, transmit samples off-device, or down-sample/compress. **This kind of back-of-the-envelope check, done first, is the embedded mindset.**

1.5 Why every later chapter inherits from this one

Almost every “rule” in firmware traces back to the constraints above:

- No abundant RAM $\Rightarrow$ avoid `malloc`; allocate statically (Chapters 8, 9).
- No MMU $\Rightarrow$ a stray pointer corrupts hardware; be disciplined with `const`, prototypes, and file structure (Chapters 11, 12).
- Memory-mapped peripherals $\Rightarrow$ the register model and `volatile` (Chapters 13, 11).
- Real-time $\Rightarrow$ interrupts and the RTOS done carefully (Chapters 2, 17).
- Cost/power $\Rightarrow$ fixed-point maths instead of an FPU, smart power switching (Chapters 18, Part VII).

Exercise 1.1

For each of the following, decide whether it argues for an MCU or an application processor (MPU): (a) a battery sensor that must last 5 years on a coin cell; (b) a smart speaker that runs voice recognition and streams audio; (c) a brushless-motor controller with a 20 kHz control loop; (d) a home router.

Solution

(a) MCU — microamp sleep currents and no OS overhead. (b) MPU — needs gigabytes of RAM, an OS, and heavy compute. (c) MCU — the tight, deterministic loop wants a predictable core with no OS jitter and direct timer access. (d) MPU — a Linux networking stack and many MB of buffers.

Key Idea

Chapter takeaway. Embedded engineering is the art of doing a fixed job correctly and forever inside a tiny, cheap, low-power box with no safety net. Keep the constraints — bytes, microamps, microseconds, decades — in your head; they justify everything that follows.

Chapter 2

Microprocessor Interrupts

If there is one mechanism you must understand cold, it is the **interrupt**. It is how an embedded CPU — which can only do one thing at a time — reacts to the outside world without wasting its life asking “are we there yet?”. Every peripheral chapter later in this book ultimately delivers its results to your code through an interrupt. This is a *review*, but a thorough one, because the subtleties (latency, priority, reentrancy, shared data) are exactly where firmware bugs live.

2.1 The problem: how do you wait for the world?

Suppose a byte arrives on a serial port roughly once a millisecond, and the CPU runs at 100 MHz (10 ns per instruction-ish). You have two ways to notice the byte.

Polling. Sit in a loop reading the “data ready” status bit:

```
1 while (1) {
2     if (UART->SR & UART_SR_RXNE) {           /* byte ready?          */
3         uint8_t b = UART->DR;                 /* read it                   */
4         process(b);
5     }
6     /* ... otherwise spin, doing nothing useful ... */
7 }
```

Between bytes the CPU executes roughly *a hundred thousand* pointless loop iterations, burning power and unable to do anything else. Polling is simple and perfectly deterministic, but it monopolises the CPU.

Interrupts. Tell the hardware: “when a byte arrives, stop whatever you are doing, jump to this handler, then go back”. The CPU is free to do real work (or sleep to save power) the rest of the time. The peripheral, not the CPU, initiates the conversation.

Key Idea

An interrupt inverts control: instead of the CPU repeatedly asking the peripheral, the peripheral tells the CPU exactly when something happened. This trades a little complexity (and some hard-to-find bugs) for enormous gains in efficiency, responsiveness, and power.

2.2 The mechanism, step by step

When an enabled interrupt source asserts its request, and the CPU is willing to take it, the following happens — mostly in hardware, automatically:

1. **Finish (or abandon) the current instruction.** The core reaches an interruptible boundary.

2. **Save context.** The return address and enough processor state to resume are pushed onto the stack. On Cortex-M the hardware automatically stacks eight registers (R0–R3, R12, LR, PC, xPSR) so a C function can serve as the handler with no assembly glue.
3. **Look up the handler.** The core reads the handler’s address from the **interrupt vector table** — an array of function pointers, one slot per interrupt source, located at a known address.
4. **Jump to the ISR** (Interrupt Service Routine) and run it.
5. **Return from interrupt.** A special return mechanism restores the saved context and resumes the interrupted code exactly where it left off — which is usually unaware anything happened.

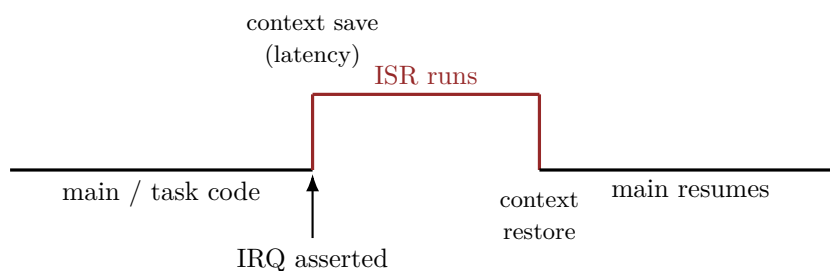


Figure 2.1: An interrupt suspends the running code, runs the ISR, and resumes. The time from the request to the first useful ISR instruction is the *interrupt latency*.

2.3 The vector table

The vector table is the bridge between a hardware event and your C function. It is just an array of addresses at a fixed location (on Cortex-M, by default at the start of flash, address 0x00000000 aliased to 0x08000000). Slot 0 is special — it holds the *initial stack pointer* — and slot 1 holds the reset handler’s address. The rest are exception and interrupt handlers in a fixed order defined by the chip.

```

1  __attribute__((section(".isr_vector")))
2  const void * const vector_table[] = {
3      (void *) &_estack,           /* 0: initial stack pointer      */
4      Reset_Handler,             /* 1: reset                      */
5      NMI_Handler,               /* 2: non-maskable interrupt     */
6      HardFault_Handler,         /* 3: catch-all fault          */
7      /* ... core exceptions ... */
8      SysTick_Handler,          /* 15: system tick timer        */
9      /* ---- device-specific IRQs from here ---- */
10     WWDG_IRQHandler,           /* 16: window watchdog          */
11     /* ... */
12     USART2_IRQHandler,         /* serial port 2                 */
13     /* ... */
14 };

```

Listing 2.1: Conceptual vector table (Cortex-M, simplified).

When USART2 raises its interrupt, the core indexes this table, finds the address of USART2_IRQHandler, and calls it. That is the *entire* magic: a hardware-driven indirect jump through a table of function pointers.

Note

On ARM Cortex-M the interrupt controller is the **NVIC** (Nested Vectored Interrupt Controller). “Vectored” means the hardware itself selects the right handler from the table (no software dispatch needed); “nested” means a higher-priority interrupt can interrupt a lower-priority ISR.

2.4 Latency, tail-chaining, and why it matters

Interrupt latency is the delay from the request being asserted to the first instruction of your ISR executing. It includes finishing the current instruction, the context save, and the vector fetch. On Cortex-M this is a fixed, published number (e.g. 12 cycles), which is precisely the kind of determinism embedded systems prize.

Two clever optimisations reduce overhead when interrupts come back-to-back:

- **Tail-chaining:** if a second interrupt is pending when an ISR finishes, the core skips the pop-then-push and goes straight into the next ISR, saving the redundant stack traffic.
- **Late arrival:** if a higher-priority interrupt arrives during the context save of a lower-priority one, the core switches to the higher-priority handler without redoing the stacking.

2.5 Priorities, preemption, and nesting

Real systems have many interrupt sources, and some matter more than others. Each interrupt is assigned a **priority**. When several are pending, the highest-priority one runs first. Crucially, if a high-priority interrupt fires while a low-priority ISR is running, the high-priority one *preempts* the low one — the low ISR is itself interrupted, the high ISR runs to completion, then the low ISR resumes. This is **nesting**.

Watch Out

On Cortex-M, lower numerical priority value = higher urgency. Priority 0 is the *most* urgent. This catches everyone at least once. Also, only the upper bits of the priority field are implemented (e.g. 4 bits $\Rightarrow$ 16 levels), so writing 1 and 2 may give the *same* effective priority.

2.6 The hard part: reentrancy and shared data

An ISR runs “asynchronously” — it can fire *between any two machine instructions* of your main code. This is the source of the nastiest firmware bugs. Consider a 32-bit counter incremented in an ISR and read in **main**:

```
1 volatile uint32_t ticks;           /* shared with ISR          */
2
3 void SysTick_Handler(void) { ticks++; } /* in interrupt context */
4
5 void main_loop(void) {
6     uint32_t now = ticks;           /* read shared variable    */
7     /* ... */
8 }
```

Two distinct problems hide here.

1. The compiler must not cache the variable. Without `volatile`, the compiler may read `ticks` once into a register and reuse the stale copy forever, never seeing the ISR’s updates. The `volatile` keyword forces an actual memory access every time (Chapter 11).

2. Access may not be atomic. On an 8- or 16-bit CPU, reading a 32-bit `ticks` takes several instructions. If the ISR fires *between* them, you can read a half-updated value (the “torn read”). The fix is a **critical section**: briefly disable the interrupt around the access so it cannot interleave.

```

1 uint32_t read_ticks(void) {
2     uint32_t t;
3     __disable_irq();           /* enter critical section */
4     t = ticks;                 /* now indivisible w.r.t. the ISR */
5     __enable_irq();           /* leave critical section */
6     return t;
7 }

```

Key Idea

Any variable shared between an ISR and the rest of the code must be (a) `volatile` so the compiler always re-reads it, and (b) accessed atomically — either because the access is naturally a single indivisible instruction, or because you protect it with a critical section.

2.7 Rules for writing good ISRs

The discipline below will save you days of debugging:

- **Keep it short.** An ISR steals the CPU from everything of lower priority. Do the minimum (read the byte, clear the flag, set a flag for `main`), then return. Defer real work to the main loop or a task.
- **Never block.** No busy-waits, no `delay()`, no waiting on another interrupt that can’t fire because you’ve preempted it.
- **No `malloc`, no `printf`.** These are slow, non-deterministic, and often not reentrant (Chapter 8).
- **Clear the interrupt flag.** If you forget, many peripherals will re-assert immediately and your ISR fires forever.
- **Mark shared variables `volatile`** and guard non-atomic access.

Note

Exceptions vs. interrupts. An *interrupt* comes from a peripheral (timer, UART). An *exception* (also called a trap or fault) comes from the core itself reacting to the instruction stream: a divide-by-zero, an unaligned access, a bus fault from a bad pointer. On Cortex-M both flow through the same vector table; faults land in `HardFault_Handler`, which is where a wild pointer in a no-MMU system usually ends its journey. A *non-maskable interrupt* (NMI) cannot be disabled and is reserved for truly critical events (e.g. a clock failure).

Worked Example 2.1 The lost-wakeup bug

A common pattern: an ISR sets a flag, and `main` sleeps until it is set.

```

1 volatile bool data_ready = false;

```

```
2  /* ISR: */ void EXTI_Handler(void){ data_ready = true; }
3  /* main: */
4  while (1) {
5      if (!data_ready) __WFI();          /* sleep until interrupt */
6      if (data_ready) { handle(); data_ready = false; }
7  }
```

What is the race? If the interrupt fires *between* the `if (!data_ready)` test and the `__WFI()` instruction, the flag is already set but the CPU still goes to sleep — and may never wake (lost wakeup). The robust fix is to disable interrupts, re-test the flag, and use the sleep-on-exit / `__WFI` semantics that wake on a *pending* interrupt, so a request latched just before `__WFI` prevents the sleep. The lesson: reasoning about “between which two instructions can the ISR fire?” is a core firmware skill.

Exercise 2.1

A 16 bit timer’s interrupt increments a 32-bit `overflow_count` in software so you can build a 48-bit timestamp. Reading the full timestamp in the main loop occasionally returns a wildly wrong value. Explain the bug and give two different fixes.

Solution

The timer’s hardware counter and the software `overflow_count` are read in two steps; the timer can overflow (and the ISR can fire) *between* the two reads, so you pair a pre-overflow counter with a post-overflow count (or vice versa). **Fix 1:** read in a critical section (disable the timer IRQ around both reads). **Fix 2:** read `overflow_count`, then the counter, then `overflow_count` again; if the two reads of `overflow_count` differ, an overflow happened in between, so retry (a lock-free “seqlock” pattern).

Key Idea

Chapter takeaway. Interrupts give you efficient, responsive, event-driven firmware — at the price of asynchrony. Master three things: the vector-table mechanism, the priority/nesting model, and the discipline of `volatile` + critical sections for shared data. Everything peripheral in this book rides on these rails.

Part II

From Source Code to a Booting Binary

Chapter 3

How an Executable is Built — the GCC Model

You write `main.c`; the chip runs ones and zeros. In between sits a *pipeline* of tools that the `gcc` command quietly orchestrates. In a PC course you can get away with treating `gcc hello.c` as a black box. In firmware you cannot: you will routinely inspect the assembly the compiler produced, check which section a variable landed in, and read a disassembly to understand a crash. This chapter opens the black box.

Key Idea

`gcc` is not a compiler — it is a *driver* that runs four tools in sequence: the preprocessor, the compiler proper, the assembler, and the linker. Knowing where each one starts and stops lets you stop the pipeline anywhere and look inside.

3.1 The four stages

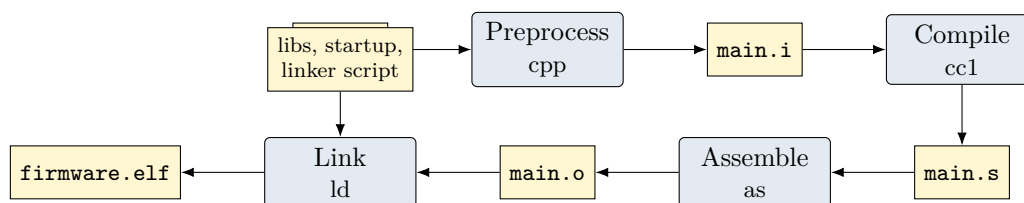


Figure 3.1: The GCC pipeline. Each arrow is a file you can ask `gcc` to stop at and hand you.

3.1.1 Stage 1 — Preprocessing (`cpp`)

Pure text substitution, before any C is understood. The preprocessor:

- pastes in `#included` files verbatim;
- expands `#define` macros;
- resolves conditional compilation (`#if`, `#ifdef`, `#ifndef`);
- strips comments.

The output is still C, but “flattened”: one big translation unit with all headers inlined and all macros expanded. See it with:

```
1 gcc -E main.c -o main.i      # -E: stop after preprocessing
```

Open `main.i` and you will find thousands of lines from the headers above your three lines of code. This is why a one-line `printf` program is not really small.

3.1.2 Stage 2 — Compilation (`cc1`)

The actual compiler. It parses the preprocessed C, type-checks it, optimises it, and emits *assembly* for the target architecture — still human-readable text, but now architecture-specific.

```
1 gcc -S main.c -o main.s      # -S: stop after compiling, emit assembly
2 gcc -S -O2 main.c -o main.s  # ... with optimisation level 2
```

Reading `main.s` (especially with and without `-O2`) is the single best way to learn what your C “really” costs — a skill we lean on heavily in Chapter 6.

3.1.3 Stage 3 — Assembly (`as`)

The assembler translates the textual assembly into **machine code**, producing a *relocatable object file* (`.o`). “Relocatable” means addresses are not yet final: the file says “call the function `foo`” by name, not by address, because it does not yet know where `foo` will live.

```
1 gcc -c main.c -o main.o      # -c: preprocess + compile + assemble, no link
```

An object file is structured (it is an ELF file, see Chapter 5): it contains sections (`.text` for code, `.data` for initialised variables, ...), a **symbol table** (names it defines and names it needs), and **relocation entries** (places that must be patched once addresses are known).

3.1.4 Stage 4 — Linking (`ld`)

The linker takes one or more object files plus libraries, and:

1. **merges sections** of the same kind from every object (all `.text` together, all `.data` together, ...);
2. **resolves symbols** — matches each “I need `foo`” to a “`foo` is here” somewhere, erroring with *undefined reference* if none exists;
3. **assigns final addresses** according to the *linker script* (in firmware) or default rules (on a PC);
4. **relocates** — patches all the placeholder addresses to their final values.

The result is an executable. On a PC that is an ELF the OS can load; in firmware it is an ELF you post-process into a raw image to flash (Chapter 4).

Note

The “one-shot” command `gcc main.c -o app` simply runs all four stages back-to-back and deletes the intermediates. Everything above just exposes the seams.

3.2 Reading the binaries you produced: `objdump` and friends

A produced `.o` or `.elf` is not a mystery box; the `binutils` suite lets you read every byte. `objdump` is the workhorse.

Table 3.1: The binutils you will actually use.

Command	What it shows
<code>objdump -h file</code>	Section headers: names, sizes, addresses (VMA/LMA).
<code>objdump -d file</code>	Disassembly of executable sections (machine code → assembly).
<code>objdump -S file</code>	Disassembly <i>interleaved with C source</i> (needs <code>-g</code>).
<code>objdump -t file</code>	The symbol table.
<code>objdump -s -j .rodata file</code>	Raw hex dump of one named section.
<code>nm file</code>	Just the symbols, compactly (T=text, D=data, B=bss, U=undefined).
<code>readelf -a file</code>	Everything about the ELF structure, exhaustively.
<code>size file</code>	The bottom line: bytes of <code>.text</code> , <code>.data</code> , <code>.bss</code> .

Worked Example 3.1 Disassembling a function

Take a trivial function and watch it turn into machine code.

```
1 int add(int a, int b) { return a + b; }
```

Compile and disassemble:

```
1 gcc -c -O2 add.c -o add.o
2 objdump -d add.o
```

On an x86-64 host you would see something like:

```
1 0000000000000000 <add>:
2   0:  8d 04 37      lea      (%rdi,%rsi,1),%eax    ; eax = a + b
3   3:  c3              ret                          ; return eax
```

The first column is the byte offset, the second the raw machine-code bytes (this is what actually sits in memory), and the rest the disassembled mnemonic. Notice the compiler used `lea` (a quirky one-instruction add) rather than a plain `add` — exactly the kind of optimisation you only discover by reading the output. Compiling the same code for ARM with `arm-none-eabi-gcc` would show `adds r0, r0, r1 ; bx lr` instead: same logic, different ISA.

Worked Example 3.2 Where did my program’s size go?

`size` answers “will it fit in the chip?” at a glance:

```
1 $ arm-none-eabi-size firmware.elf
2   text    data    bss    dec     hex filename
3  24680    112    8460   33252   81e4 firmware.elf
```

Here `text` (24 680 B) + `data` (112 B) must fit in flash, while `data` + `bss` (112+8460 = 8572 B) must fit in SRAM. We unpack exactly what these mean in Chapter 5.

Exercise 3.1

You compile `util.c` with `gcc -c util.c` and `nm util.o` lists a symbol `U memcpy`. The final link then fails with “undefined reference to `memcpy`”. What does the `U` mean, and what are two ways to fix the link?

Solution

`U` means `util.o` uses `memcpy` but does not define it — it is an unresolved external reference, exactly what the linker must satisfy. Fixes: (1) link against the C library that provides `memcpy` (e.g. add the standard library, which the `gcc` driver normally does automatically; maybe you called `ld` directly and forgot `-lc`); (2) provide your own `memcpy` definition and link it in (common in tiny bare-metal builds that avoid the standard library). The root idea: every `U` must be matched by exactly one definition somewhere at link time.

Key Idea

Chapter takeaway. Source $\rightarrow$ `.i` $\rightarrow$ `.s` $\rightarrow$ `.o` $\rightarrow$ `.elf`, driven by `gcc`. Stop the pipeline with `-E`, `-S`, `-c`; inspect the results with `objdump`, `nm`, `readelf`, and `size`. This fluency is non-negotiable for debugging firmware.

Chapter 4

Cross-Compilation for Embedded Targets

In Chapter 3 every command ran on, and produced code for, your PC. Firmware breaks that symmetry: you build on a powerful PC but the program must run on a chip with a completely different instruction set, no operating system, and no compiler of its own. That is **cross-compilation**.

4.1 Why you cannot just compile on the target

Two independent reasons:

1. **Different instruction set.** Your PC is x86-64; the MCU is ARM Thumb. A compiler emits code for *one* ISA; `gcc` on your PC emits x86-64, which the ARM core cannot execute.
2. **No resources on the target.** Even an ARM PC could not host the build: the MCU has kilobytes of RAM, no file system, no OS, no text editor. Running a compiler there is absurd.

So we use a **cross-toolchain**: a build of GCC that *runs* on the host (x86-64 Linux) but *emits* code for the target (ARM Cortex-M).

4.2 The three machines and the GNU triplet

GNU build terminology names three roles:

build the machine where the toolchain itself was compiled;

host the machine where the toolchain *runs* (your PC);

target the machine the produced code runs on (the MCU).

For ordinary native `gcc`, all three are the same. For a cross-compiler, `host` $\neq$ `target`. Toolchains are named by a **triplet** (really up to four fields):

$$\underbrace{\text{arm}}_{\text{arch}} - \underbrace{\text{none}}_{\text{vendor}} - \underbrace{\text{eabi}}_{\text{OS/ABI}}$$

The ubiquitous embedded ARM toolchain is **arm-none-eabi-gcc**:

- **arm** — the target architecture is ARM;
- **none** — no specific vendor;
- **eabi** — the “Embedded Application Binary Interface”: there is *no operating system*. (Contrast `arm-linux-gnueabi`, which *does* target Linux and its C library.)

Every tool in the suite carries the same prefix: `arm-none-eabi-gcc`, `arm-none-eabi-ld`, `arm-none-eabi-objdump`, `arm-none-eabi-objcopy`, `arm-none-eabi-gdb`.

Note

The `none` in `arm-none-eabi` is also why its C library is **newlib** (or `newlib-nano`), a small libc designed for systems with no OS, rather than the desktop `glibc`. We meet newlib’s quirks in Chapter 8.

4.3 What changes versus native gcc

The pipeline is identical; what changes is the *context*.

4.3.1 You must tell it exactly which core

A PC `gcc` assumes the machine it runs on. The cross-compiler targets a whole family, so you specify the core, instruction set, and FPU:

```
1 arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb \
2                   -mfpu=fpv4-sp-d16 -mfloat-abi=hard \
3                   -O2 -c main.c -o main.o
```

Table 4.1: The flags that pin down a Cortex-M build.

Flag	Meaning
<code>-mcpu=cortex-m4</code>	Generate code for this specific core.
<code>-mthumb</code>	Use the 16/32-bit Thumb instruction set (the only one Cortex-M supports).
<code>-mfpu=fpv4-sp-d16</code>	Target the single-precision hardware FPU.
<code>-mfloat-abi=hard</code>	Pass floats in FPU registers and use FPU instructions (vs <code>soft</code> , which emulates floats in software — see Ch. 18).

Get `-mfloat-abi` wrong and you get a confusing link error, because hard- and soft-float objects use different calling conventions and cannot be mixed.

4.3.2 There is no hosted environment

A PC program starts at `main` thanks to the OS and the C runtime setting everything up. On bare metal *you* supply that scaffolding:

- **Startup code** (often `startup_stm32xxxx.s`): sets the initial stack pointer, copies initialised data from flash to RAM, zeroes `.bss`, and finally calls `main` (details in Chapter 5).
- **A linker script** (`.ld`): tells `ld` that flash starts at `0x08000000` and RAM at `0x20000000`, and which section goes where.
- **System-call stubs**: library functions like `printf` eventually call `_write`; with no OS you either provide a stub (retarget it to a UART) or link `-specs=nosys.specs` to get empty ones.

Hence build commands sprinkle in `-nostartfiles`, `-T linker.ld`, `-specs=nano.specs`, and similar — all saying “there is no OS here, here is what to use instead.”

4.3.3 The output must be turned into a flashable image

The linker emits an `.elf`, which is rich with debug info and section metadata — great for the debugger, but not what a flash programmer wants. So the final step strips it down to a raw image with `objcopy`:

```
1 arm-none-eabi-objcopy -O binary firmware.elf firmware.bin # raw bytes
2 arm-none-eabi-objcopy -O ihex   firmware.elf firmware.hex # Intel HEX
```

The `.bin` is the literal contents of flash; the `.hex` (Intel HEX) is an ASCII format carrying address + data records that most programmers accept. The `.elf` is still what you load into the debugger, because it carries the symbols and line numbers.

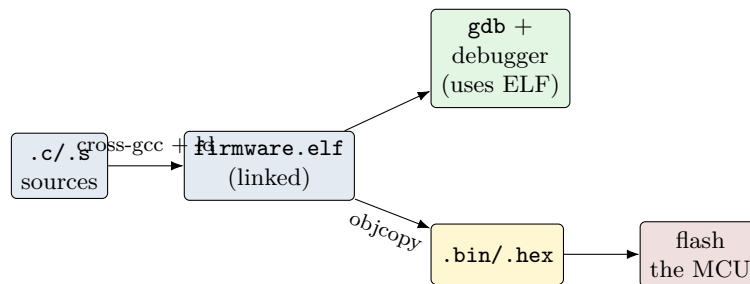


Figure 4.1: The cross build: one `.elf` feeds both the debugger and (via `objcopy`) the flash programmer.

Worked Example 4.1 Decoding a real build line

You see this in a Makefile and must explain it:

```
1 arm-none-eabi-gcc -mcpu=cortex-m0plus -mthumb -Os -g3 \
2   -ffunction-sections -fdata-sections \
3   -T STM32L011.ld -Wl,--gc-sections -nostartfiles \
4   startup.s main.c -o app.elf
```

Reading left to right: target a Cortex-M0+ in Thumb mode; `-Os` optimise for size (typical in firmware where flash is tight); `-g3` full debug info; `-ffunction-sections -fdata-sections` put each function and variable in its *own* section so that `-Wl,--gc-sections` (a linker flag, passed via `-Wl,`) can garbage-collect anything unused — a standard trio for shrinking images; `-T STM32L011.ld` use this linker script; `-nostartfiles` don't pull in the host C runtime startup (we provide `startup.s`). The output is the ELF, ready for `objcopy` and the debugger.

Exercise 4.1

A colleague reports that their bare-metal program links fine but “does nothing” on the chip — `main` never seems to run. They built with the *native* `gcc` on their x86 laptop and flashed the `.bin`. List two independent reasons this cannot possibly work.

Solution

(1) *Wrong ISA*: native `gcc` emits x86-64 machine code; the ARM core fetches those bytes as ARM/Thumb instructions and executes garbage (or faults immediately). They must use `arm-none-eabi-gcc`. (2) *Wrong layout and no startup*: native `gcc` links for a hosted OS at PC addresses, with no vector table at `0x08000000`, no reset handler, and no flash-to-RAM data copy. Even with the right ISA, without a linker script and startup code the chip would not boot. Cross-compilation is about both the instruction set *and* the runtime environment.

Key Idea

Chapter takeaway. Cross-compiling = build on the host, run on the target. Use the `arm-none-eabi-*` suite, name the exact core and float-ABI, supply the startup code and linker script the missing OS would otherwise provide, and `objcopy` the ELF into a flashable image.

Chapter 5

Sections in Object Files and Executables

Chapter 3 kept mentioning `.text`, `.data`, and `.bss`. These are **sections**: named regions inside an object or executable file, each holding a different *kind* of content. On a PC the loader and OS handle them invisibly. In firmware, sections are something you must understand and often control by hand, because *you* decide which physical memory each one lands in. Getting this wrong means a variable that silently fails to initialise, or a program that does not boot at all.

5.1 Why a file needs sections at all

Code and data are different beasts with different needs:

- Code never changes at run time $\Rightarrow$ it can live in read-only *flash*.
- A string literal `"hello"` never changes $\Rightarrow$ flash too.
- A global `int counter = 5;` must be writable $\Rightarrow$ it lives in *RAM*, but its initial value 5 must be stored somewhere non-volatile (flash) and copied to RAM at boot.
- A global `int buffer[256];` (uninitialised) must be writable RAM, but needs *no* stored initial values — just zeroing.

Grouping content by these properties is exactly what sections do.

Table 5.1: The canonical sections of an embedded image.

Section	Holds	Lives in	Stored in image?
<code>.text</code>	executable code, sometimes constants	flash	yes
<code>.rodata</code>	<code>const</code> data, string literals	flash	yes
<code>.data</code>	globals/statics with a nonzero initial value	RAM	<i>init values</i> in flash
<code>.bss</code>	globals/statics initialised to zero (or uninit.)	RAM	no (just reserved)
<code>.isr_vector</code>	the interrupt vector table	flash (start)	yes
stack	local variables, return addresses	RAM (top)	no
heap	<code>malloc</code> pool (if any)	RAM	no

Key Idea

The deciding questions for any byte are: *does it change at run time?* (flash vs RAM) and *does it need a stored initial value?* (in the image vs merely reserved). Those two questions place everything into `.text`, `.rodata`, `.data`, or `.bss`.

5.2 The crucial subtlety: VMA versus LMA

This single idea trips up everyone the first time. `.data` (initialised globals) must *run* from RAM, but its initial values must *survive power-off* in flash. So `.data` has *two* addresses:

LMA — Load Memory Address

where the bytes sit in the image (in flash).

VMA — Virtual/run Memory Address

where the program expects them at run time (in RAM).

At boot, the startup code copies `.data` from its LMA (flash) to its VMA (RAM). `.text` and `.rodata` have LMA = VMA (they run in place from flash). `.bss` has no LMA at all — there is nothing to copy, only to zero.

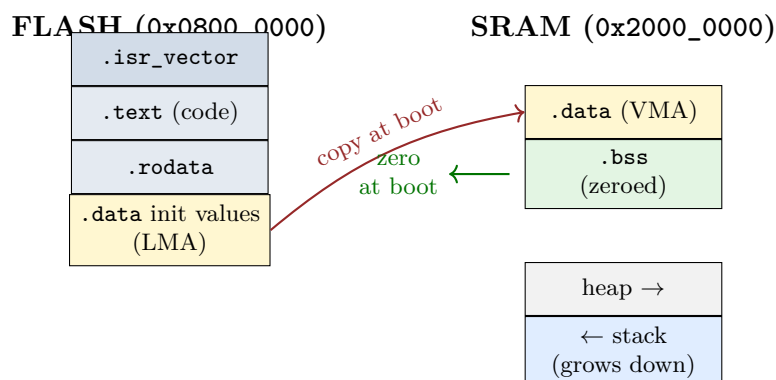


Figure 5.1: The boot-time picture. `.data`'s initial values are copied flash → RAM (LMA → VMA); `.bss` is zeroed in place; `.text` and `.rodata` execute directly from flash. Stack and heap grow toward each other in the remaining RAM.

5.3 The linker script

The linker script (`.ld`) is where you, the firmware engineer, declare the chip's memory and place sections into it. Two blocks matter:

```

1 MEMORY {
2     FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 256K
3     RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 64K
4 }
5
6 SECTIONS {
7     .isr_vector : { KEEP(*(.isr_vector)) } > FLASH
8     .text       : { *(.text*) *(.rodata*) } > FLASH
9
10    _sdata = LOADADDR(.data);           /* LMA of .data, for startup */
11    .data : {
12        _sdata = .;                     /* VMA start */
13        *(.data*)
14        _edata = .;                     /* VMA end */
15    } > RAM AT> FLASH                  /* VMA in RAM, LMA in FLASH */
16
17    .bss : {
18        _sbss = .;
19        *(.bss*) *(COMMON)
20        _ebss = .;
21    } > RAM
22 }
```

Listing 5.1: Skeleton STM32 linker script.

The phrase `> RAM AT> FLASH` is the VMA/LMA split made explicit: “run from RAM, but store the bytes in flash”. The symbols `_sidata`, `_sdata`, `_edata`, `_sbss`, `_ebss` are sign-posts the startup code uses.

5.4 What the startup code actually does

Before `main` runs, the reset handler performs the boot ritual implied by the section model:

```

1  extern uint32_t _sidata, _sdata, _edata, _sbss, _ebss;
2
3  void Reset_Handler(void) {
4      /* 1. copy .data initial values from flash (LMA) to RAM (VMA) */
5      uint32_t *src = &_amp;_sidata, *dst = &_amp;_sdata;
6      while (dst < &_amp;_edata) *dst++ = *src++;
7
8      /* 2. zero .bss */
9      for (dst = &_amp;_sbss; dst < &_amp;_ebss; ) *dst++ = 0;
10
11     /* 3. (call C++ constructors, set up the FPU, clocks, ...) */
12     main();
13     /* 4. finally, your code */
14 }
```

Listing 5.2: The essence of `Reset_Handler`.

This is why a global `int x = 5;` is 5 when `main` starts: step 1 put it there. And it is why a global `int y;` is 0: step 2 zeroed it. C guarantees “statics start at zero” — and now you know *who* actually does the zeroing.

Watch Out

If your startup code or linker script is wrong, initialised globals contain garbage and zero-initialised globals are nonzero — yet the program may *seem* to run. These bugs are maddening because the C standard *promised* the initialisation and the failure is invisible in the source. When “a global has the wrong value before I ever touch it”, suspect the `.data/.bss` setup.

5.5 Custom sections via attributes

GCC lets you place a specific symbol in a section you name, with `__attribute__((section("...")))`. This is how you exploit special memories or opt out of the normal init:

```

1  /* A buffer in fast Core-Coupled Memory (CCM), placed by the linker script */
2  uint8_t fast_buf[1024] __attribute__((section(".ccmram")));
3
4  /* A variable that must survive a warm reset: put it in a NOINIT region
5     the startup code is told NOT to zero. */
6  uint32_t boot_reason __attribute__((section(".noinit")));
7
8  /* The vector table, kept even though nothing 'calls' it. */
9  __attribute__((section(".isr_vector"))) const void *vectors[] = { ... };
```

We use exactly this mechanism in Chapter 10 to place DMA buffers in a non-cached, correctly-aligned region.

Worked Example 5.1 Reading section sizes from objdump

```

1 $ arm-none-eabi-objdump -h firmware.elf
2 Idx Name      Size      VMA      LMA      File off
3   0 .isr_vector 000001c4 08000000 08000000 ...
4   1 .text      00005a30 080001c4 080001c4 ...
5   2 .rodata    00000420 08005bf4 08005bf4 ...
6   3 .data      00000088 20000000 08006014 ... <-- VMA != LMA
7   4 .bss      0000210c 20000088 20000088 ...

```

Flash usage = end of `.data`'s LMA – flash origin $\approx 0x6014 + 0x88 = 24\,732$ B. RAM usage = `.data` + `.bss` = $0x88 + 0x210c = 8596$ B, *before* the stack and heap. Note row 3: `.data`'s VMA (`0x2000_0000`, RAM) differs from its LMA (`0x0800_6014`, flash) — the VMA/LMA split, visible in the wild.

Exercise 5.1

For each declaration at file scope, name the section it lands in and say whether any bytes are stored in the flash image for it: (a) `const char msg[] = "OK";` (b) `int gains[8] = 1,2,3,4,5,6,7,8;` (c) `static int scratch[256];` (d) `char version = 'A';`

Solution

(a) `.rodata` — stored in flash, runs from flash (it is `const`). (b) `.data` — the eight values are stored in flash (LMA) and copied to RAM (VMA) at boot. (c) `.bss` — nothing stored in the image, just RAM reserved and zeroed at boot. (d) `.data` — the single byte `'A'` is stored in flash and copied to RAM. The pattern: nonzero initialiser $\rightarrow$ `.data`; zero/no initialiser $\rightarrow$ `.bss`; `const` $\rightarrow$ `.rodata`.

Key Idea

Chapter takeaway. An image is a set of sections with two address spaces: where they load (LMA, flash) and where they run (VMA, RAM). The linker script places them and the startup code reconciles them (copy `.data`, zero `.bss`). Master this and “the chip won’t boot” / “my global is garbage” stop being mysteries.

Chapter 6

Where Variables Live: Static versus Local Allocation

Chapter 5 placed *global* data into sections. This chapter follows the other half: *local* variables, and how the compiler’s optimiser dramatically changes where everything lives. Understanding this is what lets you read a disassembly, reason about stack usage (a real hazard with no MMU), and predict performance.

6.1 Storage duration decides the home

C variables have a **storage duration** that determines their lifetime and home:

static globals, **static** locals, and file-scope variables. They exist for the whole program and live in `.data/.bss/ .rodata` (Chapter 5). One fixed address each.

automatic

ordinary locals and function parameters. They exist only while their function runs. They live on the **stack** — or, very often, purely in **CPU registers**.

6.2 The stack and a function’s stack frame

The stack is a region of RAM that grows and shrinks as functions are called and return (on ARM and x86 it grows *downward*, toward lower addresses). Each active function owns a slice called its **stack frame**, holding its saved registers, the return address, and any locals that could not stay in registers.

A function’s **prologue** (its first few instructions) allocates the frame by subtracting from the stack pointer; the **epilogue** releases it by restoring the stack pointer before returning.

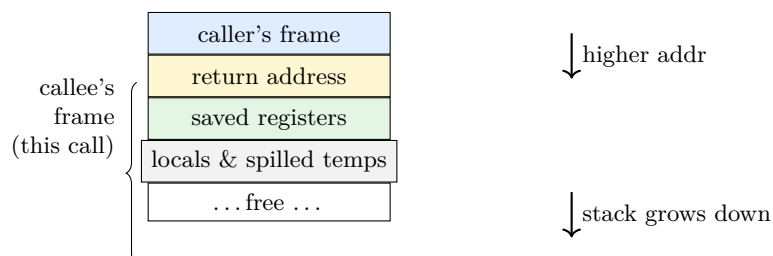


Figure 6.1: A stack frame. The stack pointer moves down on entry (prologue) and back up on exit (epilogue).

6.3 The effect of optimisation: the same code, twice

This is the heart of the topic. Compile a trivial function at `-O0` and `-O2` and the storage of the locals changes completely.

```

1 int scale(int x) {
2     int a = x + 1;
3     int b = a * 2;
4     return b;
5 }
```

Listing 6.1: The source.

6.3.1 Without optimisation (`-O0`)

The compiler is literal and debuggable: *every* local gets a slot on the stack, written and re-read at each use, so you can inspect them in a debugger. Conceptually (ARM Thumb):

```

1 scale:
2     push    {r7}                ; set up frame pointer
3     sub     sp, #12             ; reserve stack for x, a, b
4     str     r0, [sp, #4]        ; store parameter x to stack
5     ldr     r3, [sp, #4]        ; a = x + 1 (reload x!)
6     adds    r3, #1
7     str     r3, [sp, #0]        ; store a to stack
8     ldr     r3, [sp, #0]        ; b = a * 2 (reload a!)
9     lsls    r3, r3, #1
10    str     r3, [sp, #8]        ; store b to stack
11    ldr     r0, [sp, #8]        ; return value <- b
12    add     sp, #12
13    pop     {r7}
14    bx      lr
```

Lots of `str/ldr`: the variables genuinely occupy stack memory.

6.3.2 With optimisation (`-O2`)

The optimiser sees that `a` and `b` are short-lived temporaries, keeps everything in registers, folds the arithmetic $(x+1) \cdot 2 = 2x + 2$, and never touches the stack:

```

1 scale:
2     adds    r0, r0, #1          ; r0 = x + 1
3     lsls    r0, r0, #1          ; r0 = (x+1) * 2
4     bx      lr                 ; return r0 -- no stack frame at all!
```

Key Idea

“A local variable lives on the stack” is only the *unoptimised* truth. With optimisation, locals whose address is never taken often live entirely in registers and may vanish through constant-folding. The stack frame shrinks or disappears. This is why optimised code is faster — and harder to single-step.

Watch Out

Two consequences for debugging: at `-O2`, a watch on a local may show “optimised out”, and stepping jumps around because instructions were reordered. This is also why we `volatile`-qualify memory-mapped registers (Chapter 11): otherwise the optimiser would

“keep them in a register” too, which for hardware is catastrophic. **Taking a local’s address** (`&a`) forces it onto the stack, since registers have no address.

6.4 RISC versus CISC variations

How many locals stay in registers depends on the architecture, and this is where RISC and CISC diverge.

RISC (e.g. ARM Cortex-M). A *load/store* architecture: arithmetic operates only on registers, and there are *many* of them (13–16 general registers). So the compiler keeps lots of locals in registers and touches memory rarely. The calling convention (AAPCS) passes the first four integer arguments in `r0–r3` and returns the result in `r0` — so a small function may never spill to the stack at all, as we just saw.

CISC (e.g. x86). Historically *few* registers (8 on 32-bit x86) and instructions that can operate directly on memory operands (`add %eax, (%ebp)`). With fewer registers, locals “spill” to the stack more readily, and the architecture is comfortable computing on memory-resident variables. The 32-bit System V ABI even passed arguments *on the stack* by default (64-bit x86-64 added register passing).

Table 6.1: RISC vs CISC, from the variable-allocation viewpoint.

	RISC (ARM)	CISC (x86)
General registers	many (13–16)	few (8 in 32-bit)
Arithmetic operands	registers only	registers <i>or</i> memory
Tendency for locals	stay in registers	spill to stack sooner
Argument passing	<code>r0–r3</code> , then stack	32-bit: stack; 64-bit: registers
Memory traffic	explicit <code>ldr/str</code>	implicit in many ops

The portable lesson: the *C semantics* are identical, but the generated code — and thus where a local physically resides and how many memory accesses it costs — depends on both the optimisation level and the ISA.

6.5 Stack overflow: the no-MMU hazard

On a PC, blowing the stack triggers a page fault and a clean crash. On a no-MMU MCU, the stack simply grows past its reserved region and *silently overwrites* `.bss` or the heap (look again at Figure 5.1). The symptoms are bizarre, far from the cause. Hence the embedded rules: bound your recursion (better: ban it — Chapter 9), avoid large local arrays (a 4 kB local buffer is 4 kB of stack), and budget per-task stacks carefully under an RTOS (Chapter 17).

Worked Example 6.1 How big is this function’s frame?

```

1 void process(void) {
2     uint8_t line[512];      /* 512 bytes ... */
3     char    fmt[64];       /* + 64 bytes ... */
4     /* ... */
5 }
```

This single function needs *at least* $512 + 64 = 576$ B of stack the instant it is entered, plus

saved registers and anything it calls. If it runs inside an RTOS task given a 512 B stack, it overflows on entry — before executing a single useful line. Local arrays are stack, and stack is scarce.

Exercise 6.1

You compile a function at `-O0` and the debugger happily shows all its locals. You raise it to `-O2` to fix a timing problem and now several locals read “<optimised out>”. Nothing is broken. Explain what changed and how you could force one specific local to remain inspectable.

Solution

At `-O2` those locals were promoted to registers (or folded away entirely), so they no longer have a stack slot for the debugger to read — hence “optimised out”. The C semantics are unchanged; only the storage changed. To keep one inspectable, take its address somewhere (`&x`), or declare it `volatile`, both of which force it to occupy real memory. (Or, of course, debug the function at a lower optimisation level.)

Key Idea

Chapter takeaway. Static data has a fixed address in `.data/.bss`; locals are stack *or* registers, and the optimiser decides which. RISC machines keep more in registers than CISC. Watch your stack: with no MMU, overflowing it corrupts other memory silently.

Chapter 7

The On-Chip Debug Unit

When firmware misbehaves there is no screen, no log file, and often no `printf` yet. How do you see inside a running chip? Through dedicated **debug hardware** baked into the silicon, separate from the CPU's normal function. On ARM Cortex-M this hardware is part of **CoreSight**, and understanding it turns debugging from guesswork into observation.

7.1 Why hardware debug exists

A PC debugs “software-only”: a debugger is just another process the OS schedules alongside the target. An MCU has no OS to host a debugger, and you need to debug code that may have crashed the CPU itself. So the chip carries a small block of logic that can *halt the core, read and write its registers and memory, set breakpoints, and stream trace* — all over a few external pins, even when the application code is dead.

7.2 The pieces (the CoreSight family)

Debug Access Port (DAP)

the gateway. An external probe (ST-Link, J-Link) talks to the DAP over the debug pins; the DAP gives it access to the chip's internal buses — it can read/write memory and registers *while the core runs or is halted*.

Debug pins: SWD vs JTAG

JTAG is the classic 4–5-wire interface (TCK, TMS, TDI, TDO, nTRST). **SWD** (Serial Wire Debug) is ARM's 2-wire replacement (SWDIO, SWCLK) that does the same job on far fewer pins — precious on small packages — and is the norm on Cortex-M.

Breakpoint Unit (FPB/BPU)

compares the program counter against a small set of programmed addresses and halts the core when one matches: a **hardware breakpoint**.

Data Watchpoint & Trace (DWT)

watches *data* addresses and halts (or counts/traces) on access: a **watchpoint**. Also holds cycle counters for profiling.

ITM / SWO

the **Instrumentation Trace Macrocell** lets firmware emit `printf`-style messages out a single pin (**SWO**, Serial Wire Output) with almost no CPU cost — a real “embedded printf”.

ETM the **Embedded Trace Macrocell** streams the *full instruction trace* for after-the-fact reconstruction of execution (when fitted).

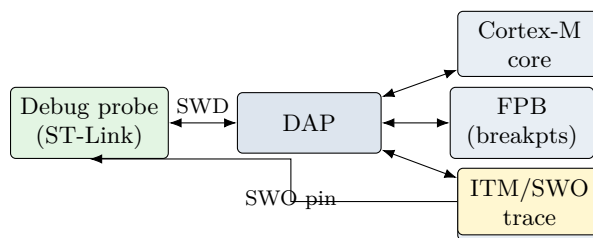


Figure 7.1: CoreSight debug architecture. A two-wire SWD link gives an external probe full access to the core, breakpoint and watchpoint units, while a single SWO pin streams trace out.

7.3 Hardware versus software breakpoints

There are two ways to make a program stop at a line, and the difference matters in firmware:

Hardware breakpoint

the FPB holds the address in a comparator; when the PC matches, the core halts. It needs *no change to the code*, so it works on code in *flash* — but there are only a handful of comparators (e.g. 6 on a Cortex-M4), so you can set only a few at once.

Software breakpoint

the debugger overwrites the instruction at the target address with a BKPT instruction and restores it afterwards. There is no count limit, but it only works in *writable* memory (RAM) — you cannot rewrite flash on the fly — which is why GDB on a flash-resident program leans on hardware breakpoints.

Watch Out

Run out of hardware breakpoints and GDB reports “cannot insert breakpoint — maybe too many hardware breakpoints?”. This is not a bug; it is the FPB’s finite comparators. Remove some breakpoints, or watch fewer addresses.

7.4 Watchpoints: stopping on data, not code

A **watchpoint** (via the DWT) halts the core when a *memory location* is read or written, regardless of which instruction did it. This is the tool of choice for the classic “some code is corrupting this variable and I have no idea who” bug — set a write-watchpoint on the variable’s address and the chip stops on the culprit, with the call stack intact.

7.5 What all this lets you do

With the debug unit and a tool like GDB + OpenOCD (or the IDE that wraps them), you can: halt and resume the core; single-step one instruction or one C line; read and write any register or memory location (including peripheral registers, live); set breakpoints and watchpoints; inspect the call stack and locals (subject to optimisation, Chapter 6); flash new firmware; and, crucially, attach to a chip that has already crashed and perform a *post-mortem* — read the fault status registers to find out *why* the HardFault fired.

7.6 Trace and “printf” without a UART

Halting changes timing, which is fatal when debugging a real-time control loop. Two non-intrusive options:

- **SWO/ITM trace:** write a byte to an ITM register and it streams out the SWO pin to the host, costing a few cycles and *never halting* the core. Retarget `printf` to ITM and you get logging in a hard-real-time loop.
- **Semihosting:** the target executes a BKPT that the *debugger* interprets as “please do this I/O on the host” (e.g. print this string). Convenient, but it *halts* the core for each call and only works with a debugger attached — so never ship it.

Worked Example 7.1 Catching memory corruption with a watchpoint

A global `config.crc` is occasionally wrong and you cannot find where it is written. In GDB:

```

1 (gdb) watch config.crc           # break when config.crc is written
2 (gdb) continue
3 Hardware watchpoint 2: config.crc
4 Old value = 0x4af2
5 New value = 0x0000
6 buggy_clear () at driver.c:88    # <-- the offending line, with full
    backtrace

```

The DWT halted the core the instant the value changed, pinning the exact instruction and call stack responsible — something no amount of `printf` sprinkling would have found cleanly.

Exercise 7.1

Your code lives in flash. You set 7 breakpoints in GDB and the 7th fails to install on a Cortex-M with 6 FPB comparators. A colleague suggests “just use software breakpoints, they’re unlimited.” Why does that advice fail here, and what actually works?

Solution

Software breakpoints work by *overwriting* the target instruction with BKPT, which requires writable memory. The code is in *flash*, which cannot be patched byte-by-byte at run time, so software breakpoints are unavailable for flash-resident code — hence GDB uses the FPB’s 6 hardware comparators and the 7th has nowhere to go. What works: use fewer simultaneous breakpoints, or replace some breakpoints with a watchpoint/condition, or (rarely) run the code from RAM where software breakpoints become possible.

Key Idea

Chapter takeaway. CoreSight gives you eyes inside a running (or dead) chip over two SWD wires: halt/step/inspect via the DAP, a few hardware breakpoints (FPB) and watchpoints (DWT), and non-intrusive logging via SWO/ITM. Hardware breakpoints work in flash but are scarce; watchpoints catch data corruption; trace lets you observe without stopping time.

Part III

Disciplined C for Embedded Systems

Chapter 8

Dynamic Memory Allocation and Why We Distrust It

On a PC, `malloc` and `free` are reflexes: need memory, ask for it, forget about it. In firmware they are viewed with deep suspicion, and in many shops they are outright banned (Chapter 9). To understand *why* — rather than just obey a rule — you have to know what `malloc` actually does under the hood, and how that collides with the embedded constraints of Chapter 1.

8.1 What `malloc` actually does

`malloc` manages a region of RAM called the **heap**. It hands out blocks on request and reclaims them on `free`. To do so it must track which parts of the heap are in use, which is done with bookkeeping *metadata* stored alongside your data.

- Each allocated block is typically preceded by a hidden **header** storing at least its size (and often flags/links). So `malloc(16)` consumes more than 16 bytes — header + your 16 + alignment padding.
- Free blocks are threaded onto a **free list**. On `malloc`, the allocator walks the list looking for a block big enough (*first-fit*: take the first that fits; *best-fit*: take the smallest that fits), and *splits* it if it is larger than needed.
- On `free`, the block is returned to the free list and, if its neighbours are also free, **coalesced** into one larger block to fight fragmentation.

Watch Out

The header lives *just before* the pointer `malloc` returns. Writing one byte before your buffer (`p[-1] = 0`) or one past its end corrupts the allocator’s metadata. The crash then happens later, inside the *next* `malloc/free`, far from the real bug — a heap corruption is one of the hardest faults to track down.

8.2 Where the heap lives, and `_sbrk`

Recall the RAM map of Figure 5.1: `.data`, then `.bss`, then the heap growing *up*, with the stack growing *down* from the top. The boundary between “heap in use” and “free” is the **program break**. When the allocator needs more raw memory it calls the system primitive `_sbrk`, which moves the break upward.

On a hosted OS, `_sbrk` is a kernel call. On bare metal with **newlib** (the `arm-none-eabi` C library) *you* provide `_sbrk`, and it typically looks like this:

```
1 extern uint8_t _end;           /* end of .bss, from the linker script */
2 extern uint8_t _estack;        /* top of RAM / stack base */
```

```

3 static uint8_t *brk = &_end; /* current program break */
4
5 void *_sbrk(ptrdiff_t incr) {
6     uint8_t *prev = brk;
7     if (brk + incr > __get_MSP() /* current stack pointer */)
8         return (void *) -1; /* heap would hit the stack: fail */
9     brk += incr;
10    return prev;
11 }

```

Listing 8.1: A typical bare-metal `_sbrk`.

Key Idea

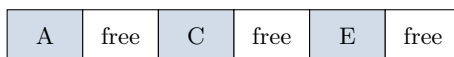
The heap and the stack grow toward each other in the *same* block of RAM. There is no MMU to police the boundary. If they meet, you either get a `malloc` failure (if `_sbrk` checks, as above) or — if it does not — silent corruption. This shared, unguarded space is the root of most embedded heap anxiety.

8.3 The four reasons we distrust the heap

8.3.1 1. Fragmentation

Even with plenty of *total* free memory, `malloc` can fail because no single *contiguous* block is large enough. This is **external fragmentation**, and it is the killer in long-running systems.

`malloc` of 2 blocks → **fails**



Total free = 3 blocks, but the largest contiguous run = 1 block.

Figure 8.1: External fragmentation: enough free memory exists, but not *contiguously*. A long-lived system that allocates and frees blocks of mixed sizes drifts into this state and eventually cannot satisfy a request it served happily an hour earlier.

8.3.2 2. Non-deterministic timing

The time `malloc` takes depends on the state of the free list: it may find a block immediately or walk a long list and split; `free` may coalesce. This variability is poison for a hard-real-time deadline, where you must bound the *worst case*.

8.3.3 3. Run-time failure with nowhere to go

`malloc` can return `NULL`. On a PC you log and exit. In a deeply embedded controller there may be no safe “exit”, and a `NULL` you forgot to check becomes a wild-pointer write (no MMU to trap it). Worse, the failure is *input-dependent and intermittent* — it appears only after the specific allocation history that fragments the heap, perhaps weeks into deployment.

8.3.4 4. Reentrancy / thread-safety

The allocator's free list is global mutable state. If an interrupt or another RTOS task calls `malloc` while a first call is mid-update, the list corrupts. newlib guards this with `__malloc_lock`/`__malloc_unlock` hooks you must implement under an RTOS — and you must *never* call `malloc` from an ISR (Chapter 2).

8.4 What we do instead

- **Static allocation:** declare everything at compile time. The memory footprint is then known exactly and checked by the linker — if it fits the link, it fits forever. (Chapter 9.)
- **Fixed-block pools:** if you truly need dynamic behaviour, pre-allocate an array of equal-sized blocks and hand them out from a free list of *identical* blocks. Equal sizes mean *no external fragmentation* and $O(1)$, deterministic alloc/free.
- **RTOS-managed heaps:** FreeRTOS ships several allocators (`heap_1...heap_5`); `heap_1` never frees (safest), `heap_4` coalesces. You choose your trade-off explicitly (Chapter 17).

Worked Example 8.1 A fragmentation time-bomb

A logger allocates a 100 B record and a 20 B tag in a loop, freeing only the records:

```

1 for (;;) {
2     char *rec = malloc(100);
3     char *tag = malloc(20);    /* never freed -- "small, who cares" */
4     use(rec, tag);
5     free(rec);                /* 100-byte holes open up ... */
6 }

```

The freed 100 B records leave a Swiss cheese of 100 B holes, but each new iteration also nibbles 20 B that is never returned. Over hours the heap fills with un-coalescable gaps and the 20 B tag eventually returns NULL — *despite* hundreds of freed bytes lying around. On a PC the process would be restarted nightly and nobody would notice; the device is expected to run for years.

Exercise 8.1

A junior engineer argues: “We have 64 kB of RAM and only ever hold 10 kB of live objects, so `malloc` can never fail.” Give the one-word phenomenon that refutes this, and state the single design change that makes the claim true.

Solution

Fragmentation. Live bytes never exceeding 10 kB does not guarantee a contiguous 10 kB (or even a contiguous large block) is available, because frees of mixed-size blocks scatter the free memory. The fix that restores the guarantee: allocate only *fixed-size* blocks from a pool (or allocate everything statically). With one block size, any freed block exactly fits any future request, so external fragmentation cannot occur.

Key Idea

Chapter takeaway. `malloc` keeps a free list with per-block metadata and grows the heap toward the stack via `_sbrk`. In embedded use it brings fragmentation, non-deterministic timing, hard-to-handle failures, and reentrancy hazards. Prefer static allocation or fixed-size

pools, whose worst case you can prove.

Chapter 9

C Constructs to Avoid in Embedded

C gives you a lot of rope. On a PC, sloppy-but-working code gets patched on the next release. In firmware — safety-critical, un-debuggable in the field, running for years — the cost of a latent defect is far higher, so the community has codified *defensive* subsets of C (most famously **MISRA C**). This chapter covers the specific rules in your syllabus, plus the closely related classics, and — more importantly — the reasoning behind each, because a rule you understand is a rule you apply correctly.

Key Idea

The unifying principle: *prefer code whose behaviour and resource use you can determine by reading it, without running it*. Every rule below removes a way for the program to do something surprising.

9.1 Always use braces, even for one-line blocks

This looks like a style nit. It is not; it has crashed real systems.

```
1 if (error)           /* NO braces */
2     log_error();
3     cleanup();       /* MISLEADING: runs ALWAYS, not only on error */
```

The indentation lies: `cleanup()` is not part of the `if`. Worse is the famous Apple “goto fail” TLS bug (2014), whose skeleton was:

```
1 if ((err = check_signature(...)) != 0)
2     goto fail;
3     goto fail;       /* a duplicated line, NOT guarded by any if */
4 /* ... security checks that are now skipped ... */
5 fail:
6     return err;
```

The second `goto fail` always executes, skipping the remaining checks, so *every* certificate was accepted. Mandatory braces make this impossible to write by accident:

```
1 if (error) {
2     log_error();
3 }
4 cleanup();           /* unambiguous */
```

Watch Out

The `goto fail` bug shipped in production and broke TLS for millions of devices. The fix is one mechanical rule: *braces on every if, else, for, while, no exceptions*. Cost: two characters.

9.2 No unlimited recursion

Each recursive call adds a stack frame (Chapter 6). With no MMU and a few kilobytes of stack, recursion whose depth depends on input is a stack overflow waiting to happen — and the overflow corrupts other RAM silently.

```

1  /* DANGER: depth = n, unknown at compile time */
2  unsigned long fact(unsigned n) {
3      return (n <= 1) ? 1 : n * fact(n - 1);
4  }
5  /* SAFE: bounded stack, trivially analysable */
6  unsigned long fact_iter(unsigned n) {
7      unsigned long r = 1;
8      for (unsigned i = 2; i <= n; i++) r *= i;
9      return r;
10 }
```

Iteration uses one frame regardless of *n*. If recursion is genuinely the clearest expression of an algorithm, it is tolerated *only* when the depth is bounded by a small compile-time constant (e.g. traversing a balanced tree of known maximum height).

9.3 No malloc/free

Chapter 8 made the full case. The rule: do not use the standard heap. Allocate statically; if you need pooling, use fixed-size blocks.

```

1  /* AVOID */                                /* PREFER */
2  char *buf = malloc(256);                  static char buf[256];
3  /* ... use ... */                        /* ... use ... (lifetime = program) */
4  free(buf);
```

The static version cannot fragment, cannot return NULL, costs zero run-time allocation, and its footprint is verified by the linker.

9.4 Prefer indexes to pointers for array traversal

Both of these sum an array:

```

1  /* pointer-walking */                      /* index-based */
2  int sum_p(const int *a, size_t n) {        int sum_i(const int *a, size_t n) {
3      int s = 0;                             int s = 0;
4      for (const int *p = a; p < a+n; ++p)   for (size_t i = 0; i < n; ++i)
5          s += *p;                             s += a[i];
6      return s;                               return s;
7  }                                           }
```

The index version is preferred in defensive embedded code because:

- the loop variable *i* is *checkable* against *n*, making a bounds check natural and the intent obvious;
- it avoids pointer arithmetic, a rich source of undefined behaviour (forming a pointer past one-past-the-end, or comparing pointers into different objects);
- it reads as “the *i*-th element”, which matches how the data is documented.

Note

This is a *readability and safety* preference, not a performance claim. On a good optimiser the two compile to nearly identical code (the compiler turns `a[i]` into pointer arithmetic anyway). MISRA's concern is human error and analysability, not speed. Where a tight DSP loop genuinely benefits from pointer walking, that is a deliberate, reviewed exception — not the default.

9.5 The other classics worth internalising

Use fixed-width types.

Prefer `uint32_t`, `int16_t` over `int/long`, whose widths vary by platform. Register fields and protocol formats are defined in exact bits.

Initialise every variable

at declaration. Reading an uninitialised automatic is undefined behaviour, and there is no OS to zero it for you.

Beware integer promotion and signedness.

`uint8_t a = 250, b = 10; a + b` promotes to `int` (so it is 260, not 4), while comparing a signed and unsigned value silently converts the signed one. These rules cause real bugs; know them.

No magic numbers.

`#define ADC_MAX 4095u` or an enum; never sprinkle bare 4095 through the code.

Parenthesise macro parameters

(and the whole body): `#define SQ(x) ((x)*(x))`, or `SQ(a+b)` expands to `a+b*a+b`. Better still, prefer `static inline` functions, which are type-safe and evaluate arguments once.

Avoid side effects in conditions.

`if (x = 5)` (assignment) versus `if (x == 5)` (comparison) — write constants on the left (`if (5 == x)`) so a typo'd = fails to compile.

goto only for forward error-cleanup.

The single tolerated use is jumping forward to a common cleanup label; never backwards, never into a block.

No variable-length arrays (VLAs).

`int buf[n];` with runtime `n` silently allocates an unbounded amount of stack — the recursion hazard in disguise.

Worked Example 9.1 The unsigned-underflow loop that never ends

```
1 void clear_backwards(uint8_t *a, size_t n) {
2     for (size_t i = n - 1; i >= 0; i--) /* BUG */
3         a[i] = 0;
4 }
```

`size_t` is *unsigned*, so `i >= 0` is *always true*; when `i` is 0 and decrements, it wraps to `SIZE_MAX` instead of going negative, the loop continues, and `a[SIZE_MAX]` writes far out of bounds — corrupting memory with no MMU to catch it. This is exactly the signedness trap above. Fixes: count up (`for (size_t i = 0; i < n; i++)`), or guard the wrap (`while (i- > 0)`). The index discipline and the signedness rule together would have flagged it in review.

Exercise 9.1

Rewrite this fragment to comply with the rules of this chapter, and name each violation you fix:

```
1 #define HALF(x) x/2
2 int *tmp = malloc(len * sizeof(int));
3 for (int i = 0; i < len; i++)
4     if (data[i] > 0)
5         tmp[i] = HALF(data[i] + 2);
```

Solution

Violations: (1) unparenthesised macro — `HALF(data[i]+2)` expands to `data[i]+2/2 = data[i]+1`, not the intended `(data[i]+2)/2`; (2) `malloc` on the heap; (3) missing braces on the `if` (and on the `for`). A compliant version:

```
1 static int tmp[LEN_MAX];      /* static, sized for the worst case */
2 for (size_t i = 0; i < len; i++) {          /* braces + size_t index */
3     if (data[i] > 0) {
4         tmp[i] = (data[i] + 2) / 2;          /* explicit, no macro */
5     }
6 }
```

(Using a `static inline int half(int x)` instead of the macro would also be acceptable and type-safe.)

Key Idea

Chapter takeaway. These rules are not bureaucracy; each closes a door on a class of field failures — the missing brace (`goto fail`), the unbounded stack (recursion/VLAs), the fragmenting heap (`malloc`), the silent wrap (signedness), the textual macro. Write code that can be *read* and *reasoned about*, because in the field you cannot run it.

Chapter 10

Dedicated Memory for I/O Buffers and DMA

Direct Memory Access (DMA) lets a peripheral move blocks of data to or from RAM *without the CPU copying each byte*. The ADC fills a buffer while the core sleeps; the UART drains a transmit buffer in the background. DMA is essential for throughput and power — but the moment hardware other than the CPU touches your memory, *where* that memory lives and *how* it is aligned stops being a detail and becomes correctness. This chapter is about the three traps and how dedicated memory areas avoid them.

10.1 What DMA is, briefly

A DMA controller is a little engine programmed with a source address, a destination address, and a length. Triggered by a peripheral event (“ADC conversion done”, “UART ready for a byte”), it performs the transfer over the bus, optionally generating an interrupt when finished. The CPU sets it up once and is then free. Because the transfer happens *concurrently* with the CPU, the buffer is shared mutable state between two masters — exactly the situation Chapter 2 warned about, now in hardware.

10.2 Trap 1 — the DMA must be able to reach the memory

Not all RAM is equal. Many STM32 parts have **Core-Coupled Memory (CCM)**, a fast SRAM wired *directly* to the core — and *not* on the bus matrix the DMA controller uses. Put a DMA buffer in CCM and the transfer silently fails or faults, because the DMA engine cannot see that address.

Key Idea

DMA buffers must live in RAM that the chosen DMA controller can physically address. CCM/TCM is great for the stack and hot variables, useless for DMA. The linker script and a placement attribute let you put each buffer in the right RAM.

10.3 Trap 2 — cache coherency

On cached cores (Cortex-M7 has a data cache; the smaller M0/M3/M4 do not), the CPU reads and writes go through the **D-cache**, while the DMA engine talks to *main RAM*. The two can disagree:

- **CPU → peripheral (TX):** you fill a buffer; the bytes sit in the cache, not yet written back to RAM. DMA reads RAM and sends *stale* data. Fix: *clean* (flush) the cache before starting the DMA.

- **Peripheral → CPU (RX):** DMA writes fresh data into RAM, but the CPU still has the old contents cached and reads the *stale* cached copy. Fix: *invalidate* the cache for that buffer after the DMA completes, forcing a re-read from RAM.

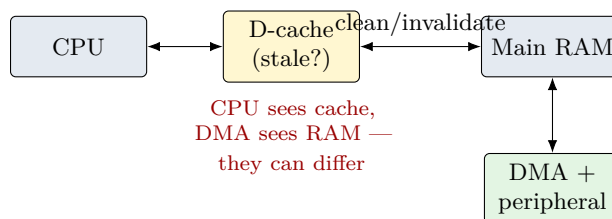


Figure 10.1: The coherency problem: the CPU works through the cache, the DMA works on RAM. Without cache maintenance (or a non-cacheable buffer) one side reads stale data.

The robust alternative to manual clean/invalidate is to mark the DMA buffer's region **non-cacheable** using the MPU, so CPU accesses bypass the cache entirely. Slower per access, but impossible to get a coherency bug.

10.4 Trap 3 — alignment

Two alignment requirements stack up:

- **Transfer-width alignment:** a DMA configured for 32-bit transfers wants a 4-byte-aligned buffer; a misaligned start may fault or transfer wrongly.
- **Cache-line alignment:** cache maintenance operates on whole lines (32 bytes on Cortex-M7). If a buffer shares a cache line with an unrelated variable, invalidating the line to refresh the buffer also throws away the neighbour's freshly written value — corrupting data you never touched. So DMA buffers are aligned to, and padded to a multiple of, the cache-line size.

10.5 Putting it together: a dedicated, safe buffer

You combine an alignment attribute with a section the linker places in DMA-reachable (and ideally non-cacheable) RAM:

```

1  /* 32-byte aligned, sized to a whole number of 32-byte cache lines,
2     and parked in a region the linker maps to non-cacheable DMA RAM. */
3  #define CACHE_LINE 32
4  __attribute__((aligned(CACHE_LINE), section(".dma_buffer")))
5  static uint8_t rx_buf[2 * 256]; /* ping-pong: two 256-byte halves */

```

Listing 10.1: A correctly placed, aligned DMA buffer.

The matching linker-script fragment carves out the region (see Chapter 5 for the syntax):

```

1  .dma_buffer (NOLOAD) : { . = ALIGN(32); *(.dma_buffer) } > RAM_D2

```

Double buffering (ping-pong). Notice the buffer holds *two* halves. While the DMA fills half A, the CPU processes the just-filled half B; when the DMA finishes A it interrupts, swaps, and fills B while the CPU works on A. This keeps a continuous stream flowing with no gaps and no tearing — a staple of audio and ADC streaming.

Watch Out

A DMA target buffer is written by hardware behind the compiler's back, so reads of it from C should be treated like any ISR-shared data: the compiler must not assume it is unchanged (*volatile* semantics, or a memory barrier plus cache invalidate). And never let a DMA buffer be a *local* array — its lifetime would end when the function returns while the DMA is still writing into that reclaimed stack space.

Worked Example 10.1 The “it works on the F4, breaks on the F7” bug

Code that streams ADC samples via DMA works perfectly on an STM32F4 (no cache) but returns stale, repeating samples on an STM32F7 (with D-cache). Why? On the F7, after the DMA writes new samples to RAM, the CPU reads its cached copy of the buffer — the *previous* contents. The buffer was never invalidated. Fix: after the transfer-complete interrupt, call `SCB_InvalidateDCache_by_Addr(buf, len)` before reading, *or* place `buf` in an MPU-defined non-cacheable region. The bug is invisible until you move to a cached core — which is exactly why “where the buffer lives” is part of the design, not an afterthought.

Exercise 10.1

You define `uint8_t dma_buf[20];` as a normal global on a Cortex-M7 and start a UART receive into it. Sometimes the first few bytes of an *adjacent* global, `status`, get clobbered after you invalidate the cache for `dma_buf`. Explain the mechanism and give the one-line fix.

Solution

`dma_buf` is only 20 bytes and unaligned, so it shares a 32-byte cache line with `status`. When you invalidate that line to refresh `dma_buf`, you also discard the cached (and more recent) value of `status`, which reverts to whatever was last in RAM — corruption of a variable you never DMA'd. Fix: force the buffer onto its own cache line(s): `__attribute__((aligned(32))) uint8_t dma_buf[32];` (aligned start *and* padded to a full line). Cache maintenance then touches only the buffer.

Key Idea

Chapter takeaway. A buffer shared with DMA must (1) live in RAM the DMA can reach, (2) be coherent with any data cache — clean before TX, invalidate after RX, or make it non-cacheable — and (3) be aligned and padded to a cache line so maintenance does not collateral-damage its neighbours. Dedicate the memory deliberately via attributes and the linker script.

Chapter 11

The `const` and `volatile` Modifiers

Two little keywords, endlessly misunderstood, and central to firmware. They are not opposites and they are not about the same thing: `const` is a *promise you make to the compiler*; `volatile` is a *warning the compiler makes to itself*. Get them right and your register accesses work and your ROM tables save RAM. Get them wrong and a polling loop hangs forever or a constant table eats precious SRAM.

11.1 `const`: “I promise not to write through this”

`const` tells the compiler that a value will not be modified *through this lvalue*. Two payoffs:

1. **Diagnostics:** an accidental write is a compile error, not a run-time corruption.
2. **Placement:** a `const` object with static storage can live in `.rodata` — i.e. in *flash*, not RAM (Chapter 5). On a chip with 2 kB of spare SRAM, moving a 1 kB lookup table to flash is the difference between fitting and not.

```
1  /* Lives in flash, costs zero RAM: */
2  const uint16_t sine_lut[256] = { /* ... precomputed ... */ };
3
4  /* Lives in RAM (no const), wastes 512 bytes of SRAM for read-only data: */
5  uint16_t sine_lut_bad[256] = { /* ... */ };
```

Pointer subtleties — read right-to-left.

```
1  const int *p;          /* pointer to const int: *p is read-only, p can move */
2  int * const p;          /* const pointer to int: p is fixed, *p is writable */
3  const int * const p;    /* const pointer to const int: neither can change */
```

The rule: read the declaration from the variable name outward. `const int *p` is the everyday “don’t let me modify what you lent me” used on function parameters: `size_t strlen(const char *s)` promises not to alter your string.

11.2 `volatile`: “this may change behind your back”

By default the optimiser assumes memory only changes when *your program* changes it, so it freely caches values in registers, removes “redundant” reads, and reorders accesses. `volatile` switches that off for an object: *every* access in the source becomes a real memory access, in program order, none elided.

There are exactly three situations that need it:

1. **Memory-mapped peripheral registers.** The hardware changes a status register; the value the CPU last wrote is not what is there now.

2. **Variables shared with an ISR** (Chapter 2). The ISR changes the variable asynchronously.
3. **Variables shared with another execution context** such as a DMA buffer (Chapter 10) or a second core.

Watch Out

`volatile` is *not* a synchronisation primitive. It guarantees the access *happens* and is not optimised away — it says *nothing* about atomicity or ordering relative to other variables. A 32-bit `volatile` on an 8-bit CPU still tears (Chapter 2); two `volatile` writes are ordered with respect to each other but not necessarily visible atomically to another core. For atomicity you still need a critical section or a true atomic operation. *Volatile means “re-read me”, not “protect me”.*

11.3 The classic `volatile` bug: the vanishing poll

```
1 uint32_t *const SR = (uint32_t *)0x40011000;    /* NOT volatile -- bug */
2 while ((*SR & READY) == 0) { }                /* wait for hardware */
```

The compiler reasons: “`*SR` is not modified inside the loop, so its value cannot change — I will read it once into a register and loop on that.” If the bit was 0 when first read, the loop spins *forever*, even though the hardware has long since set `READY` in the actual register. The fix is to tell the compiler the location is `volatile`:

```
1 volatile uint32_t *const SR = (volatile uint32_t *)0x40011000;
2 while ((*SR & READY) == 0) { }                /* now re-reads the register each time */
```

Note

A delay loop for `(volatile int i = 0; i < 1000; i++)`; uses the same mechanism: without `volatile` the optimiser deletes the empty loop entirely (it has no observable effect). `volatile` forces the iterations to actually occur. (That said, prefer a hardware timer to a calibrated busy-loop.)

11.4 `const volatile`: yes, both at once

A read-only *status* register is the canonical case: *you* must never write it (`const`), but it changes on its own (`volatile`). Both qualifiers apply:

```
1 const volatile uint32_t *const STATUS = (const volatile uint32_t *)0x40011000;
2 /* const    -> writing *STATUS is a compile error (it's read-only hardware) */
3 /* volatile-> every read fetches the live value from the peripheral          */
```

This is precisely how peripheral register structures are declared, which we formalise in Chapter 13.

Worked Example 11.1 `const` that saves RAM, measured

A font for a small display is 1.5kB of bitmap data. Declared `static const uint8_t font[]`, it lands in `.rodata` (flash) and the `size` tool shows it under `text`; SRAM usage is unchanged. Drop the `const` and the same array moves to `.data`: now 1.5kB of your scarce SRAM is consumed *and* 1.5kB of flash is still used to hold the initial values to copy at boot (Chapter 5). On a part with 8kB SRAM that single missing `const` can be the difference

between linking and “region RAM overflowed”.

Exercise 11.1

Classify each as needing `const`, `volatile`, `const volatile`, or neither: (a) a hardware *data* register you both read received bytes from and write transmit bytes to; (b) a compile-time lookup table of CRC constants; (c) a flag set by an ISR and cleared by `main`; (d) a loop counter local to one function at `-02`.

Solution

(a) `volatile` alone — it changes on its own *and* you write it, so it cannot be `const`. (b) `const` — never changes, belongs in flash. (c) `volatile` — changed asynchronously by the ISR; (and access it atomically, Chapter 2). (d) neither — it is private to one function and changes only through the program; `volatile` would just pessimise it by forcing needless memory traffic.

Key Idea

Chapter takeaway. `const` = “I won’t write this” (enables flash placement and catches mistakes). `volatile` = “re-read this every time, it changes behind my back” (required for registers, ISR-shared, and DMA data). They combine for read-only hardware. `volatile` is not a lock — it is about *visibility*, not *atomicity*.

Chapter 12

The Anatomy of .c and .h Files

C has no modules; it has *translation units* (.c files compiled separately) glued together by the linker, with .h files as the hand-maintained “contracts” between them. There is a right way to split code across these files, and following it prevents an entire genus of bugs — multiple-definition errors, mismatched function calls, and the dreaded implicit declaration. This chapter codifies the discipline.

12.1 The mental model: declaration versus definition

Declaration

introduces a name and its type: “foo is a function taking `int` and returning `int`.” Promises it exists somewhere. You may declare a thing many times.

Definition

actually creates it: allocates the storage for a variable, or provides the body of a function. Exactly *one* definition may exist across the whole program (the *One-Definition Rule*).

Headers carry *declarations*; source files carry *definitions*. That single sentence explains most of what follows.

12.2 What belongs in a header (.h)

A header is the public interface of a module — what *other* files need to use it. It should contain only things that are safe to include in many translation units:

- an **include guard** (below);
- type definitions: `typedef`, `struct/union/enum`;
- macro definitions (`#define`) and `static inline` functions;
- **function prototypes** for the module’s public functions;
- **extern declarations** of public global variables (the **extern** is essential — see below).

It must *not* contain function bodies or variable definitions (non-inline), because a header included by N source files would then create N definitions and the linker would reject the duplicates.

12.2.1 Include guards

A header may be reached more than once (A includes B and C; both B and C include D). Without protection, D’s contents would be pasted twice into A, re-defining its types — a compile error. The include guard makes the second inclusion a no-op:

```

1  #ifndef SENSOR_H          /* if not yet seen ... */
2  #define SENSOR_H          /* ... mark it seen ... */
3
4  #include <stdint.h>
5
6  typedef struct { int32_t raw; float celsius; } sensor_reading_t;
7
8  #define SENSOR_MAX_RAW 4095u
9
10 void sensor_init(void);          /* prototypes (declarations) */
11 int sensor_read(sensor_reading_t *out);
12
13 extern uint32_t sensor_error_count; /* DECLARE the global */
14
15 #endif /* SENSOR_H */          /* ... otherwise skip to here */

```

Listing 12.1: `sensor.h` skeleton with an include guard.

(The non-standard but ubiquitous `#pragma once` does the same job in one line; the `#ifndef` form is universally portable.)

12.3 What belongs in a source file (.c)

The source file is the *implementation*:

- it `#includes` its own header first (so the compiler checks the definitions against the declarations);
- it *defines* the public functions declared in the header;
- it *defines* the public globals (here is the one real definition the header’s `extern` promised);
- it holds private helpers and private state, marked `static` so they have **internal linkage** — invisible to other files.

```

1  #include "sensor.h"          /* check our definitions against our promises */
2
3  uint32_t sensor_error_count = 0; /* the ONE definition */
4
5  static int32_t last_raw;      /* private: internal linkage */
6  static int32_t convert(int32_t raw) { /* private helper, file-local */
7      return raw * 100 / SENSOR_MAX_RAW;
8  }
9
10 void sensor_init(void) { last_raw = 0; sensor_error_count = 0; }
11
12 int sensor_read(sensor_reading_t *out) {
13     /* ... read hardware ... */
14     out->celsius = (float) convert(out->raw);
15     return 0;
16 }

```

Listing 12.2: `sensor.c` matching the header.

Key Idea

`extern` in the header *declares* a global (“this exists”); the matching plain definition in exactly one .c *creates* it. Put the definition in the header by mistake and every file that

includes it defines the variable, giving a “multiple definition” link error.

12.4 Linkage: `static` and `extern`

Linkage decides whether a name is shared across translation units:

external linkage

(default for file-scope functions and globals): the name is visible to the linker and shareable across files.

internal linkage

(`static` at file scope): the name is private to its translation unit. Two files can each have a `static int count`; with no clash. Use `static` for everything not in the public header — it shrinks the interface and lets the compiler optimise more aggressively (it knows no one else calls it).

Watch Out

`static` is overloaded: at *file scope* it means internal linkage (private to the file); on a *local* variable it means static storage duration (persists across calls, lives in `.data/.bss`, not the stack). Same keyword, two unrelated meanings. Context tells you which.

12.5 Function prototyping is mandatory

A **prototype** is a function declaration that includes the parameter types — for instance `int sensor_read(sensor_reading_t *out);`. Why insist on it?

Without a visible prototype, a (pre-C99) compiler would *implicitly* assume the function returns `int` and accept *any* arguments unchecked. Call `sensor_read(3.14)` and nothing warns you — the bits of a `double` are reinterpreted as a pointer, and you get a wild access at run time (no MMU to catch it). With the prototype in scope (via the header), the compiler checks the argument count and types and converts them correctly, turning a run-time catastrophe into a compile error. Modern C makes calling an undeclared function an error, but the discipline — *every function declared in a header, every caller includes it* — is what guarantees the check actually happens.

Worked Example 12.1 How a missing prototype reinterprets your bits

`math_util.c` defines `double scale(double x)`. `main.c` forgets to include `math_util.h` and calls `int y = scale(2);`. With no prototype, the compiler passes the *integer* 2 where a `double` is expected and assumes `scale` returns `int`. At run time `scale` interprets the integer bit pattern as a `double` (a tiny denormal, not 2.0) and returns a `double` that the caller reads as an `int`: total garbage, no warning, crashes far away. Including the header makes the compiler convert 2 to 2.0 and read the result as `double` — or, if the types truly mismatched, refuse to compile.

Exercise 12.1

You add `int g_mode = 0;` (no `extern`) to `config.h` so several files can share it. The build fails at the *link* step with “multiple definition of `g_mode`”. Explain precisely why, and give the correct two-part fix.

Solution

`int g_mode = 0;` in a header is a *definition*, and the header is included by several .c files, so each translation unit defines its own `g_mode` — violating the One-Definition Rule, which the linker catches. Fix: put a *declaration* in the header, `extern int g_mode;`, and the single *definition* `int g_mode = 0;` in exactly one .c file. Now every file sees the same shared variable, defined once.

Key Idea

Chapter takeaway. Headers declare the contract (guards, types, macros, prototypes, `extern` globals); source files implement it (definitions, `static` privates). One definition per entity. Always prototype, and have callers include the header, so the compiler — not the field — catches type errors.

Part IV

Peripherals and the STM32 Ecosystem

Chapter 13

Peripherals in the Memory Map

We have repeatedly said “peripherals are memory-mapped registers”. Now we make that concrete, because it is the single most important idea in practical firmware: *every* timer, UART, ADC, and GPIO pin is controlled by reading and writing ordinary-looking addresses. Once you see how a C `struct` overlays the silicon’s register block, the vendor headers stop being magic and you can read any datasheet’s register map directly into code.

13.1 Memory-mapped I/O

A Cortex-M has a single, flat 32-bit address space (4 GB, 0x00000000–0xFFFFFFFF). Most of it is empty; specific ranges are wired to specific things. Crucially, a slab of the address space is connected not to memory cells but to *peripheral logic*: when the CPU executes a store to such an address, the bits land in a hardware register that *does something* — drives a pin high, sets a baud rate, starts a conversion. This is **memory-mapped I/O**: no special “port” instructions (as old x86 had), just loads and stores to magic addresses.

System / PPB	0xE000_0000	NVIC, SysTick (in PPB); GPIO, TIM, USART, ADC, SPI, I ² C all live in this band
External device		
Peripherals	0x4000_0000	
SRAM	0x2000_0000	
Code / Flash	0x0000_0000	

Figure 13.1: The Cortex-M address space (bottom = low address). The 0x4000_0000 region is where peripheral registers live; storing to an address there drives hardware directly.

13.2 Three flavours of register

Datasheets and your syllabus distinguish three roles a register can play, and each maps to a different access pattern:

Data register

carries the payload through the peripheral — the byte you transmit on a UART, the sample the ADC produced. Read and/or write.

Status register

reports the peripheral’s current state — “transmit buffer empty”, “conversion complete”, “error”. Typically *read-only* and changed *by the hardware*: the `const volatile` case from Chapter 11.

Control / configuration register

holds the settings *you* program — enable the peripheral, pick a baud rate, set an

interrupt-enable bit. Read/write.

A typical driver loop is: configure (control), wait on a flag (status), move the payload (data) — e.g. “set baud (CR), wait for TXE (SR), write the byte (DR).”

13.3 Getting at individual bits

Registers are dense: one 32-bit control register may pack a dozen independent fields. You manipulate bits with masks, never by overwriting the whole register (which would clobber the other fields):

```

1 #define TIM_CR1_CEN    (1u << 0)    /* counter enable bit */
2 TIMx->CR1 |= TIM_CR1_CEN;           /* set bit, leave others alone */
3 TIMx->CR1 &= ~TIM_CR1_CEN;          /* clear bit */
4 if (USARTx->SR & USART_SR_TXE) { /*...*/ /* test a status flag */
5 USARTx->CR1 = (USARTx->CR1 & ~MASK) | (val << SHIFT); /* multi-bit field */

```

Watch Out

Read-modify-write (`|=`, `&=`) on a hardware register is *not atomic* and can race an ISR that also touches the register (Chapter 2). Also beware “**write-1-to-clear**” status bits: for many flags you clear them by writing a 1, so a blind `SR |= x` can clear flags you did not mean to. Always read the bit’s access type in the datasheet.

13.4 The typedef-struct overlay

Here is the elegant trick that vendor headers (CMSIS) use. The registers of one peripheral sit at consecutive addresses, in a fixed order, starting at a **base address**. So you describe that layout once as a **struct** of **volatile** words — where each field’s *offset* matches the datasheet — then cast the base address to a pointer to that struct. Field access then compiles to exactly the right load/store.

```

1 typedef struct {
2     volatile uint32_t MODER;    /* offset 0x00: mode (in/out/alt/analog) */
3     volatile uint32_t OTYPER;   /* offset 0x04: output type */
4     volatile uint32_t OSPEEDR;  /* offset 0x08: output speed */
5     volatile uint32_t PUPDR;    /* offset 0x0C: pull-up/pull-down */
6     volatile const uint32_t IDR; /* offset 0x10: input data (READ-ONLY) */
7     volatile uint32_t ODR;      /* offset 0x14: output data */
8     volatile uint32_t BSRR;     /* offset 0x18: atomic bit set/reset */
9     /* ... */
10 } GPIO_TypeDef;
11
12 #define GPIOA_BASE 0x40020000u
13 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE) /* the magic cast */

```

Listing 13.1: How CMSIS overlays a struct on the GPIO register block.

Now `GPIOA->ODR = 0x20`; writes to address `0x40020000 + 0x14` — the output-data register — because `ODR` is the field at offset `0x14`. The compiler does the offset arithmetic; you write readable names. Note the `volatile` on every field (the hardware changes them) and `const` on the read-only input register `IDR` — the exact `const volatile` pattern from Chapter 11, now earning its keep.

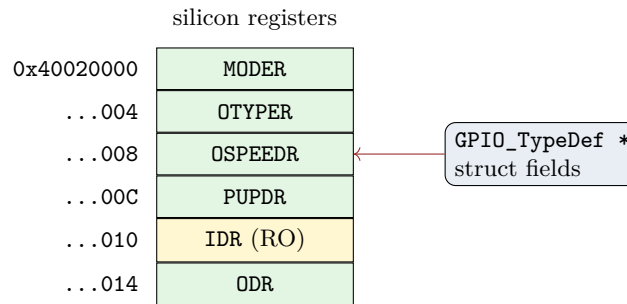


Figure 13.2: The struct's field offsets are laid exactly over the peripheral's register addresses. GPIOA->ODR resolves to base + 0x14.

Worked Example 13.1 Blinking an LED by writing registers

Toggle PA5 (the LED on many STM32 boards) with nothing but register writes:

```

1  /* 1. enable GPIOA clock (in RCC, another peripheral) */
2  RCC->AHB1ENR |= (1u << 0);
3  /* 2. set PA5 as a general-purpose output: MODER[11:10] = 01 */
4  GPIOA->MODER = (GPIOA->MODER & ~(3u << 10)) | (1u << 10);
5  /* 3. toggle forever */
6  for (;;) {
7      GPIOA->ODR ^= (1u << 5);    /* flip bit 5 */
8      for (volatile int d = 0; d < 200000; d++) { } /* crude delay */
9  }

```

No HAL, no library — just the memory-mapped model. Every line corresponds to a row in the reference manual. The BSRR register exists precisely so you can set/clear a single pin *atomically* (write a 1 to the right half-word) without the read-modify-write race on ODR. This bare-metal view is what the HAL (Chapter 15) wraps in friendlier functions.

Exercise 13.1

A datasheet shows a UART with SR at offset 0x00, DR at 0x04, and BRR (baud rate) at 0x08, base 0x40004400. Write the typedef struct (with correct qualifiers) and the base-pointer macro, then a line that waits for the TXE flag (bit 7 of SR) and writes a byte c.

Solution

```

1  typedef struct {
2      volatile uint32_t SR;    /* status: hardware-driven, but some bits
3                               are write-1-to-clear so not const here */
4      volatile uint32_t DR;    /* data */
5      volatile uint32_t BRR;   /* baud rate */
6  } USART_TypeDef;
7  #define USART2 ((USART_TypeDef *) 0x40004400u)
8
9  while (!(USART2->SR & (1u << 7))) { } /* wait for TXE */
10 USART2->DR = c;                       /* transmit byte */

```

Each field is volatile so the compiler re-reads/re-writes the hardware; the offsets match the struct member order (0, 4, 8). The base cast turns arithmetic into named fields.

Key Idea

Chapter takeaway. Peripherals are control/status/data registers at fixed addresses. Describe a peripheral's register block once as a **struct** of **volatile** words whose offsets mirror the datasheet, cast the base address to a pointer, and you get readable, correct hardware access. This overlay is the foundation every higher-level driver is built on.

Chapter 14

The STM32CubeIDE Workflow

Writing every register by hand (Chapter 13) is educational but impractical for a real project with dozens of pins, a complex clock tree, and USB or an RTOS to bring up. **STM32CubeIDE** is ST’s free, all-in-one environment that automates the tedious parts while leaving you in control of the logic. Knowing its workflow — and, importantly, where the boundary between “generated” and “your” code lies — is what the syllabus asks for.

14.1 What is in the box

STM32CubeIDE bundles four things that you would otherwise assemble yourself:

An Eclipse-based editor/IDE

for editing, building, and managing the project.

The GNU Arm toolchain

— the `arm-none-eabi-gcc` cross-compiler, linker, and `gdb` from Chapter 4, pre-configured.

STM32CubeMX

a *graphical configurator*: you click pins, clocks and peripherals, and it *generates* the initialisation C code.

A debugger front-end

driving the on-chip debug unit (Chapter 7) through an ST-Link probe — flash, break-points, watch windows, the works.

14.2 The configuration-first workflow

A CubeIDE project is created config-first, which is a different rhythm from a PC project:

1. **Pick the target.** Choose the exact MCU part number or a supported board (e.g. a Nucleo). This pins down the memory sizes, peripheral set, and a correct default linker script and startup file — the scaffolding of Chapters 4 and 5, done for you.
2. **Configure pins (the Pinout view).** Click a pin and assign its function (this pin is `USART2_TX`, that one is the LED GPIO). CubeMX shows conflicts in red — two peripherals fighting for one pin.
3. **Configure the clock tree.** A graphical diagram of oscillators, PLLs, prescalers and bus clocks. You set the desired core frequency and the tool solves the divider chain (or flags an impossible target). *This step is not optional*: every peripheral’s timing — baud rates, timer periods (Chapter 16) — derives from these clocks.
4. **Configure peripherals and middleware.** Enable and parameterise each peripheral (ADC resolution, SPI mode, timer prescaler), and switch on middleware: **FreeRTOS**, USB, FatFS, LwIP.

5. **Generate code.** CubeMX writes the project: startup file, linker script, `system_stm32xx.c` (clock setup), the HAL/LL drivers (Chapter 15), and `main.c` with all the `MX_<periph>_Init()` calls already wired up.
6. **Write your logic, build, flash, debug** — the normal edit/run loop, now on real silicon.

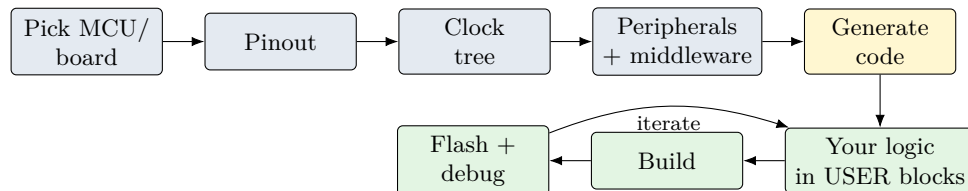


Figure 14.1: The CubeIDE loop: configure graphically, generate, then iterate on your logic.

14.3 The “USER CODE” contract

Here is the detail that bites everyone who skips it. CubeMX can *re-generate* the project whenever you change the configuration — adding a pin, say. To avoid destroying your hand-written code, the generated files are peppered with guarded regions:

```

1  /* USER CODE BEGIN 2 */
2  sensor_init();           /* <-- YOUR code goes between the markers */
3  /* USER CODE END 2 */

```

Watch Out

Code *between* `USER CODE BEGIN/END` markers survives re-generation; code *outside* them is overwritten without warning the next time you regenerate. The number-one beginner disaster is writing application code outside the markers, tweaking a pin in CubeMX, regenerating, and watching hours of work vanish. Stay inside the lines.

14.4 FreeRTOS integration

Enabling FreeRTOS in CubeMX (under Middleware) wires the kernel in for you: it adds the FreeRTOS source, configures the **CMSIS-RTOS** API wrapper, and — importantly — reassigns the system timebase. By default the HAL uses the `SysTick` timer for its millisecond tick; FreeRTOS also wants `SysTick` for scheduling, so CubeMX moves the HAL timebase to a spare hardware timer to avoid the clash. You then define tasks in the configurator (name, priority, stack size) and fill in their bodies in the USER blocks. We cover what to *do* with those tasks, queues and events in Chapter 17.

Note

CubeIDE generates *HAL*-based code by default (Chapter 15). It is ordinary C — you can read every generated line, mix in direct register access from Chapter 13, and even strip the HAL out entirely. The IDE accelerates bring-up; it does not hide the machine from you, and everything you learned in Parts I–III still applies underneath.

Exercise 14.1

After a week of work you add one extra GPIO in CubeMX and click “Generate Code”. Your carefully written initialisation in `main.c` disappears, but your code in a separate file `app.c` that you created is untouched. Explain both outcomes and state the rule that would have prevented the loss.

Solution

`main.c` is a *generated* file; your init code was placed *outside* the `USER CODE BEGIN/END` markers, so regeneration overwrote that region. `app.c` is your own file that CubeMX does not generate or manage, so it is never touched. The rule: put hand-written code only *inside* the `USER` markers of generated files, or — better for anything substantial — in your own separate `.c/.h` modules (Chapter 12) that the generator never rewrites.

Key Idea

Chapter takeaway. STM32CubeIDE = editor + Arm toolchain + graphical configurator (CubeMX) + debugger. Work config-first (chip → pins → clocks → peripherals/middleware → generate), keep your code inside the `USER CODE` markers or in your own modules, and let CubeMX manage the startup, linker script, and driver boilerplate you would otherwise hand-write.

Chapter 15

HAL vs. LL, Callbacks, and Weak Symbols

Chapter 13 showed peripheral control at the register level; Chapter 14 showed CubeMX generating drivers. Those drivers come in *two* flavours — HAL and LL — and you must know when to reach for which. This chapter also explains **callbacks** and the **weak symbol** mechanism that makes them (and the whole vector table) work, because the moment you handle your first interrupt you meet all three at once.

15.1 Two libraries, two philosophies

ST ships two driver layers on top of the bare registers:

HAL — Hardware Abstraction Layer

high-level and portable. You work with handle structures (`UART_HandleTypeDef`) and call functions like `HAL_UART_Transmit(&huart2, data, len, timeout)`. It hides register details and is largely consistent across the whole STM32 line, so code ports easily between chips. The cost: it is bulky, has call overhead, and its generality makes the exact timing hard to predict.

LL — Low-Layer

thin, mostly `static inline` wrappers that map almost one-to-one onto register operations: `LL_USART_TransmitData8(USART2, c)` compiles to essentially the register write you would have done by hand. Tiny and fast and deterministic — but closer to the metal and less portable.

Table 15.1: HAL versus LL.

	HAL	LL
Abstraction	high (handles, state machines)	minimal (register-level)
Code size / speed	larger, slower, more overhead	tiny, fast, inlined
Determinism	harder to bound	easy to bound
Portability	high across STM32 families	lower (closer to a part)
Ease of use	quick bring-up, friendly	you manage the details
Typical use	application logic, prototyping	ISRs, hot paths, tiny parts

Key Idea

Use HAL to bring a system up quickly and for code that is not timing-critical; drop to LL (or raw registers) on the hot paths — inside interrupt handlers, high-rate loops, or on a flash-starved chip — where you need speed, determinism, or size. They can coexist in one project.

15.2 Interrupt handling through HAL and LL

Recall (Chapter 2) that an interrupt vectors to a named handler. The two libraries split the work differently.

HAL: a three-layer dispatch. The vector-table entry `USART2_IRQHandler` calls `HAL_UART_IRQHandler(&huart2)`, a generic routine that reads the status register, figures out *which* event fired, clears the flag, advances its internal state machine — and then calls a **callback** for you to react:

```

1 void USART2_IRQHandler(void) {           /* in the vector table */
2     HAL_UART_IRQHandler(&huart2);        /* HAL decodes the cause... */
3 }
4 /* ...and eventually calls this, which YOU override: */
5 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
6     /* a byte/frame arrived -- handle it (keep it short!) */
7 }

```

You never touch a status bit; you just implement the callback. Convenient, but several function calls deep.

LL: you write the handler. With LL there is no dispatch layer — the IRQ handler *is* your code, checking and clearing flags directly:

```

1 void USART2_IRQHandler(void) {
2     if (LL_USART_IsActiveFlag_RXNE(USART2)) { /* check status */
3         uint8_t c = LL_USART_ReceiveData8(USART2); /* read data + clears flag */
4         ring_push(c);
5     }
6 }

```

Fewer cycles, full control, no hidden state — but you are responsible for every flag, exactly as in Chapter 13.

15.3 Callbacks: the concept

A **callback** is a function *you* write that the *library* calls when an event occurs — inversion of control again (Chapter 2). The library cannot know what your application wants to do when a byte arrives, so it exposes a named hook (`HAL_UART_RxCpltCallback`) and promises to call it. You provide the body; the library provides the timing.

But how can the HAL *call* a function you might not have written? If you do not implement `HAL_UART_RxCpltCallback`, the call would be an unresolved symbol and the link would fail. The answer is the weak symbol.

15.4 Weak symbols

A symbol marked `__attribute__((weak))` provides a *default* definition that the linker uses *only if no other (strong) definition exists*. If you define a function with the same name *without weak*, your strong definition silently overrides the weak one — no error, no configuration, it just wins.

The HAL ships an empty weak default for every callback:

```

1  /* In the HAL library: a do-nothing default. */
2  __attribute__((weak))
3  void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
4      (void) huart;          /* weak: silently does nothing unless overridden */
5  }

```

So the link always succeeds (the weak default satisfies the reference). If you write your own `HAL_UART_RxCpltCallback`, *yours* is linked and the empty one is dropped. You override exactly the callbacks you care about and ignore the rest.

Key Idea

Weak symbol = “use this definition *unless* someone provides a real one.” It lets a library reference a hook that may or may not be implemented: the weak default keeps the link valid, and your strong definition transparently replaces it when present.

The same trick powers the vector table. Every entry in the startup file is a weak alias to a single `Default_Handler` (an infinite loop):

```

1  void Default_Handler(void) { for (;;) { } }          /* catch-all */
2  void TIM2_IRQHandler(void) __attribute__((weak, alias("Default_Handler")));
3  void USART2_IRQHandler(void) __attribute__((weak, alias("Default_Handler")));

```

If you never use `TIM2`, its vector still points somewhere valid (the trap loop). The instant you define a strong `TIM2_IRQHandler`, the vector points at your code instead. This is why simply *defining* a function with the exact handler name “magically” installs an ISR — you are overriding a weak alias.

Watch Out

Override a weak callback by matching its name and signature *exactly*. A typo (`HAL_UART_RxCallback` instead of `...RxCpltCallback`) does not error — it just creates an unrelated function, leaves the empty weak default in place, and your handler is silently never called. “My callback never runs” is almost always a name/signature mismatch.

Worked Example 15.1 Why your ISR “does nothing”

A student copies a `TIM2` timer interrupt example but pastes the body into a function named `Timer2_IRQHandler`. It compiles, links, and runs — but the interrupt seemingly does nothing. The real vector slot, `TIM2_IRQHandler`, still resolves to the weak `Default_Handler` (which spins), and `Timer2_IRQHandler` is just dead code nobody calls. The fix is to name it *exactly* `TIM2_IRQHandler`, the symbol the startup file declared weak. The weak-override mechanism is unforgiving about spelling precisely because it works by name matching.

Exercise 15.1

You want a strong, fast UART receive ISR using LL, but the project was generated with HAL and already contains `void USART2_IRQHandler(void) { HAL_UART_IRQHandler(&huart2); }`. You add your own `void USART2_IRQHandler(void)` in another file and the link fails with “multiple definition”. Why — given weak symbols are supposed to allow overriding — and how do you resolve it?

Solution

The generated `USART2_IRQHandler` is a *strong* definition (CubeMX emits a real, non-weak handler), not a weak default. Two strong definitions of the same symbol is a multiple-definition error — weak overriding only works when exactly one definition is weak. Resolve it by editing the generated handler (inside its `USER CODE` markers) to call your LL code, or by removing/disabling the generated handler so only your strong definition remains. The override mechanism needs a weak target; here both were strong.

Key Idea

Chapter takeaway. HAL = portable and easy but heavy; LL = thin and fast but manual. Interrupts surface to you as *callbacks*, which work because the library defines them as *weak* defaults that your strong definition overrides — the same mechanism that lets you install ISRs just by naming them correctly. Match names and signatures exactly.

Chapter 16

The Timer Peripheral: Prescaler and Period

Timers are the most-used peripheral in firmware. They generate periodic interrupts (the system tick, control-loop trigger), produce **PWM** to drive motors and LEDs (Part VII), measure input pulse widths, and clock conversions. At heart a timer is just a *counter clocked at a known rate*, and two registers — the **prescaler** and the **auto-reload** — turn that counter into any frequency you need. Master those two knobs and timers stop being intimidating.

16.1 The counter and its two knobs

A timer counts pulses of an input clock f_{clk} . Two settings shape its behaviour:

Prescaler (PSC)

divides the input clock *before* it reaches the counter, so the counter ticks at $f_{\text{clk}}/(\text{PSC}+1)$. Use it to slow a fast bus clock down to a manageable counting rate.

Auto-reload (ARR, the “period”)

the value the counter counts *up to*. When the counter reaches ARR it generates an **update event** (typically an interrupt) and wraps back to 0.

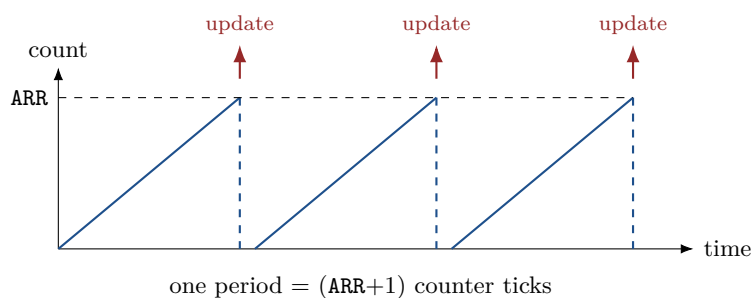


Figure 16.1: The timer counts up from 0 to ARR, fires an update event, and reloads. The prescaler sets how fast each tick is; ARR sets how many ticks per period.

16.2 The master formula

Combining both knobs, the update-event (overflow) frequency is

$$f_{\text{update}} = \frac{f_{\text{clk}}}{(\text{PSC} + 1)(\text{ARR} + 1)}$$

The “+1” on each register is the classic gotcha: the registers are *zero-based*, so a value of N means “divide by $N + 1$ ”. Forget it and every frequency is off by a little — maddening when you are calibrating a baud rate or a servo.

Worked Example 16.1 A 1 kHz control-loop tick from 72 MHz

You want a 1 kHz interrupt and the timer clock is $f_{\text{clk}} = 72 \text{ MHz}$. You need $(\text{PSC} + 1)(\text{ARR} + 1) = 72 \times 10^6 / 1000 = 72\,000$. A clean split: prescale by 72 (so $\text{PSC} = 71$), giving a 1 MHz counter, then count 1000 ticks ($\text{ARR} = 999$):

$$f_{\text{update}} = \frac{72 \text{ MHz}}{(71 + 1)(999 + 1)} = \frac{72 \times 10^6}{72\,000} = 1 \text{ kHz. } \checkmark$$

Choosing $\text{PSC} = 71$ to make the counter run at a round 1 MHz (1 μs per tick) is good practice: it makes ARR read directly in microseconds and leaves headroom in the 16-bit ARR .

16.3 What timers do for you

The same counter, lightly reconfigured, serves many roles:

- **Periodic interrupt:** fire the update interrupt every period — your RTOS tick, control loop, or scheduled sampling.
- **PWM output:** an additional *compare* register (CCR) sets a threshold; the output pin is high while $\text{count} < \text{CCR}$ and low after. The *duty cycle* is $\text{CCR} / (\text{ARR} + 1)$, while ARR sets the PWM *frequency*. This is how you dim an LED or set a motor's speed (Chapter 26).
- **Input capture:** latch the counter value on an input edge to *measure* a pulse width or external frequency.
- **Output compare / one-shot, encoder counting,** and triggering ADC/DMA at precise instants.

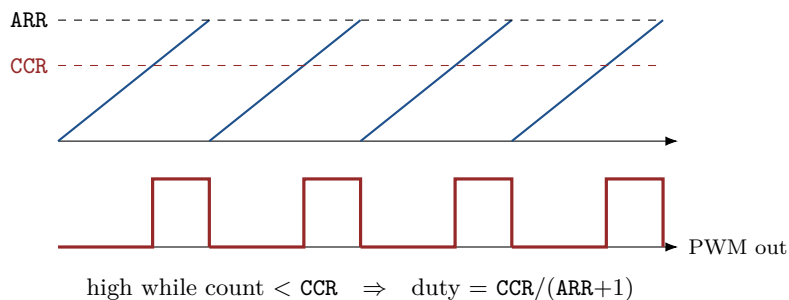


Figure 16.2: PWM generation. ARR fixes the period (frequency); the compare value CCR sets the high-time, i.e. the duty cycle.

Worked Example 16.2 Driving a hobby servo: 50 Hz, 1 ms to 2 ms pulses

A servo expects a 50 Hz PWM (20 ms period) where a 1 ms high-pulse means -90° and 2 ms means $+90^\circ$. From $f_{\text{clk}} = 72 \text{ MHz}$, prescale to a 1 MHz counter ($\text{PSC} = 71$, 1 $\mu\text{s}/\text{tick}$). For 20 ms set $\text{ARR} = 19\,999$ (20 000 ticks). Then the pulse width in microseconds equals CCR : $\text{CCR} = 1000$ gives 1 ms (-90°), $\text{CCR} = 1500$ centres it, and $\text{CCR} = 2000$ gives 2 ms ($+90^\circ$). The duty cycle is tiny ($1500/20000 = 7.5\%$) — for servos the *pulse width*, not the duty, carries the command. Writing one value to CCR sets the angle; the timer does the rest in hardware with zero CPU load.

Note

Timers come in 16-bit and 32-bit widths. A 16-bit ARR maxes at 65 535, which bounds how slow (long-period) a 16-bit timer can go for a given prescaler — sometimes you need a big prescaler or a 32-bit timer to reach very low frequencies. Conversely, a smaller prescaler gives finer time resolution per tick. Choosing the PSC/ARR split trades range against resolution.

Exercise 16.1

A timer is clocked at 84 MHz. (a) Find a PSC/ARR pair giving exactly 100 Hz. (b) Using your pair, what CCR gives a 25 % duty-cycle PWM? (c) Why might you prefer a prescaler that makes the counter tick at a round 1 MHz?

Solution

(a) Need $(\text{PSC} + 1)(\text{ARR} + 1) = 84 \times 10^6 / 100 = 840\,000$. Take $\text{PSC} = 83$ (counter at 1 MHz) and $\text{ARR} = 9999$ ($840000/84 = 10000$ ticks). Check: $84 \times 10^6 / (84 \cdot 10000) = 100$ Hz. ✓
(b) Duty = $\text{CCR} / (\text{ARR} + 1)$, so $\text{CCR} = 0.25 \times 10000 = 2500$. (c) A 1 MHz counter means each tick is exactly 1 μs , so ARR and CCR read directly in microseconds — easier to reason about and to calibrate, and it keeps values within the 16-bit range here.

Key Idea

Chapter takeaway. A timer is a counter clocked at f_{clk} ; the prescaler divides the clock and the auto-reload sets the count length, giving $f_{\text{update}} = f_{\text{clk}} / ((\text{PSC} + 1)(\text{ARR} + 1))$ — mind the +1s. A compare register turns the same counter into PWM, where ARR sets frequency and CCR sets duty. Timers underpin ticks, PWM, and measurement throughout firmware.

Part V

Concurrency and Numerics

Chapter 17

FreeRTOS Best Practices: Tasks, Queues, and Events

A bare-metal “super-loop” (do A, do B, do C, repeat) works until you have several activities with different rates and urgencies — a 1 kHz control loop, a chatty UART, a slow display, a button. Hand-scheduling these is fragile. A **Real-Time Operating System** like FreeRTOS lets you write each activity as an independent **task** and have a priority-based scheduler run them, blocking those with nothing to do so the CPU (or its sleep mode) is never wasted. This chapter is about using it *well* — the primitives are easy; the discipline is what keeps a multi-tasking system correct.

17.1 What the scheduler gives you

FreeRTOS is a **preemptive, priority-based** scheduler. At each tick (and on certain events) it runs the highest-priority task that is *ready*. A task can be in one of a few states:

Running

currently executing (only one per core).

Ready able to run, waiting for the CPU because something higher priority is running.

Blocked

waiting for an event (a queue item, a delay, a semaphore) — *consuming no CPU*.

Suspended

explicitly parked.

Key Idea

The central skill is to make every task *block* when it has no work, so the scheduler can run someone else (or sleep the chip). A task that busy-waits or polls in a tight loop starves lower-priority tasks and defeats the entire point of an RTOS. *Good RTOS code spends most of its time blocked.*

17.2 Tasks

A task is a C function that never returns; it loops, and inside the loop it *blocks* on something:

```
1 void sensor_task(void *arg) {
2     for (;;) {
3         /* block here until the next 10 ms tick -- zero CPU while waiting */
4         vTaskDelay(pdMS_TO_TICKS(10));
5         int t = read_sensor();
6         xQueueSend(temp_queue, &t, 0);    /* hand result to a consumer */
7     }
```

```
8 }
```

Listing 17.1: The canonical task shape.

Each task has its *own stack* (Chapter 6), sized when you create it. Get this wrong and the task overflows its stack into a neighbour — the silent, no-MMU corruption of Chapter 6. FreeRTOS offers a stack-overflow check hook and a high-water-mark API (`uxTaskGetStackHighWaterMark`) to right-size stacks; use them.

Assigning priorities. Give the tightest-deadline work the highest priority (the motor loop above the UI). But beware: a high-priority task that never blocks will *never* let lower ones run. Priority orders *who goes first*, not *who is more important to finish eventually*.

17.3 Talking between tasks: queues first

The cardinal rule of RTOS communication: **do not share global variables between tasks with ad-hoc locking; pass messages through queues**. A FreeRTOS **queue** is a thread-safe FIFO that *copies* items in and out and blocks the sender when full / the receiver when empty (with an optional timeout):

```
1 QueueHandle_t q = xQueueCreate(8, sizeof(reading_t)); /* 8-deep, by value */
2
3 /* producer task */ xQueueSend(q, &reading, portMAX_DELAY);
4 /* consumer task */ reading_t r; xQueueReceive(q, &r, portMAX_DELAY);
```

Because the queue copies the data and handles all the locking internally, the producer and consumer never touch shared memory directly — the entire class of race conditions from Chapter 2 simply does not arise.

17.4 Mutexes, semaphores, and priority inversion

When tasks *must* share a resource (a single SPI bus, say), protect it with a **mutex** (mutual exclusion lock). Mutexes differ from binary semaphores in one crucial way: they support **priority inheritance**.

Watch Out

Priority inversion: a low-priority task holds a mutex; a high-priority task blocks waiting for it; a *medium*-priority task (needing no mutex) then preempts the low task — so the medium task indirectly blocks the high one, *inverting* priorities. This bug famously nearly killed the Mars Pathfinder mission. A *mutex* (not a plain semaphore) fixes it via priority inheritance: while a high task waits, the low holder is *temporarily* bumped to the high priority so it finishes and releases the lock quickly. Use mutexes, not semaphores, for resource locking.

Use the primitives for their intended jobs: **mutex** for protecting a shared resource; **binary semaphore** for signalling “an event happened” (often from an ISR to a task); **counting semaphore** for managing a pool of N identical resources; **event groups** and **task notifications** for efficient flag-style synchronisation (notifications are the fastest, lightest option when signalling exactly one task).

17.5 The ISR boundary: deferred interrupt handling

Two iron rules connect Chapter 2 to FreeRTOS:

1. **From an ISR, call only the ...FromISR API variants** (`xQueueSendFromISR`, `xSemaphoreGiveFromISR`). The normal blocking versions must never be called in interrupt context — an ISR cannot block.
2. **Keep ISRs minimal; defer the work to a task.** The ISR does the bare minimum (grab the byte, clear the flag) and signals a task to do the real processing. This is *deferred interrupt handling*, and it keeps interrupt latency tiny for everyone.

```

1 void USART2_IRQHandler(void) {                               /* short ISR */
2     uint8_t c = USART2->DR;                                   /* read byte (clears flag) */
3     BaseType_t woken = pdFALSE;
4     xQueueSendFromISR(rx_queue, &c, &woken);                 /* hand off, FromISR variant */
5     portYIELD_FROM_ISR(woken);                               /* if a higher-prio task woke, switch now */
6 }
7 void uart_task(void *arg) {                                   /* heavy lifting in a task */
8     uint8_t c;
9     for (;;) {
10         xQueueReceive(rx_queue, &c, portMAX_DELAY);          /* block until a byte */
11         parse_protocol(c);                                    /* slow work, safe here */
12     }
13 }
```

Listing 17.2: ISR → queue → task: the standard pattern.

The `portYIELD_FROM_ISR(woken)` is important: if sending to the queue unblocked a task more urgent than the one that was interrupted, it requests a context switch on ISR exit so the urgent task runs *immediately* rather than after the current task's time slice.

Worked Example 17.1 Why moving work out of the ISR fixed the dropped bytes

A team handled UART bytes *entirely* inside the ISR, calling `parse_protocol()` (which occasionally does a slow CRC) right there. At high baud rates bytes were dropped. Why? While the long ISR ran, the next byte arrived and overran the receive register before the ISR finished — and it also delayed every other interrupt in the system. Re-architecting to the pattern above (ISR just enqueues; a task parses) cut the ISR to a few microseconds, so it always finished before the next byte, and the slow parsing happened in a preemptible task. The lesson from Chapter 2 — *ISRs do the minimum* — is enforced structurally by the RTOS.

Exercise 17.1

A high-priority `control_task` and a low-priority `log_task` both write to the same SD card via one SPI bus. Occasionally the high-priority task stalls for tens of milliseconds even though it is the most urgent. Name the likely mechanism and the specific FreeRTOS object that fixes it, and explain why the obvious “just use a binary semaphore” is the wrong choice.

Solution

The mechanism is **priority inversion**: `log_task` holds the bus lock, `control_task` blocks on it, and a medium-priority task preempts `log_task`, so `control_task` waits indirectly on the medium task. The fix is a **mutex** (created with `xSemaphoreCreateMutex`), which implements *priority inheritance*: while `control_task` waits, `log_task` is temporarily raised

to `control_task`'s priority so it finishes and releases the bus quickly. A plain binary semaphore offers no priority inheritance, so it would *not* bound the inversion — which is why mutexes, not semaphores, are the correct tool for protecting a shared resource.

Key Idea

Chapter takeaway. Model concurrent activities as tasks that *block* when idle; communicate by *copying through queues* rather than sharing globals; protect shared resources with *mutexes* (priority inheritance), and signal with semaphores/notifications. At the ISR boundary, use `FromISR` APIs and defer real work to tasks. Size every stack.

Chapter 18

Fixed-Point versus Floating-Point Arithmetic

Embedded code does a lot of number crunching — filtering sensor data, scaling ADC counts to engineering units, running control maths. *How* you represent fractional numbers has a big impact on speed, code size, accuracy, and even whether the maths runs in hardware or is emulated in software. The two choices are floating-point and fixed-point, and a good firmware engineer picks deliberately.

18.1 Floating-point: IEEE 754

A floating-point number stores a sign, an exponent, and a fraction (mantissa), representing $\pm 1.f \times 2^e$ — like scientific notation in binary. The **IEEE 754** formats you meet are:

- **float** — single precision, 32 bits: 1 sign, 8 exponent, 23 fraction. About 7 decimal digits of precision, range $\sim 10^{\pm 38}$.
- **double** — double precision, 64 bits: 1 sign, 11 exponent, 52 fraction. About 15–16 digits.

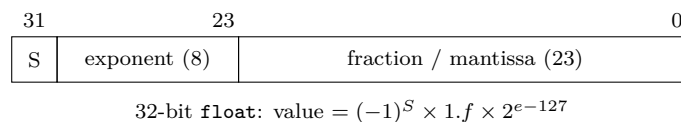


Figure 18.1: IEEE 754 single precision. The exponent gives floating-point its huge dynamic range; the mantissa gives a *relative* (not absolute) precision.

The strength is **dynamic range**: the same **float** comfortably holds 0.0001 and 1 000 000. The weakness is that precision is *relative* — big numbers have coarse spacing — and most values are not exactly representable (famously 0.1).

18.1.1 Hardware FPU versus software emulation

This is the part that surprises newcomers and the part the syllabus cares about:

- A Cortex-M4F or M7 has a **hardware FPU** — a **float** multiply is a single fast instruction (recall `-mfpu/ -mfloat-abi` from Chapter 4).
- A Cortex-M0/M0+/M3 has *no* FPU. Every **float** operation is then **emulated in software** by a library routine — tens to hundreds of cycles each. The code still works, but it is slow and bloats the image.
- Even on an M4F (which has only a *single*-precision FPU), a **double** is *still* emulated in software. Writing `3.14` (a **double** literal) instead of `3.14f` can silently drag in the slow software-double library.

Watch Out

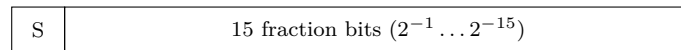
On a chip with a single-precision FPU, sprinkle the `f` suffix on float literals (`0.5f`, not `0.5`) and use `float` math functions (`sinf`, not `sin`). A stray `double` forces software emulation and can multiply a routine's runtime by $10\times$ while quietly adding kilobytes of library code. Also: `printf("%f", x)` pulls in a large floating-point formatter — often the biggest single chunk of a small image.

18.2 Fixed-point: integers with an implied scale

Fixed-point represents fractions using plain *integers* with an *implied* binary point. In the $Q_{m.n}$ format you reserve n bits for the fraction: the stored integer `raw` represents the real value

$$x = \frac{\text{raw}}{2^n}.$$

For example, **Q15** (a signed 16-bit value, $n = 15$) represents numbers in $[-1, +1)$ with a resolution of $2^{-15} \approx 3.05 \times 10^{-5}$ — the workhorse of audio and DSP. To store 0.75 in Q15: $0.75 \times 2^{15} = 24576$.



Q15: real value = `raw`/ 2^{15} , range $[-1, 1)$

Figure 18.2: Q15 fixed-point: one sign bit, an implied binary point, 15 fractional bits. No exponent — so uniform absolute resolution, but limited range.

18.2.1 Doing arithmetic

- **Add/subtract** two same-Q values: just add the integers (watch for overflow). `c = a + b;`
- **Multiply** two Q_n values: the product has $2n$ fraction bits, so you must shift back by n — and use a *wider* intermediate to avoid overflow:

```

1  /* Q15 * Q15 -> Q15, with a 32-bit intermediate */
2  int16_t q15_mul(int16_t a, int16_t b) {
3      int32_t p = (int32_t)a * b;    /* product is Q30 in 32 bits */
4      return (int16_t)(p >> 15);    /* shift back to Q15 */
5  }
```

18.2.2 The trade-offs

Fixed-point is **fast and deterministic on any CPU** (it is just integer math — no FPU needed, identical cost everywhere) and has *uniform* absolute precision. The price: *limited range* (overflow is a real risk you must manage with scaling and saturation), and *you* carry the mental burden of tracking the binary point and the shifts. It trades programmer convenience for predictability and hardware-independence.

Key Idea

Rule of thumb: on a Cortex-M4F/M7 with an FPU, use `float` (with the `f` suffix) — it is fast and easy. On an FPU-less M0/M3, or when you need bit-exact, deterministic DSP, or the smallest code, use fixed-point. Reserve `double` for when you genuinely need its precision

Table 18.1: Choosing a representation.

	Floating-point	Fixed-point
Dynamic range	huge ($10^{\pm 38}$)	narrow (set by m)
Precision	relative (7 digits)	uniform absolute (2^{-n})
Speed w/ FPU	fast (1 instruction)	fast (integer)
Speed w/o FPU	<i>slow</i> (emulated)	fast (integer)
Code size	libs can be large	minimal
Ease of use	natural, forgiving	manual scaling, overflow care
Determinism	good (with FPU)	excellent, identical everywhere

and can pay for the software emulation.

Worked Example 18.1 Scaling an ADC reading two ways

A 12-bit ADC (0–4095) measures 0 V to 3.3 V. Convert raw count to volts. *Float (M4F)*: `float v = raw * (3.3f / 4095.0f);` — one multiply on the FPU, trivially readable; note the `f` suffixes keep it single-precision. *Fixed-point (M0, millivolts)*: `uint32_t mv = (raw * 3300u) / 4095u;` — pure integer, exact to the millivolt, runs identically fast on any core, no library. If you needed 0.01-volt resolution without floats you would scale by 100 instead of 1000 and track the implied decimal. The float version is friendlier; the integer version is bullet-proof on the smallest chip.

Exercise 18.1

You port a working signal-processing routine from a Cortex-M4F board to a cheaper Cortex-M0+ to cut cost. Functionally it is correct, but it now misses its deadline by a wide margin and the binary barely fits in flash. (a) What is the root cause? (b) Give two distinct remedies, and note the trade-off of each.

Solution

(a) The M0+ has *no FPU*, so every `float/double` operation that was a single instruction on the M4F is now an emulated software routine — tens of times slower, and the emulation/formatting libraries bloat the image. (b) Remedy 1: **convert the hot maths to fixed-point** (e.g. Q15), which is plain integer arithmetic and fast on any core — trade-off: you must manage scaling and overflow, and accept limited range. Remedy 2: ensure everything is `float` not `double` (add `f` suffixes, use `sinf` etc.) and avoid `printf("%f")` — trade-off: it is still software-emulated on an M0+ (only smaller/faster than double), so it may help but not be enough; on a no-FPU part, fixed-point is usually the real fix.

Key Idea

Chapter takeaway. Floating-point (IEEE 754) buys huge dynamic range and ease, but only a hardware FPU makes it fast — and `double` is emulated even on single-precision FPUs. Fixed-point is integer math with an implied binary point: fast and deterministic everywhere, at the cost of range and manual scaling. Match the representation to the silicon and the requirements.

Part VI

Sensors and Serial Buses

Analog Sensor Interfacing: the NTC Thermistor

A sensor turns a physical quantity into an electrical one. Many are *analog and resistive*: their resistance varies with the thing you want to measure. To read one with a microcontroller you must (1) convert resistance into a *voltage* the ADC can sample, and (2) convert the digitised voltage back into engineering units in firmware. The **NTC thermistor** — a cheap, ubiquitous temperature sensor — is the perfect vehicle for the whole chain, which is exactly why it is in your syllabus.

19.1 What an NTC is

NTC stands for *Negative Temperature Coefficient*: its resistance *falls* as temperature *rises*. The relationship is strongly *nonlinear* — roughly exponential — which is both its strength (high sensitivity) and its complication (you cannot just scale it linearly).

A serviceable model is the β (**Beta**) **equation**, relating resistance R at absolute temperature T (in kelvin) to a known reference R_0 at T_0 (usually $25^\circ\text{C} = 298.15\text{ K}$):

$$R(T) = R_0 \exp\left[\beta\left(\frac{1}{T} - \frac{1}{T_0}\right)\right], \quad \text{equivalently} \quad \frac{1}{T} = \frac{1}{T_0} + \frac{1}{\beta} \ln \frac{R}{R_0}.$$

The second form is what firmware uses: measure R , get T . For higher accuracy the three-coefficient **Steinhart–Hart** equation $\frac{1}{T} = A + B \ln R + C(\ln R)^3$ is used, but β is enough to understand the principle.

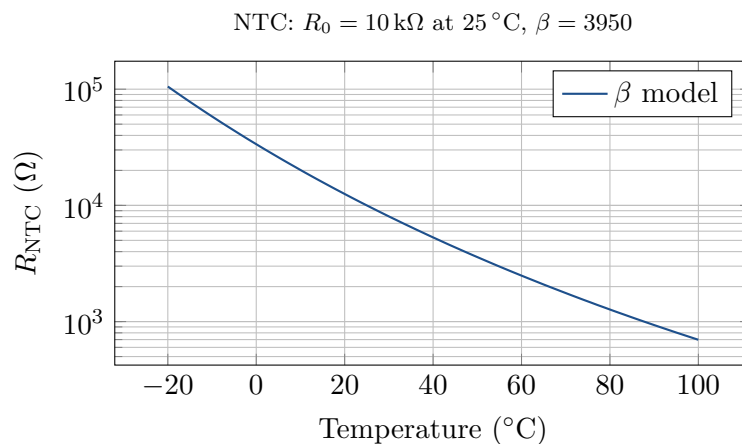


Figure 19.1: An NTC’s resistance versus temperature (log scale). It changes by orders of magnitude — highly sensitive but very nonlinear, so firmware must apply the inverse model.

19.2 Resistance to voltage: the divider

An ADC measures voltage, not resistance. The standard, near-free conversion is a **voltage divider**: put the NTC in series with a fixed resistor R_s across the supply, and tap the midpoint.

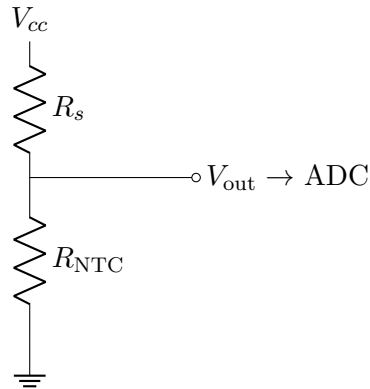


Figure 19.2: NTC voltage divider. As temperature rises, R_{NTC} falls, so V_{out} rises. The ADC samples V_{out} ; firmware inverts the divider to recover R_{NTC} , then the β model to recover T .

With the NTC on the bottom leg, the divider gives

$$V_{out} = V_{cc} \frac{R_{NTC}}{R_s + R_{NTC}} \implies R_{NTC} = R_s \frac{V_{out}}{V_{cc} - V_{out}}.$$

The right-hand form is the one firmware runs: from the measured V_{out} , recover R_{NTC} .

Key Idea

A *ratimetric* trick makes this robust: if the ADC's reference *is* V_{cc} , then the ADC code is $\text{code} = N_{\max} V_{out}/V_{cc}$, so V_{cc} *cancels* when you compute $R_{NTC} = R_s \cdot \text{code}/(N_{\max} - \text{code})$. Supply variation no longer affects the reading — you never even need to know V_{cc} accurately.

19.3 The full firmware chain

1. Sample the ADC: get `code` in $[0, N_{\max}]$ (12-bit: $N_{\max} = 4095$).
2. Recover resistance: $R_{NTC} = R_s \cdot \text{code}/(N_{\max} - \text{code})$.
3. Recover temperature via the inverted β model.

```

1 float ntc_temperature_C(uint16_t code) {
2     const float Rs = 10000.0f, R0 = 10000.0f;
3     const float BETA = 3950.0f, T0 = 298.15f, NMAX = 4095.0f;
4     float r = Rs * (code / (NMAX - code));          /* step 2 */
5     float invT = 1.0f/T0 + (1.0f/BETA) * logf(r / R0); /* step 3 */
6     return 1.0f/invT - 273.15f;                     /* K -> C */
7 }

```

Listing 19.1: NTC code-to-temperature (ratimetric).

(Note the `f` suffixes and `logf` — Chapter 18 — to stay single-precision. On a no-FPU part you would precompute a lookup table from code straight to temperature and skip the `logf` entirely.)

19.4 Design choices that matter

Pick R_s near R_{NTC} at your target temperature.

Divider sensitivity ($\partial V_{\text{out}}/\partial R$) is greatest when the two resistances are comparable. If you mostly care about room temperature, $R_s \approx R_0$ (here 10 k Ω) maximises resolution there.

Mind self-heating.

Current through the NTC dissipates power ($P = V^2/R$) and warms it — the sensor heats itself and reads high. Keep the divider current low (larger resistors), or sample briefly and power the divider only during conversion.

Linearise where it counts.

Because the curve is so nonlinear, resolution varies across the range. A lookup table or piecewise linearisation around the region of interest often beats evaluating the full equation, and is mandatory without an FPU.

Filter.

ADC readings are noisy; average several samples (or run a simple IIR filter) before inverting the nonlinear model, since nonlinearity amplifies noise unevenly.

Worked Example 19.1 From ADC count to degrees

A 12-bit ADC reads `code=1500` on the divider above ($R_s = 10\text{ k}\Omega$, $R_0 = 10\text{ k}\Omega$, $\beta = 3950$). Resistance: $R = 10000 \cdot 1500 / (4095 - 1500) = 10000 \cdot 1500 / 2595 \approx 5780\ \Omega$. Since $R < R_0$, it is warmer than 25 °C. Temperature: $\ln(5780/10000) = \ln(0.578) = -0.548$; $1/T = 1/298.15 + (-0.548)/3950 = 3.354 \times 10^{-3} - 1.39 \times 10^{-4} = 3.215 \times 10^{-3}$; $T = 311.0\text{ K} = 37.9\text{ °C}$. A plausible skin temperature — the whole pipeline in three lines of arithmetic.

Exercise 19.1

Your NTC divider works but the readings drift whenever the 3.3 V supply sags under load. A colleague suggests adding a precision voltage reference to measure V_{cc} . Describe a simpler fix that eliminates the sensitivity entirely, and explain why it works.

Solution

Use the ADC **ratiometrically**: drive the divider from the *same* node that is the ADC's reference (i.e. $V_{ref} = V_{cc}$). Then the ADC code is $\text{code} = N_{\text{max}} R_{\text{NTC}} / (R_s + R_{\text{NTC}})$ — V_{cc} appears in both the divider output and the reference and *cancels*. Computing $R_{\text{NTC}} = R_s \cdot \text{code} / (N_{\text{max}} - \text{code})$ then depends only on R_s and the code, not on the supply voltage, so sag no longer matters. No extra reference chip is needed.

Key Idea

Chapter takeaway. An analog resistive sensor is read by forming a voltage divider, sampling it with the ADC, then inverting — first the divider (R from code) and then the sensor model (T from R , via β or Steinhart–Hart). Make it ratiometric to cancel supply error, pick R_s for sensitivity, and watch self-heating and the nonlinearity.

Chapter 20

Digital-Interface Sensors: the Accelerometer

The NTC of Chapter 19 put all the work on you: a divider, an ADC channel, noise filtering, and nonlinear maths. A **digital-interface sensor** flips that around: it contains its own ADC, calibration, and logic, and hands you ready-made numbers over a serial bus (SPI or I²C, Chapter 21). The MEMS **accelerometer** is the textbook example — and its use in hard-disk drives is a beautiful illustration of why sensors earn their place in a product.

20.1 Why a digital interface is nicer

- **Noise immunity:** the analog signal is digitised *inside* the sensor, next to the transducer, so the fragile millivolt-level signal never travels across a noisy board — only robust digital bits do.
- **Calibration on-chip:** the sensor is trimmed at the factory and reports in real units (e.g. milli-*g*); you do not characterise each device.
- **Configurability:** registers let you set the full-scale range, output data rate, and filters at run time.
- **Smart interrupts:** the sensor can watch *itself* and interrupt the MCU only on a condition — motion, tap, or *free-fall* — so the CPU sleeps until something happens.
- **Many on one bus:** several digital sensors share two or four wires (Chapter 21), versus one ADC channel each.

20.2 What a MEMS accelerometer measures, briefly

Inside is a microscopic **proof mass** on silicon springs. Acceleration displaces the mass; the displacement changes a tiny capacitance, which on-chip electronics turn into a number. Crucially, it senses the *gravitational* field too: at rest on a desk it reads 1 *g* ($\approx 9.81 \text{ m/s}^2$) on the vertical axis. A 3-axis device reports a_x, a_y, a_z .

displacement $\propto a \rightarrow$ capacitance change $\rightarrow$ digital value

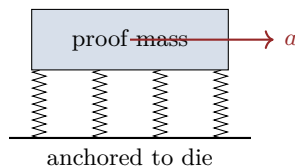


Figure 20.1: A MEMS accelerometer: acceleration deflects a spring-mounted proof mass; the resulting capacitance change is digitised on-chip and read over a serial bus.

20.3 The register interface

You talk to it exactly like the memory-mapped peripherals of Chapter 13, except the “bus” is SPI/I²C instead of the internal one. Typical registers:

- WHO_AM_I — a fixed ID byte; read it first to confirm the device is present and the bus works.
- CTRL_REG* — enable axes, set data rate and full-scale range.
- OUT_X/Y/Z — the latest acceleration samples.
- INT_CFG/INT_SRC — configure and read the cause of the sensor’s interrupt (motion, free-fall threshold, etc.).

20.4 The hard-disk-drive free-fall story

A spinning hard-disk drive has read/write heads “flying” *nanometres* above platters turning at thousands of RPM. A sharp impact can slam a head into the platter — a *head crash* — destroying data. The clever defence: detect that the laptop is *falling* and park the heads to a safe ramp before it lands.

How do you detect a fall? Here is the elegant physics: an accelerometer in *free fall* reads $\approx 0g$ on *all* axes. Normally gravity shows up as a persistent 1g on whichever axis points down; but a body in free fall accelerates *with* gravity, so the proof mass and its frame fall together and there is no relative deflection — the sensor goes weightless. So the detector watches the *magnitude*

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

and triggers when it drops well below 1g for a few milliseconds. Modern accelerometers implement this as a built-in **free-fall interrupt** that fires within milliseconds — comfortably faster than the tens to hundreds of milliseconds a short drop gives you — so the firmware’s ISR just issues the “park heads” command.

Worked Example 20.1 A free-fall detector in firmware

```

1  /* Called when the accelerometer's free-fall interrupt fires (or polled). */
2  bool in_free_fall(int16_t ax, int16_t ay, int16_t az) {
3      /* values in milli-g; 1 g normally present on some axis */
4      int32_t mag2 = (int32_t)ax*ax + (int32_t)ay*ay + (int32_t)az*az;
5      const int32_t THRESH_mg = 300;                               /* ~0.3 g */
6      return mag2 < THRESH_mg * THRESH_mg;                       /* compare squared, no
7      sqrt */
8  }
```

Two firmware touches worth noting: we compare the *squared* magnitude against a squared threshold to avoid a costly `sqrt` (and any FPU dependence, Chapter 18); and the widened `int32_t` products prevent the overflow that squaring 16-bit values would cause (Chapter 9). Better still, let the sensor’s hardware free-fall interrupt do the detection and wake the MCU — the CPU need not poll at all.

Exercise 20.1

Compare reading temperature with the NTC of Chapter 19 versus reading acceleration with this digital accelerometer, in terms of (a) what hardware the MCU needs, (b) where the

calibration lives, and (c) how the MCU can stay asleep until something interesting happens.

Solution

(a) The NTC needs an analog front end — a divider plus an ADC channel — and firmware maths; the accelerometer needs only a digital bus (SPI/I²C) and no ADC. (b) The NTC is uncalibrated: *you* supply R_0 , β (or Steinhart coefficients) and invert the model in firmware; the accelerometer is factory-calibrated and reports real units directly. (c) The NTC is passive — the MCU must wake, power the divider, sample, and compute periodically; the accelerometer has *smart interrupts* (motion/free-fall), so it can watch itself and wake the MCU only on a threshold event, letting the CPU sleep the rest of the time. Digital sensors move work — and watchfulness — off the MCU.

Key Idea

Chapter takeaway. A digital-interface sensor packs the ADC, calibration, configuration, and even event detection inside itself, exposing registers over SPI/I²C. You read calibrated numbers and let the sensor's own interrupts wake the MCU. The HDD free-fall detector — spot $\sim 0g$ on all axes and park the heads — shows a sensor turning physics into a few milliseconds of life-saving warning.

Chapter 21

The I²C and SPI Protocols

The digital sensors of Chapter 20 have to talk to the MCU *somehow*. The two dominant short-range buses are **SPI** and **I²C**. Both are *synchronous* (a clock line tells the receiver when to sample, unlike the clock-less, async UART), both are built into every STM32, and choosing between them is a routine design decision. This chapter covers how each works on the wire, their trade-offs, and how the STM32 peripheral presents them.

21.1 SPI — Serial Peripheral Interface

SPI is fast, simple, and full-duplex. One **controller** (master) drives up to four wires:

- SCLK** the clock, generated by the controller.
- MOSI** Controller Out, Peripheral In — data master→slave.
- MISO** Controller In, Peripheral Out — data slave→master.
- $\overline{\text{CS}}$** Chip Select, one per peripheral, active-low.

There is *no addressing* and *no acknowledgement*: the controller selects a device by pulling its $\overline{\text{CS}}$ low, then clocks bits. Internally it is just two shift registers connected in a ring — on each clock edge a bit goes out MOSI and a bit comes in MISO, so a transfer is inherently an *exchange*: to read, you also write (often dummy bytes).

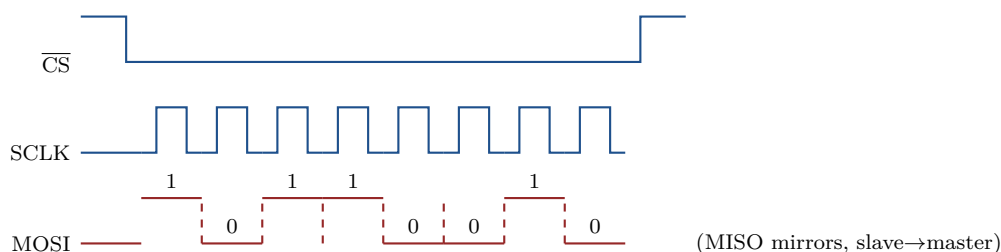


Figure 21.1: SPI transfer. $\overline{\text{CS}}$ frames the byte; eight SCLK pulses clock eight bits on MOSI (and simultaneously eight in on MISO). No address, no ACK — selection is by the dedicated chip-select line.

Clock polarity and phase (CPOL/CPHA). SPI has four *modes* (0–3) set by two bits: **CPOL** (idle clock high or low) and **CPHA** (sample on the first or second edge). Controller and peripheral *must* agree, or you read garbage. The datasheet of every SPI device states its required mode; a mismatched mode is the classic “SPI returns 0x00 or 0xFF” bring-up bug.

21.2 I²C — Inter-Integrated Circuit

I²C trades speed for wires: just *two*, shared by every device on the bus.

SDA serial data (bidirectional).

SCL serial clock (driven by the controller).

Both lines are **open-drain** with external **pull-up resistors**: a device can only pull a line *low* (or release it to float *high*). This is what lets many devices share the wires without conflict, and it enables acknowledgement and clock stretching. Because there is no chip-select line, each peripheral has a **7-bit address**; the controller broadcasts the address and only the matching device responds.

A transaction is framed by special conditions on the otherwise data-carrying SDA line:

- **START**: SDA falls *while SCL is high* (normally SDA only changes while SCL is low).
- **Address + R/ $\overline{W}$** : 7 address bits + 1 direction bit.
- **ACK/NACK**: after each byte the *receiver* pulls SDA low for one clock to acknowledge (or leaves it high to NACK). This is the handshake SPI lacks.
- **Data bytes**, then a **STOP**: SDA rises while SCL is high.

A peripheral that needs time can hold SCL low — **clock stretching** — pausing the controller until it is ready.

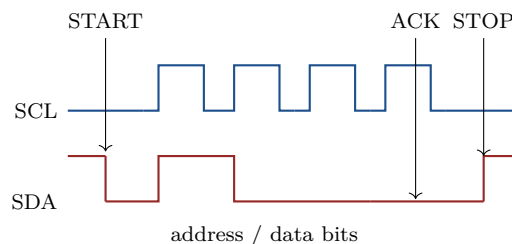


Figure 21.2: I²C framing. START (SDA falls while SCL high), then addressed bytes each followed by an ACK pulled low by the receiver, then STOP (SDA rises while SCL high). Two wires, many devices, with a built-in handshake.

21.3 Side by side

Key Idea

Choose SPI for raw speed and full-duplex streaming (SD cards, displays, high-rate ADCs), accepting an extra wire per device. **Choose I²C** when pins are scarce and you have many slow-ish devices, accepting lower speed and a fussier protocol. Many boards use both: SPI for the fast stuff, I²C for the housekeeping sensors.

21.4 In the STM32: the peripheral view

On-chip SPI and I²C controllers are just more memory-mapped peripherals (Chapter 13): control registers to set mode/speed, status registers with “TX empty”/“RX not empty”/“busy” flags, and a

Table 21.1: I²C versus SPI — the cheat sheet.

	SPI	I ² C
Wires	3 + 1 per peripheral ($\overline{\text{CS}}$)	2 total (SDA, SCL)
Duplex	full (simultaneous in/out)	half
Typical speed	tens of MHz	100 kHz/400 kHz/1 MHz
Device selection	dedicated chip-select line	7-bit address on the bus
Acknowledgement	none	per-byte ACK/NACK
Pull-ups needed	no	yes (open-drain)
Pins for N devices	3 + N	2 (shared)
Complexity	trivial protocol	START/STOP/ACK/stretch
Best for	fast streams (ADC, flash, display)	many slow sensors, few pins

data register you write/read. They support interrupts and DMA (Chapter 10) so large transfers run in the background. The HAL wraps them (Chapter 15) as `HAL_SPI_TransmitReceive(&hspi, tx, rx, n, timeout)` or `HAL_I2C_Mem_Read(&hi2c, addr, reg, ...)`; LL gives you the register-level versions for tight ISRs.

Worked Example 21.1 Reading WHO_AM_I, both ways

Confirming a sensor is alive is always step one. Over **SPI**, you select the chip, send the register address (with the read bit set), then clock a dummy byte to shift the answer back:

```

1  uint8_t tx[2] = { REG_WHO_AM_I | 0x80, 0x00 }; /* 0x80 = read flag */
2  uint8_t rx[2];
3  cs_low();
4  HAL_SPI_TransmitReceive(&hspi1, tx, rx, 2, 10); /* full-duplex exchange */
5  cs_high();
6  uint8_t id = rx[1]; /* the ID byte */

```

Over **I²C**, there is no chip-select — you address the device, write the register pointer, then read, with the HAL hiding the START/ADDR/ACK/STOP dance:

```

1  uint8_t id;
2  HAL_I2C_Mem_Read(&hi2c1, DEV_ADDR << 1, REG_WHO_AM_I,
3                  I2C_MEMADD_SIZE_8BIT, &id, 1, 10);

```

Same goal, same register inside the sensor — but SPI frames it with a chip-select and a full-duplex exchange, while I²C frames it with an address and ACKs. (Note `DEV_ADDR << 1`: the STM32 HAL wants the 7-bit address shifted into the upper bits, leaving room for the R/ $\overline{\text{W}}$ bit — a perennial source of “device not responding” confusion.)

Exercise 21.1

You must connect eight identical temperature sensors to one MCU that has very few spare pins. Each sensor supports *both* SPI and I²C. (a) Which bus minimises pin count, and how many pins does it need versus SPI? (b) What feature of that bus makes connecting eight *identical* parts potentially problematic, and how is it usually solved?

Solution

(a) **I²C**: all eight share just two pins (SDA, SCL) plus the two pull-ups. SPI would need 3 shared lines *plus* a separate $\overline{\text{CS}}$ per device = 3 + 8 = 11 pins — far more. (b) I²C selects devices by *address*, so eight *identical* parts would share the same default address and collide. The usual solutions: parts that expose one or more address-select pins (strap them to give each a unique address), choosing a part family that offers several addresses, or — if neither

— an I²C multiplexer. This addressing constraint is precisely the trade-off for using only two wires.

Key Idea

Chapter takeaway. SPI = 4 wires, full-duplex, fast, chip-select selection, no ACK, mind CPOL/CPHA. I²C = 2 open-drain wires with pull-ups, addressed, per-byte ACK, slower, with START/STOP framing and clock stretching. Both are memory-mapped STM32 peripherals with HAL/LL drivers and DMA; pick SPI for speed, I²C for pin economy.

Part VII

Power Electronics for Firmware Engineers

Chapter 22

Power Electronics from Scratch

This part hands firmware a new job: moving real *power* in the physical world. The four chapters that follow — load switches, smart high-side drivers, dissipation and base-resistor sizing, and full-bridges — all lean on the same small set of device-physics ideas. If those ideas are not solid, the later chapters read as a wall of V_{GS} , β , “saturation” and “charge pump” jargon. So this chapter builds them from the ground up. Nothing here is new syllabus; it is the floor the rest of the part stands on. Read it once and the later chapters become bookkeeping.

22.1 A transistor is a controlled valve

Everything in this part exists to solve one mismatch. A microcontroller pin is a *logic* output — a few milliamps at 3.3 V. The things firmware wants to switch — motors, heaters, lamps, solenoids — want *amperes*, often at 12 V to 48 V or more. You cannot drive the load from the pin. Instead you drive a **transistor**, and the transistor drives the load.

The mental model that makes the rest of this part easy is a **valve**. A valve has a big pipe carrying the real flow and a small handle that opens or closes it. Working the handle takes almost no effort; the pipe carries all the water. A transistor used as a switch is exactly this:

- the **main path** — drain–source in a MOSFET, collector–emitter in a BJT — is the big pipe. It sits in series with the load and the supply and carries the load current.
- the **control terminal** — the *gate* of a MOSFET, the *base* of a BJT — is the handle. The MCU pin works it, spending almost nothing.

The pin never carries the load current. It only tells the valve to open or shut (Figure 22.1).

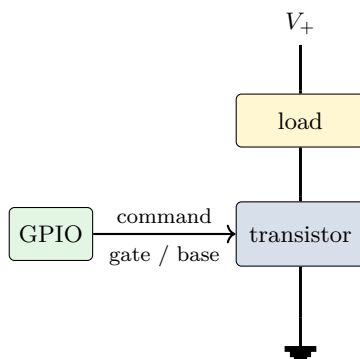


Figure 22.1: A switching transistor as a controlled valve: the GPIO works the handle (gate/base) with almost no current, while the main path (drain–source / collector–emitter) carries the load current between V_+ and ground. Here the switch is *below* the load — a low-side arrangement (Chapter 23).

22.2 Two kinds of valve: voltage-operated vs current-operated

There are two kinds of handle, and they work on different physical quantities. This single distinction explains most of the choices in the coming chapters.

MOSFET — a *voltage-operated* valve. A MOSFET opens when the *voltage* on its gate, measured against its source, exceeds a threshold V_{th} . The gate is one plate of a tiny capacitor; once it is charged, holding the valve open costs essentially *no* steady current. A “logic-level” MOSFET is simply one whose V_{th} is low enough that a 3.3 V (or 5 V) logic high opens it fully.

BJT — a *current-operated* valve. A bipolar transistor opens in proportion to the *current* you push into its base. In its “amplifier” (active) region the collector current is $I_C = \beta I_B$, where the current gain β is typically 50 to 200. Holding it open costs a continuous base current.

That contrast — voltage vs current, free-to-hold vs costs-current-to-hold — is why MOSFETs have largely displaced BJTs for switching, and why sizing a BJT’s base drive (Chapter 25) is a real design step whereas a MOSFET’s gate is almost free.

22.3 What actually turns a MOSFET on

Here is the single most useful fact in this part, and the one most often glossed over: a MOSFET cares about the voltage from its gate *to its own source*, $V_{GS} = V_G - V_S$ — **not** the gate voltage measured against ground. The channel opens when

$$V_{GS} = V_G - V_S \geq V_{th}.$$

So the only question that ever matters is: *where is the source sitting?*

- If the source is at **ground** — the transistor sits *below* the load, a *low-side* switch — then V_{GS} is just the gate voltage, and a 3.3 V pin delivers 3.3 V of V_{GS} . Easy.
- If the source floats up near V_+ — the transistor sits *above* the load, a *high-side* switch — then to get the same V_{GS} the gate must be driven *above* V_+ . A 12 V rail needs a gate near 17 V, which a 3.3 V pin cannot produce on its own.

The same logic governs a BJT through $V_{BE} = V_B - V_E$: with the emitter at ground (low-side) the base is easy to drive; lift the emitter toward V_+ and it is not.

Note

This one idea — “it is gate-to-*source*, so where is the source?” — is the seed of the entire low-side-versus-high-side story in Chapter 23, and the reason integrated “smart” high-side switches (Chapter 24) exist at all. If you remember nothing else from this chapter, remember this.

22.4 The BJT as a switch: saturation and overdrive

To use a BJT as an *amplifier* you keep it in its **active** region, where $I_C = \beta I_B$ holds. To use it as a *switch* you want the opposite: drive it so hard that it stops obeying that law. As you increase the base current, the drop across the main path V_{CE} falls; past a point it can fall no further and

sticks at $V_{CE(\text{sat})} \approx 0.1\text{ V}$ to 0.3 V . The transistor is now **saturated** — it behaves like a closed switch with a small fixed drop, and pushing yet more base current barely changes I_C .

A switch wants to be *solidly* saturated, with margin against the wide part-to-part and temperature spread in β . So you deliberately **overdrive** the base: instead of the bare minimum $I_B = I_C/\beta$, you design for a much smaller, made-up gain — a “**forced** β ” of 10 to 20 — i.e. you feed roughly $I_C/15$ of base current even though the device’s real β might be 100. That is precisely what the phrase “a forced β of 10–20” means whenever you meet it; it is the number that sets the base resistor in Chapter 25.

22.5 Making a voltage above the rail

The previous section left a loose end: a high-side N-channel switch needs its gate *above* the supply rail, and that voltage does not exist anywhere on the board. So you *make* one. A **charge pump** (or a **bootstrap** capacitor) is a small arrangement of switches and a capacitor that charges the cap and then stacks it on top of the rail, producing a node a few volts *above* V_+ — enough to fully enhance the high-side gate.

Note

As a firmware engineer you almost never build this yourself; you just need to know the block exists. It is exactly what lets an integrated high-side switch (Chapter 24) turn on an N-channel device from your humble logic pin.

22.6 Why switching wastes energy

A real switch dissipates power $P = V \cdot I$ across its main path. Look at the two *steady* states:

- **Fully on:** the drop V across the switch is tiny ($I R_{DS(\text{on})}$ for a MOSFET, $V_{CE(\text{sat})}$ for a BJT), so $P = VI$ is small.
- **Fully off:** the current I through it is essentially zero, so $P = VI$ is essentially zero.

Neither steady state is where the energy goes. The loss happens *during the flip*: for the brief moment the switch is changing state, V has not yet collapsed *and* I has already risen — both are appreciable at once and $P = VI$ spikes (Figure 22.2). Each transition therefore burns a packet of energy of order $\frac{1}{2}VI t_{\text{sw}}$; do it f times a second and the average **switching loss** is $\sim \frac{1}{2}VI t_{\text{sw}} f$.

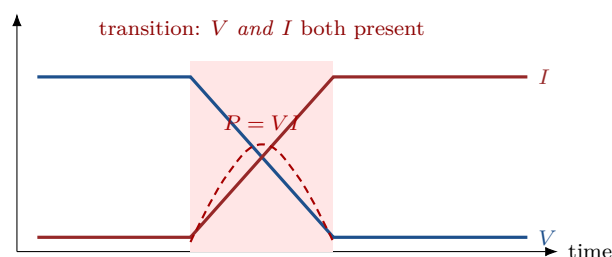


Figure 22.2: Where switching loss comes from. Fully on $V \approx 0$; fully off $I \approx 0$ — so $P = VI$ is negligible in both steady states. Only *during the edge* are V and I simultaneously appreciable, and P spikes (shaded). Faster edges shrink that window (less loss) but ring and radiate (EMI) — the trade-off behind Chapters 24 and 25.

Two consequences you will meet again: faster edges (shorter t_{sw}) waste less energy per flip but ring and radiate (EMI); and switching loss grows with frequency, so it dominates at high PWM rates. That is the *slew-rate* knob of Chapter 24 and the loss budget of Chapter 25.

22.7 Heat: Ohm’s law for temperature

Where does the dissipated power go? Into heat, which raises the silicon’s temperature. The bookkeeping is **Ohm’s law again**, with the quantities renamed:

$$\underbrace{\Delta T}_{\text{like } V} = \underbrace{P}_{\text{like } I} \times \underbrace{R_{th}}_{\text{like } R}.$$

Power plays the role of current, **thermal resistance** R_{th} (in °C/W) the role of resistance, and the temperature *rise* the role of voltage. So the junction settles at

$$T_j = T_{amb} + P R_{th(j-a)},$$

and keeping T_j under the device’s $T_{j(max)}$ is what decides whether you need a bigger device, a heatsink (a lower R_{th}), or more copper area. Chapter 25 turns this into numbers.

Key Idea

The six ideas this part stands on. (1) A switching transistor is a *valve*: the gate/base commands it, the drain–source/collector–emitter carries the load. (2) A MOSFET is *voltage*-operated (free to hold on); a BJT is *current*-operated (costs base current). (3) A MOSFET turns on by gate-to-*source* voltage $\geq V_{th}$, so where the source sits makes low-side easy and high-side hard. (4) A BJT *switch* is overdriven into saturation (a low “forced β ”). (5) A charge pump makes the above-rail gate voltage a high-side N-FET needs. (6) Loss happens during the *transition* (a $P = VI$ spike), and heat follows $\Delta T = P R_{th}$. From here: *where* to put the switch (Ch. 23), the chip that does high-side for you (Ch. 24), the loss/thermal/base-resistor numbers (Ch. 25), and driving a load both ways (Ch. 26).

Chapter 23

Load Switches: Low-Side and High-Side

Firmware eventually has to *do* something in the physical world — turn on a motor, a heater, a solenoid, a string of LEDs. A microcontroller pin cannot do this directly: a GPIO sources a few milliamps at 3.3 V, while these loads want amperes, often at a higher voltage. The bridge is a **load switch**: a transistor the GPIO controls, which in turn switches the real current. Where you put that switch — between the load and ground (*low-side*) or between the supply and the load (*high-side*) — is the first design decision, and it has real consequences. (The device physics behind every claim here — what opens a MOSFET, what saturation means, why a high-side gate is hard — was built up in Chapter 22.)

23.1 Why a transistor at all

A GPIO is a logic output, not a power source. Driving a 1 A motor from a 20 mA pin would either do nothing or destroy the pin. So the GPIO instead controls the *gate* (MOSFET) or *base* (BJT) of a transistor, which acts as a controlled switch in the high-current path. The pin spends almost no current; the transistor carries the load.

23.2 Low-side switch

In a **low-side** switch the transistor sits *between the load and ground*. The load's top terminal is permanently tied to V_+ ; the transistor connects (or disconnects) the load's bottom terminal to ground.

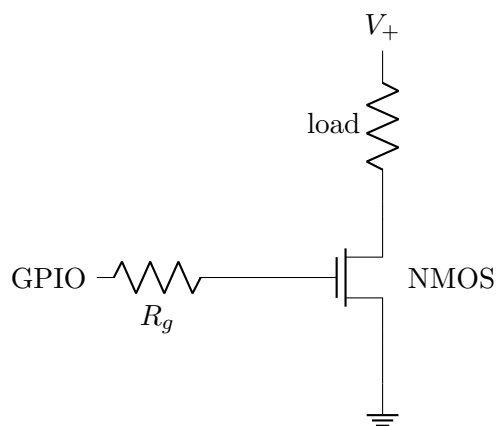


Figure 23.1: Low-side NMOS switch. The load hangs from V_+ ; the NMOS pulls its return to ground when the gate is driven high. The gate reference (ground) is the *same* as the MCU's, so the GPIO drives it directly.

Why it is easy. An NMOS turns on when its gate is above its source by the threshold V_{th} . In the low-side position the source is at *ground* — the same reference as the MCU — so a 3.3 V GPIO directly provides the needed V_{GS} . No level shifting, no charge pump. The same logic makes an NPN BJT (base referenced to ground) easy to drive here.

The catch. When the switch is off, the load’s bottom is floating and its top is still at V_+ — the load is *not* fully de-energised, it just has no return path. For a motor or heater this is usually fine, but it means the load’s wiring sits at V_+ even when “off”, which can be a safety or fault-tolerance concern (a short from the load to ground would energise it without the MCU’s consent).

23.3 High-side switch

A **high-side** switch sits *between* V_+ and the load; the load’s bottom is permanently grounded. When on, the load sees the full supply; when off, the load is at 0 V — genuinely disconnected from power.

Why it is wanted. The load is *truly off* (zero volts across it), which is safer and is mandatory in many systems — automotive especially, where a low-side fault to chassis ground could turn something on unexpectedly. High-side switching puts the controlled disconnect on the *live* side.

Why it is hard. To turn on an NMOS high-side switch, its *source* is now at the load (near V_+ when conducting), so the gate must be driven *above* V_+ — e.g. 12 V rail needs a gate at ~ 22 V. A 3.3 V GPIO cannot do that. You need a **gate driver** with a *charge pump* (to generate a voltage above the rail) or a level shifter — or you use a P-channel device (driven *below* V_+ , easier but with worse on-resistance and cost). This difficulty is exactly why integrated high-side drivers exist (Chapter 24).

23.4 NPN BJT as a low-side switch, in detail

The BJT version is worth a close look because the syllabus asks for it and because base-current sizing (Chapter 25) is a classic exam question. An NPN in the low-side position:

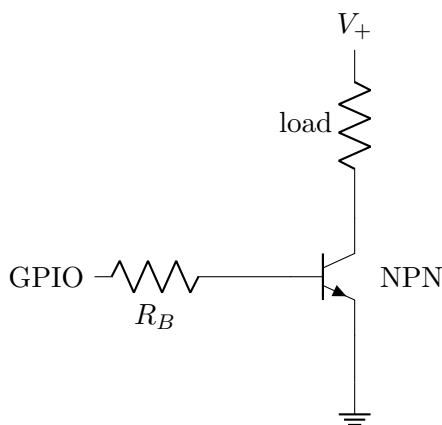


Figure 23.2: Low-side NPN switch. The base current set by R_B drives the transistor hard into *saturation* so V_{CE} is small; the collector carries the load current, the emitter returns to ground.

To use a BJT as a *switch* (not an amplifier) you drive it into **saturation**: supply enough base current that the collector-emitter voltage $V_{CE(\text{sat})}$ collapses to a few tenths of a volt, so it behaves like a closed switch with a small drop. “Enough” means more than I_C/β — in practice you deliberately *overdrive* with a *forced* β of 10–20 (well below the device’s nominal β) to guarantee deep saturation. The base resistor R_B sets that base current, and sizing it is the subject of Chapter 25.

Watch Out

For any *inductive* load (motor, solenoid, relay), switching it off makes the inductor try to keep current flowing, generating a large reverse voltage spike ($v = L di/dt$) that will destroy the transistor. You *must* add a **flyback diode** across the load (or a MOSFET with an adequate avalanche rating) to give that current a safe path. Forgetting the flyback diode is one of the most common — and most destructive — beginner mistakes in power switching.

Table 23.1: Low-side versus high-side switching.

	Low-side	High-side
Switch is between	load and GND	V_+ and load
Load when off	floating, top still at V_+	at 0 V, fully off
Gate/base drive	easy (referenced to GND)	hard (must exceed V_+)
Extra circuitry	none	charge pump / gate driver / PMOS
Safety	a load-to-GND fault energises it	inherently safer
Typical use	simple on/off of grounded loads	automotive, safety-critical

Worked Example 23.1 Picking a side for a 12 V water pump

You must switch a 12 V, 2 A pump from a 3.3 V MCU. *Low-side* is the quick answer: one logic-level NMOS, gate straight to the GPIO (with a small series resistor and a gate pull-down), plus a flyback diode for the pump’s inductance. It works and is cheap. *But* if safety rules require the pump to be provably de-energised when off (so a chassis short cannot run it), you need *high-side* switching — and then a discrete NMOS needs a gate driver with a charge pump, which is fiddly enough that you would reach for an integrated high-side switch instead (Chapter 24). The electrical job is identical; the safety requirement chooses the topology.

Exercise 23.1

A designer uses a low-side NMOS to switch a grounded 24 V heater and is happy with how trivially the 3.3 V GPIO drives the gate. Their reviewer insists it be changed to high-side. (a) Give the safety argument the reviewer is making. (b) Explain why the gate drive that was trivial now becomes the hard part, and name the circuit block that solves it.

Solution

(a) With low-side switching the heater element sits at 24 V even when “off” (only its return is open). If the element shorts to ground anywhere, it energises with no way for the MCU to stop it — a fire/safety hazard. High-side switching keeps the element at 0 V when off, so it is genuinely de-energised. (b) Low-side was easy because the NMOS source was at ground, so a 3.3 V gate exceeded V_{th} . High-side puts the NMOS source up near 24 V when conducting, so the gate must be driven *above* 24 V — impossible from a 3.3 V pin. The block that solves it is a gate driver with a **charge pump** (or bootstrap) that generates a gate voltage above the rail.

Key Idea

Chapter takeaway. A load switch is a transistor the GPIO controls. *Low-side* (switch to ground) is easy to drive but leaves the load at V_+ when off; *high-side* (switch from V_+) truly de-energises the load but needs a gate voltage above the rail (charge pump / driver). Drive switching transistors into saturation/full-on, and always add a flyback diode for inductive loads.

Chapter 24

Integrated High-Side Drivers

Chapter 23 ended on a problem: a discrete high-side NMOS needs a gate driven above the supply rail, plus protection, plus diagnostics — a lot of fiddly analog circuitry around one switch. The industry’s answer is the **integrated high-side driver** (a “smart high-side switch”): a single chip that contains the power transistor, the gate-drive circuitry to turn it on properly, and a suite of protection and diagnostic features. For a firmware engineer these chips are a delight — you toggle a logic pin and get a robust, self-protecting power switch.

24.1 What is inside

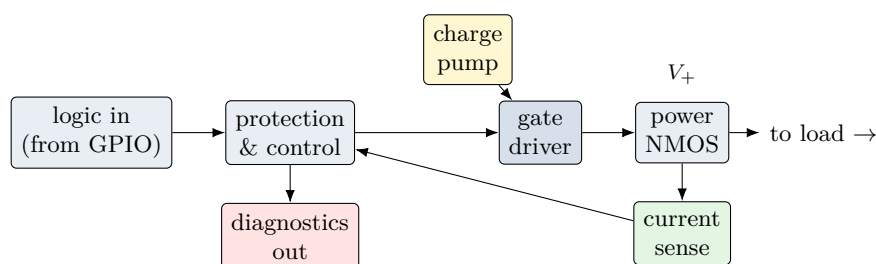


Figure 24.1: Inside a smart high-side switch: an internal charge pump lets the gate driver fully enhance the high-side NMOS; a current-sense element feeds protection logic that limits current, shuts down on faults, and reports status back to the MCU.

24.2 The features you get for free

Charge pump + gate driver.

Solves the “gate above the rail” problem of Chapter 23 internally, so an N-channel high-side switch turns fully on. You never see the high voltage.

Current limiting.

The chip measures its own output current and caps it, protecting itself and the wiring from a short or an inrush surge instead of letting the current run away.

Over-temperature shutdown.

An on-die thermal sensor turns the switch off if it gets too hot, then recovers — self-preservation no discrete FET offers.

Short-circuit & open-load detection.

Reports a shorted output (over- current) and, when off or on, a disconnected/broken load — valuable for diagnostics in the field.

Slew-rate (turn-on/turn-off speed) control.

The chip shapes how fast the output edge rises and falls (next section).

Diagnostic feedback / current sense.

A status pin or a current-mirror output lets firmware *read back* the switch's health and even the load current — closing the loop between the power stage and your code.

24.3 Why turn-on/turn-off speed matters

Controlling the switching edge *speed* (slew rate) is a genuine engineering trade-off, not a luxury:

- **Switch faster** and you spend less time in the lossy transition region (less *switching loss*, Chapter 25). But fast edges (high dV/dt and di/dt) ring against parasitic inductance and capacitance, radiating **electromagnetic interference (EMI)** and producing voltage overshoot that can exceed ratings or fail EMC tests.
- **Switch slower** and you tame EMI and overshoot, but dissipate more energy in each transition (more heat) and limit your maximum switching frequency.

An integrated driver lets you (or the chip) pick a *controlled* slew rate that balances efficiency against EMI — often a single resistor or register setting. This is why “turn-on/turn-off speed control” is listed as a headline feature: it is the knob that trades heat for noise.

Key Idea

A smart high-side switch turns a hard analog problem (drive a gate above the rail, protect against shorts and heat, sense current, manage EMI) into a digital one (toggle a pin, read a status bit). It internalises the charge pump and adds current limiting, thermal and short-circuit protection, open-load detection, and slew-rate control — robustness that is impractical to build discretely around every switch.

Worked Example 24.1 Why the smart switch survived what killed the discrete one

Two designs switch the same 12 V lamp. Design A uses a bare NMOS plus a home-made high-side gate driver. Design B uses an integrated high-side switch. A cold lamp filament is nearly a short, so at turn-on it draws a huge *inrush* current. Design A's NMOS sees the full inrush and, with no current limit, can exceed its safe operating area and fail. Design B's internal current limit clamps the inrush to a safe value and, if the surge persists, the thermal and short-circuit protection fold the current back or latch off — then it reports the fault on its diagnostic pin so firmware can retry or flag a bad lamp. Same load, but B's built-in protection turns a destructive event into a logged, survivable one.

Exercise 24.1

Your product fails its EMC (radiated-emissions) test, with the worst emissions at harmonics traceable to a high-side switch's turn-on edge. The integrated driver has a slew-rate control input. (a) Which way do you adjust the slew rate, and why does it help? (b) What does that adjustment cost you, and where will you see the cost?

Solution

(a) *Slow the edge down* (lower slew rate). EMI is driven by fast $dV/dt/di/dt$ ringing against parasitics; gentler edges have less high-frequency content, lowering the radiated harmonics so you pass the test. (b) Slower switching means the transistor spends more time in the partially-on transition region where both voltage and current are appreciable, so **switching**

losses (and thus heat) rise — you will see it as higher device temperature, and it limits how high you can push the switching frequency. Slew control trades EMI against switching loss; you tune to pass EMC while staying within the thermal budget (Chapter 25).

Key Idea

Chapter takeaway. The integrated high-side driver packages the power FET, an internal charge pump and gate driver, current limiting, thermal and short-circuit protection, open-load and diagnostics, and slew-rate control into one chip — letting firmware command robust high-side power with a logic pin and read its health back. Slew-rate control is the deliberate EMI-versus-loss knob.

Switch Power Dissipation and Base-Resistor Sizing

A switch is never a *perfect* switch: it drops a little voltage while carrying current, so it dissipates power and heats up. Get the numbers wrong and the transistor cooks. This chapter shows where the power goes in a MOSFET and a BJT, how to check it against the thermal budget, and — the syllabus centrepiece — how to size the base resistor of an NPN switch (and why a GPIO sometimes cannot drive it at all).

25.1 Where the power goes in a MOSFET

A MOSFET switch dissipates power in two distinct ways:

Conduction loss

while fully on, it looks like a small resistor $R_{DS(on)}$, so it burns

$$P_{\text{cond}} = I_{\text{load}}^2 R_{DS(on)}.$$

This dominates at high current and/or low switching frequency.

Switching loss

during each turn-on/turn-off the device passes through a region where it has *both* appreciable voltage and current simultaneously. Per transition the energy is roughly $\frac{1}{2} V I t_{\text{sw}}$, so at switching frequency f :

$$P_{\text{sw}} \approx \frac{1}{2} V I (t_{\text{on}} + t_{\text{off}}) f.$$

This dominates at high frequency — and it is exactly what slew-rate control (Chapter 24) trades against EMI.

25.2 Where the power goes in a BJT

A saturated BJT switch drops $V_{CE(\text{sat})}$ (typically 0.1 V to 0.3 V) while carrying the collector current, plus a smaller base-drive loss:

$$P_{\text{BJT}} \approx V_{CE(\text{sat})} I_C + V_{BE} I_B.$$

Note the contrast with the MOSFET: the BJT's conduction drop is a roughly *constant voltage* ($V_{CE(\text{sat})}$), whereas the MOSFET's is *resistive* ($I \cdot R_{DS(on)}$). At low currents the MOSFET (tiny $I R_{DS}$) usually wins; the BJT also needs continuous base current to stay on, which the MOSFET (voltage-controlled) does not.

25.3 The thermal check

Dissipated power raises the junction temperature above ambient through the thermal resistance $R_{th(j-a)}$ (junction-to-ambient, in $^{\circ}\text{C}/\text{W}$):

$$T_j = T_{\text{amb}} + P_{\text{total}} R_{th(j-a)} < T_{j(\text{max})}.$$

This inequality decides whether you need a bigger device, a heatsink (lower R_{th}), or more copper area. It is the bridge between the electrical numbers above and physical survival.

Worked Example 25.1 MOSFET dissipation and temperature

A logic-level NMOS switches 3 A continuously, with $R_{DS(\text{on})} = 40 \text{ m}\Omega$, at a low frequency so switching loss is negligible. Conduction loss: $P = I^2 R = 3^2 \times 0.040 = 0.36 \text{ W}$. With a modest $R_{th(j-a)} = 60 ^{\circ}\text{C}/\text{W}$ (no heatsink) and $T_{\text{amb}} = 25 ^{\circ}\text{C}$: $T_j = 25 + 0.36 \times 60 = 46.6 ^{\circ}\text{C}$ — comfortably below a $150 ^{\circ}\text{C}$ limit. Double the current to 6 A and conduction loss *quadruples* to 1.44 W ($T_j = 111 ^{\circ}\text{C}$): the I^2 dependence means current, not voltage, is what bites you.

25.4 Sizing an NPN base resistor

This is the canonical design calculation. To use an NPN as a *switch* it must be in **saturation**, which requires enough base current. The recipe:

1. **Find the load (collector) current** I_C — set by the load and supply.
2. **Choose a forced beta** β_{forced} , deliberately low (10 to 20) to *overdrive* the base and guarantee deep saturation despite the device's minimum β and temperature. Then the required base current is

$$I_B = \frac{I_C}{\beta_{\text{forced}}}.$$

3. **Size the resistor** from the drive voltage. The GPIO drives V_{drive} (e.g. 3.3 V); $V_{BE(\text{sat})} \approx 0.7 \text{ V}$ appears base-to-emitter; the rest is across R_B :

$$R_B = \frac{V_{\text{drive}} - V_{BE(\text{sat})}}{I_B}.$$

Worked Example 25.2 Driving a 0.5 A relay coil with an NPN

$I_C = 0.5 \text{ A}$, drive from a 3.3 V GPIO, choose $\beta_{\text{forced}} = 10$. Base current: $I_B = 0.5/10 = 50 \text{ mA}$. Resistor: $R_B = (3.3 - 0.7)/0.05 = 2.6/0.05 = 52 \Omega$ (pick the next standard value, 47Ω , to slightly *overdrive* — safer for saturation).

Now the reality check that catches everyone: this demands 50 mA *out of the GPIO pin*. A typical STM32 pin sources only 8 mA to 20 mA, so the pin *cannot* supply 50 mA — it would sag, the BJT would leave saturation, and both pin and transistor could overheat. The fix: either use a higher- β arrangement (a Darlington, or raise β_{forced} if the device's guaranteed β permits), or — far better — switch with a *logic-level MOSFET*, which is voltage-controlled and draws essentially *zero* steady gate current. This is a big reason MOSFETs have largely displaced BJTs for switching.

Watch Out

Always verify that the GPIO can actually *source* (or sink) the base current your sizing demands, against the pin's datasheet limit *and* the whole-port/whole-chip aggregate limit. A base-resistor calculation that ignores the pin's drive capability is a classic exam trap — and a classic blown pin.

Exercise 25.1

You must switch a 1 A load with an NPN whose guaranteed minimum β at 1 A is 30, driven from a 3.3 V, 12 mA-max GPIO ($V_{BE(\text{sat})} = 0.8$ V). (a) Using $\beta_{\text{forced}} = 20$, compute I_B and R_B . (b) Is the pin able to drive it? (c) State the better solution and one sentence on why.

Solution

(a) $I_B = I_C/\beta_{\text{forced}} = 1/20 = 50$ mA; $R_B = (3.3 - 0.8)/0.05 = 2.5/0.05 = 50$ Ω . (b) **No** — it needs 50 mA but the pin sources at most 12 mA. (Even using the full $\beta = 30$, $I_B = 33$ mA still exceeds 12 mA.) (c) Use a **logic-level MOSFET** (or a gate-driver/Darlington stage): a MOSFET is voltage-controlled and needs only a tiny transient gate current, so the weak GPIO can fully turn it on and the steady drive current is essentially zero.

Key Idea

Chapter takeaway. MOSFET loss = $I^2 R_{DS(\text{on})}$ (conduction, $\propto I^2$) + $\frac{1}{2} V I t_{\text{sw}} f$ (switching); BJT loss $\approx V_{CE(\text{sat})} I_C$ plus base drive. Check $T_j = T_{\text{amb}} + P R_{th} < T_{j(\text{max})}$. Size an NPN base resistor from $I_B = I_C/\beta_{\text{forced}}$ and $R_B = (V_{\text{drive}} - V_{BE})/I_B$ — then *verify the GPIO can supply I_B* , or use a MOSFET.

Chapter 26

Full-Bridges and Shoot-Through

A single switch turns a load on and off. To drive a DC motor *both ways* (forwards and backwards), or to generate an AC waveform, you need the **full-bridge** (H-bridge): four switches that can apply the supply across a load in *either* polarity. The H-bridge is everywhere — motor drivers, inverters, class-D audio — and it carries one spectacular failure mode, **shoot-through**, that every firmware engineer driving one must understand and design against.

26.1 Topology and operation

Four switches form two “legs” (half-bridges) between V_+ and ground, with the load bridging the two leg midpoints — the shape gives the “H”.

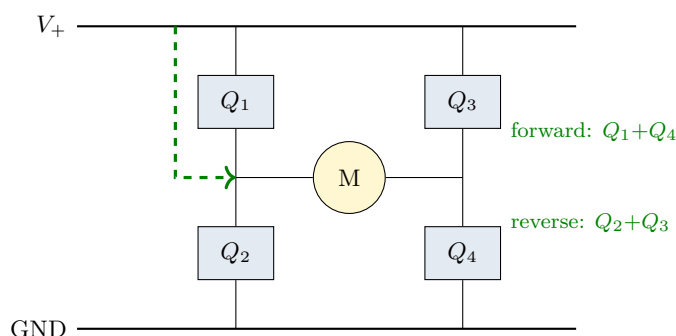


Figure 26.1: Full-bridge (H-bridge). Turning on the diagonal Q_1+Q_4 pushes current left-to-right through the motor; the diagonal Q_2+Q_3 reverses it. PWM on the active switches sets the speed.

- **Forward:** Q_1 (top-left) and Q_4 (bottom-right) on $\rightarrow$ current flows $V_+ \rightarrow Q_1 \rightarrow$ motor $\rightarrow Q_4 \rightarrow$ GND (left to right).
- **Reverse:** Q_2 and Q_3 on $\rightarrow$ current flows the other way.
- **Speed:** PWM (Chapter 16) the active switches; the duty cycle sets the average voltage, hence speed.
- **Brake:** turn on both *low* switches (or both high) to short the motor terminals, dissipating its kinetic energy.

26.2 Shoot-through: the cardinal sin

Look at one leg — say Q_1 (top) and Q_2 (bottom). They must *never* be on at the same time, because that would connect V_+ straight to ground through the two transistors, with only their tiny on-resistance to limit the current. The result is an enormous current spike — **shoot-through** (or cross-conduction) — that destroys the transistors in microseconds.

Watch Out

Shoot-through does not require a logic mistake. It happens during *normal commutation* because turning a switch off is *not instantaneous*: gate charge must be removed, a BJT has storage time, gate-driver propagation adds delay. If you command “ Q_1 off, Q_2 on” at the *same instant*, Q_1 is still partly conducting when Q_2 turns on — and for that overlap both conduct and the leg shorts the rail. The faster you switch, the more this matters.

26.3 The fix: dead-time

The defence is **dead-time**: a deliberate interval, inserted between turning one switch in a leg off and the complementary switch on, during which *both are off*. Make the dead-time longer than the worst-case turn-off delay and the outgoing switch is fully off before the incoming one starts to conduct, so there is never an overlap.

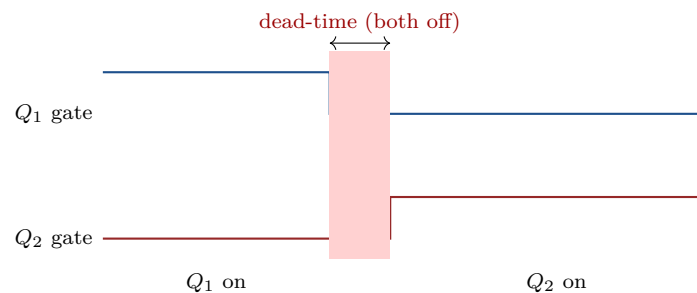


Figure 26.2: Complementary PWM with dead-time. Q_1 turns off, then after a guard interval (both off) Q_2 turns on — so the two never conduct together. The band must exceed the worst-case turn-off delay.

Dead-time is implemented in one of two places:

Timer-based

STM32 *advanced-control* timers (TIM1, TIM8) generate **complementary** PWM outputs with a built-in **dead-time generator**: you program a dead-time value and the hardware guarantees the gap on every edge, with no CPU involvement. This is the usual route for motor control.

Driver-based

many gate-driver and integrated bridge ICs include **cross-conduction prevention**: they internally enforce a minimum dead-time (and may ignore an illegal “both on” command), protecting the bridge even if the PWM signals are imperfect.

Key Idea

A full-bridge can short its supply through one leg if the high and low switches ever conduct together (shoot-through). Because switches turn off in finite time, you must insert **dead-time** — a both-off guard interval at every commutation — using the timer’s dead-time generator or a driver IC’s built-in protection. Size it above the worst-case turn-off delay.

Worked Example 26.1 Sizing the dead-time

A bridge uses MOSFETs whose worst-case turn-off delay (including gate-driver propagation) is 200 ns, switching at 20 kHz. You set a dead-time of, say, 300 ns — comfortably above

200 ns for margin. The cost: for 300 ns of each 50 μ s PWM period (the 1/20 kHz period) the output is uncontrolled (both off), i.e. about 0.6 % of the period — a small distortion of the output voltage. Set it *too long* and you waste more of the period and distort the waveform (and lose efficiency); *too short* and you risk shoot-through. The sweet spot is “just longer than the worst-case turn-off,” which is why fast, well-characterised switches let you use less dead-time.

Exercise 26.1

A student drives an H-bridge from two independent GPIOs per leg, writing the high and low gate signals in software “at the same time” in their control loop. The MOSFETs run hot and occasionally fail, even though the logic “never sets both high”. (a) Explain how shoot-through still occurs. (b) Give the proper hardware mechanism that prevents it and say why it is superior to fixing the software timing.

Solution

(a) Even if the *commanded* states are never “both high”, the transition is not instantaneous: when the loop flips a leg, the switch being turned off has a finite turn-off delay (gate discharge, propagation), so it is still partly on when the other switch turns on — a brief overlap that shorts the rail on every commutation, heating and eventually destroying the devices. Software cannot reliably insert sub-microsecond, jitter-free gaps. (b) Use **complementary PWM with a hardware dead-time generator** (an STM32 advanced timer) or a gate driver with built-in cross-conduction prevention: the hardware guarantees a both-off guard interval on every edge, deterministically and without CPU jitter — something software timing in a control loop cannot match.

Key Idea

Chapter takeaway. The H-bridge applies the supply across a load in either direction with four switches in two legs, enabling forward/reverse, PWM speed control, and braking. Its mortal danger is shoot-through — both switches in a leg conducting during the finite turn-off time. Prevent it with hardware dead-time (advanced-timer dead-time generator or a protective gate driver), sized just above the worst-case turn-off delay.

Part VIII

Laboratories

The chapters in this part are *lab companions*: they explain, at the level of *what is happening* and *why*, the firmware exercises run on the course's STM32L496 board (STM32CubeIDE, HAL/LL, FreeRTOS via CMSIS-OS v2). They are deliberately **not** a copy of any submitted solution — every code block is a *suggested skeleton*: the bare-minimum prototypes and structure you fill in. Each lab leans on the theory chapters: interrupts (Ch. 2), the timer (Ch. 16), HAL vs LL and callbacks (Ch. 15), FreeRTOS tasks/queues/events (Ch. 17), the NTC divider (Ch. 19), fixed-point arithmetic (Ch. 18) and the serial buses (Ch. 21).

Note

A small *board-support* module (call it `io`) hides the pin wiring behind names: `led0..3` for the LEDs, `getSwitch0..7` for the DIP switches, and `lcdWriteDigit/lcdUpdateDisplay` for the 4-digit display. The labs call that layer rather than poking GPIO registers directly.

Chapter 27

Lab: Digital I/O and Timers (HAL vs. LL)

What is happening. A single FreeRTOS task polls the DIP switches and, for whichever switch is held, drives an output pin in a different way. The point is to *compare* the techniques on a scope: software toggling through HAL, the same through LL, and a pin driven by a *timer* so the hardware — not the CPU — sets the timing. It is the concrete payoff of Chapters 15 and 16.

Key Idea

HAL and LL are two libraries for the same silicon: HAL is portable and safe but heavy (layers of dispatch); LL is a thin wrapper over the registers — fast, with nothing between you and the peripheral. Toggling a pin both ways and measuring the edge rate makes the difference visible.

The dispatcher reads the board and selects a behaviour:

```
1 void StartDefaultTask(void *argument) {
2     for (;;) {
3         if (getSwitch0()) { // HAL: portable, slower
4             while (getSwitch0())
5                 HAL_GPIO_TogglePin(IOA_Port, IOA_Pin);
6         } else if (getSwitch1()) { // LL: register-near, fast
7             while (getSwitch1())
8                 LL_GPIO_TogglePin(IOB_Port, IOB_Pin);
9         } else if (getSwitch2()) { // hardware-timed pulse
10             start_timer_pulse(); // see below
11             while (getSwitch2()) { /* timer + ISR do the work */ }
12             stop_timer_pulse();
13         }
14         osDelay(1);
15     }
16 }
```

Listing 27.1: Suggested skeleton: switch-dispatched I/O task.

Timer-driven output. Instead of busy-waiting, configure the timer's period and let its update interrupt toggle the pin. A callback can be attached at run time (HAL) or simply by defining the weak IRQ handler (Ch. 15):

```
1 // period from prescaler/ARR (Ch. timer); see the master formula there
2 static void start_timer_pulse(void) {
3     HAL_TIM_RegisterCallback(&htim, HAL_TIM_PERIOD_ELAPSED_CB_ID, on_tim_period);
4     HAL_TIM_Base_Start_IT(&htim);
5 }
6 void on_tim_period(TIM_HandleTypeDef *h) { // runs in ISR context
7     HAL_GPIO_TogglePin(IOA_Port, IOA_Pin); // jitter-free, CPU-free timing
8 }
```

Listing 27.2: Suggested skeleton: timer period callback.

Note

Busy-wait toggling shows the *raw* maximum pin rate (and how much faster LL is than HAL); the timer path shows *precise, low-jitter* timing that does not depend on what the CPU is doing. Different tools for different jobs.

Chapter 28

Lab: An NTC Thermometer

What is happening. A periodic timer kicks off a measurement; an *acquisition* task reads the thermistor through the ADC, converts the reading to a temperature, and hands it to a *display* task that writes the LCD. It is the canonical multi-task pattern of Chapter 17 wrapped around the NTC divider of Chapter 19.

Key Idea

Keep the interrupt and the work apart. The timer ISR only *signals* (an event flag); the acquisition task does the slow ADC work; a *queue* carries the result to the display. **Event flags signal that something happened; queues carry data.**

Data flow. timer ISR → FLAG_MEASURE → acquisition task → start ADC(s) in interrupt mode → conversion-complete ISR sets FLAG_DRDY → compute T → display queue → display task.

```
1 void StartAcquisitionTask(void *argument) {
2     for (;;) {
3         osEventFlagsWait(measureFlag, FLAG_MEASURE, osFlagsWaitAny, osWaitForever
4             );
5         HAL_ADC_Start_IT(&hadc_ntc);           // V across the NTC
6         HAL_ADC_Start_IT(&hadc_ref);           // a divided copy of Vcc
7         // wait for BOTH conversions (rendezvous)
8         osEventFlagsWait(drdyFlag, FLAG_NTC | FLAG_REF, osFlagsWaitAll,
9             osWaitForever);
10
11         uint32_t v_ntc = adc_ntc;
12         uint32_t v_cc  = adc_ref * 2;           // undo the /2 divider on Vcc
13         uint32_t r_ntc = (v_ntc * R_REF) / (v_cc - v_ntc); // divider ->
14             resistance
15         uint16_t t_dK  = GetTemperature(r_ntc); // tenths of a
16             kelvin
17
18         osMessageQueuePut(displayQueue, &t_dK, 0, osWaitForever);
19     }
20 }
```

Listing 28.1: Suggested skeleton: acquisition task with an event-flag handshake.

The conversion-complete callback is shared by both ADCs and just records the value and raises the matching flag:

```
1 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *h) {
2     if (h->Instance == ADC_NTC) { adc_ntc = HAL_ADC_GetValue(h); osEventFlagsSet(
3         drdyFlag, FLAG_NTC); }
4     else { adc_ref = HAL_ADC_GetValue(h); osEventFlagsSet(
5         drdyFlag, FLAG_REF); }
6 }
```

Listing 28.2: Suggested skeleton: ADC complete callback.

Resistance to temperature. An NTC is non-linear, so rather than evaluate a logarithm on the MCU the lab uses a *lookup table* of resistance per degree and interpolates — deterministic and

cheap (Ch. 18). Note the table is *descending* (resistance falls as temperature rises), which inverts the usual binary-search comparison.

```

1 // table[i] = NTC resistance at i degrees C, strictly DECREASING
2 uint16_t GetTemperature(uint32_t r) {
3     // binary search on a descending table, then linear-interpolate
4     // return temperature in tenths of a kelvin: (deci-Celsius) + 2732
5     ...
6 }

```

Listing 28.3: Suggested skeleton: LUT lookup, returns tenths of a kelvin.

Key Idea

Work in **fixed point**: carry the temperature as tenths of a kelvin ($T_{\times 10} = 10 T_{\circ\text{C}} + 2732$). Integer math is exact and fast on a core whose FPU you may not want to pay for (Ch. 18); the display just splits the integer into digits.

```

1 void StartDisplayTask(void *argument) {
2     for (;;) {
3         uint16_t v; osMessageQueueGet(displayQueue, &v, NULL, osWaitForever);
4         lcdWriteDigit('0' + (v / 1000) % 10, 0);
5         lcdWriteDigit('0' + (v / 100) % 10, 1);
6         lcdWriteDigit('0' + (v / 10) % 10, 2);
7         lcdWriteDigit('0' + (v % 10), 3);
8         lcdUpdateDisplay();
9     }
10 }

```

Listing 28.4: Suggested skeleton: display task (4 digits, value in tenths of K).

Watch Out

The queues here are depth 1: “latest value wins”. That is what you want for a live reading — never a backlog of stale temperatures — but it means a slow consumer silently drops samples. Choose the depth on purpose.

Chapter 29

Lab: Closed-Loop Heater Control

What is happening. The thermometer gains an *actuator*. The dispatcher now fans the measured temperature out to *two* consumers — the display and a new *heater* task — and the heater task switches a resistive heating element on or off to hold a set-point. The element is driven through a load switch, exactly the kind of stage in Chapter 23.

```
1 osMessageQueueGet(acqQueue, &t_dK, NULL, osWaitForever);
2 osMessageQueuePut(displayQueue, &t_dK, 0, osWaitForever);
3 osMessageQueuePut(controlQueue, &t_dK, 0, osWaitForever);    // NEW: to the heater
```

Listing 29.1: Suggested skeleton: single producer, two consumers (fan-out).

The controller. The simplest closed loop is *bang-bang* (on/off) with *hysteresis*: turn the heater on below ($setpoint - h$), off above ($setpoint + h$). The dead-band h stops the output chattering when the temperature sits at the set-point.

```
1 #define SET_DK    3232    // 50.0 C, in tenths of a kelvin
2 #define HYST_DK   10     // +/- 1.0 K dead-band
3
4 void StartHeaterTask(void *argument) {
5     bool on = false;
6     heater_off();
7     for (;;) {
8         uint16_t t; osMessageQueueGet(controlQueue, &t, NULL, osWaitForever);
9         if (on && t > SET_DK + HYST_DK) { on = false; heater_off(); led0(OFF); }
10        else if (!on && t < SET_DK - HYST_DK) { on = true; heater_on(); led0(ON); }
11    }
12 }
```

Listing 29.2: Suggested skeleton: bang-bang heater task with hysteresis.

Key Idea

Hysteresis is the whole trick: with a single threshold the heater would switch thousands of times a second as noise crosses the line. The $\pm h$ band trades a little temperature ripple for a sane switching rate — and `led0` mirrors the heater so you can see it work.

Note

Bang-bang is the floor, not the ceiling. When overshoot or ripple matters you move to proportional or PID control — but the task/queue plumbing built here is exactly what a PID loop would reuse.

Chapter 30

Lab: An I²C Memory (Final Project)

What is happening. The board talks to an external I²C memory (an EEPROM-style device) through a tiny driver — `memInit`, `memRead`, `memWrite` — built on the HAL I²C API. This is Chapter 21 made concrete: a two-wire bus, a 7-bit device address, and read/write *transactions* framed by START/STOP.

```
1 // 7-bit device address; memory addressed by an internal byte/word pointer
2 int memInit (I2C_HandleTypeDef *bus, uint8_t dev_addr);
3 int memRead (uint16_t mem_addr, uint8_t *buf, size_t n);
4 int memWrite(uint16_t mem_addr, const uint8_t *buf, size_t n);
```

Listing 30.1: Suggested skeleton: the driver API.

The transaction. A read is: START, send (device address + write) and the memory pointer, a *repeated* START, send (device address + read), clock out *n* bytes ACK-ing all but the last, STOP. HAL packages this as a single “mem” transfer:

```
1 int memRead(uint16_t a, uint8_t *buf, size_t n) {
2     return HAL_I2C_Mem_Read (bus, dev<<1, a, I2C_MEMADD_SIZE_8BIT, buf, n,
3     TIMEOUT);
4 }
5 int memWrite(uint16_t a, const uint8_t *buf, size_t n) {
6     return HAL_I2C_Mem_Write(bus, dev<<1, a, I2C_MEMADD_SIZE_8BIT, (uint8_t*)buf,
7     n, TIMEOUT);
8 }
```

Listing 30.2: Suggested skeleton: read/write via HAL mem transfers.

Watch Out

Two classic traps. **Addressing:** HAL wants the address *left-shifted* by one (the low bit is R/ $\bar{W}$), so a device at 0x50 is passed as 0xA0. **Page writes:** an EEPROM writes in fixed-size pages and ignores data past a page boundary (it wraps), and after a write it is busy for a few milliseconds — poll for ACK (or wait the spec’d time) before the next access.

Exercise 30.1

A 256 B memory has 16-byte pages. You call `memWrite` with `mem_addr=10` and `n=20`. What goes wrong, and how do you fix it in the driver?

Solution

Bytes 10–15 fill the rest of page 0, then the device *wraps* within the same page and overwrites bytes 0–3 instead of advancing to page 1. Fix: have `memWrite` split the buffer at page boundaries, issuing one transfer per page and waiting for the write cycle (ACK polling) between them.

Appendix A

Syllabus Coverage Map

This book was built directly from the *Firmware Design* syllabus. The table below maps every syllabus item to the chapter that treats it, so you can navigate straight from the course outline to the relevant material.

Table A.1: Syllabus item to chapter.

Syllabus item	Chapter
Classic data-processing systems vs. embedded systems (HW & SW)	1
Review of microprocessor interrupt management	2
Executable code generation; GCC model; <code>gcc</code> CLI; <code>objdump</code>	3
Cross-compilation environments for embedded systems	4
Use of sections in object files and executables	5
Static vs. local allocation; optimisation; RISC vs. CISC	6
Debug hardware unit in microcontrollers	7
Dynamic memory allocation: C standard-library behaviour	8
C constructs to avoid (braces, recursion, <code>malloc</code> , indexes vs. pointers)	9
Dedicated memory areas for I/O buffers and DMA	10
Meaning of <code>const</code> and <code>volatile</code>	11
Mandatory content of <code>.c/.h</code> files; function prototyping	12
Peripheral representation in the memory map; register types; <code>typedef</code> structs	13
STM32CubeIDE: projects, deployment, debugging; FreeRTOS integration	14
HAL and LL libraries; interrupt management; callbacks	15
Use of “weak” symbols	15
Timer peripheral; prescaler and period	16
FreeRTOS best practices: tasks, queues, events	17
Fixed-point vs. floating-point in number crunching	18
Sensor interfacing; NTC thermistor; resistance-to-voltage	19
Digital-interface sensors; accelerometer; HDD free-fall	20
I ² C and SPI protocols; peripheral examples	21
Transistor-as-switch fundamentals (MOSFET/BJT, V_{GS} , saturation, charge pump, $P = VI$, thermal)	22
Load switches: low-side and high-side; NMOS and NPN BJT	23

Continued on next page

Table A.1: Syllabus item to chapter. (Continued)

Syllabus item	Chapter
Integrated high-side drivers (current limit, turn-on/off speed)	24
Switch power dissipation; NPN base-resistor sizing	25
Full-bridges and shoot-through; protection (timer/driver)	26
Lab: digital I/O & timers, HAL vs. LL, timer-driven pin	27
Lab: NTC thermometer; tasks/queues/events; ADC; fixed-point LUT	28
Lab: closed-loop heater; fan-out; bang-bang + hysteresis	29
Lab: I ² C memory (final project); addressing; transactions	30

Appendix B

Glossary of Acronyms

ABI	Application Binary Interface — the calling/layout conventions binaries must agree on (Ch. 4, 6).
ADC	Analog-to-Digital Converter (Ch. 19).
AAPCS	ARM Architecture Procedure Call Standard — the ARM ABI (Ch. 6).
BJT	Bipolar Junction Transistor (Ch. 23, 25).
BSS	“Block Started by Symbol” — zero-initialised data section (Ch. 5).
CCM	Core-Coupled Memory — fast RAM not on the DMA bus matrix (Ch. 10).
CISC	Complex Instruction Set Computer (Ch. 6).
CMSIS	Cortex Microcontroller Software Interface Standard (Ch. 13).
CPHA/CPOL	Clock Phase / Clock Polarity — SPI mode bits (Ch. 21).
DAP	Debug Access Port (Ch. 7).
DMA	Direct Memory Access (Ch. 10).
DWT	Data Watchpoint and Trace unit (Ch. 7).
ELF	Executable and Linkable Format (Ch. 3, 5).
EMI	Electromagnetic Interference (Ch. 24).
FPB	Flash Patch and Breakpoint unit (Ch. 7).
FPU	Floating-Point Unit (Ch. 18).
GPIO	General-Purpose Input/Output (Ch. 13).
HAL	Hardware Abstraction Layer (Ch. 15).
I2C	Inter-Integrated Circuit bus (Ch. 21).
ISR	Interrupt Service Routine (Ch. 2).
ITM	Instrumentation Trace Macrocell (Ch. 7).
JTAG	Joint Test Action Group debug interface (Ch. 7).
LL	Low-Layer driver library (Ch. 15).
LMA/VMA	Load / Virtual (run) Memory Address (Ch. 5).
MCU/MPU	Microcontroller / Microprocessor (also Memory-Protection Unit) (Ch. 1).

MEMS	Micro-Electro-Mechanical System (Ch. 20).
MISRA	Motor Industry Software Reliability Association — C coding standard (Ch. 9).
MMU	Memory-Management Unit (Ch. 1).
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor (Ch. 23).
NMI	Non-Maskable Interrupt (Ch. 2).
NTC	Negative Temperature Coefficient thermistor (Ch. 19).
NVIC	Nested Vectored Interrupt Controller (Ch. 2).
PWM	Pulse-Width Modulation (Ch. 16, 26).
RISC	Reduced Instruction Set Computer (Ch. 6).
RTOS	Real-Time Operating System (Ch. 17).
SPI	Serial Peripheral Interface (Ch. 21).
SRAM	Static Random-Access Memory (Ch. 1).
SWD/SWO	Serial Wire Debug / Serial Wire Output (Ch. 7).

End of the book. You now hold, in one volume, the whole arc from a `gcc` command line to a full-bridge dead-time. Go build something.